

stmlab + stmgui

The complete Igor STM setup in Python, with its panels

Igor's three panels, nine graph windows and two macros, rebuilt on top of the eight translated experiments — and the reasoning behind every control, every file, and every deviation from the original.

Read this first. This manual has three parts.

Chapter 05, the structure map, comes first and stands alone: the whole code on a few pages — what every file is, which file calls which function in which other file (extracted from the source, not written), one button click followed top to bottom, and one break-junction trace. Read it before anything else; return to it whenever a chapter loses you.

Part I, the GUI (chapters 40–51), is new to this folder. Chapter 40 is the tour; chapter 41 is the lab flow you know from Igor, button by button — Start Writing, Approach, Find Offset, Find Suppress, Start Measurement — and is the one to read before the first run. Chapters 42–45 are the panels and the graphs, control by control. Chapter 47 is what gets written to disk. Chapter 48 is how the code is built, for the day something needs changing. Chapters 50–51 are the quick reference and the complete binding table: every Igor global and where it lives now.

Part II, the core (chapters 00–30), is the stmlab manual: the platform, one chapter per experiment, the support systems and the command-line quick reference. It is unchanged from STMLAB_Manual.pdf; the GUI calls exactly this code. Chapter numbers in print are the file numbers in manual/, which is why they are not consecutive.

Everything runs against a simulated junction with `--simulate`. Every figure in Part I was produced by the GUI itself, on the simulator.

Trust order. The constant-bias path carries the reference tests and the 1 G_0 self-check on the simulator. The four ramp modes, both campaigns and the support modules are tested end to end on the simulator, through the GUI as well as the command line. Every nidaqmx call, the Keithley over GPIB, the second card and the electrochemical cell have **never met their instruments**. The GUI changes none of this: it calls the same functions, and is exactly as trustworthy as they are. Simulate first. Scope second. Tip last.

python_code_entire_igor_w_GUI/ · 3 panels, 9 graphs, 8 experiments · 121 tests · translated from igor_code/

Contents

I	Start here: the map of the code	10
5	The structure map: which file calls what	11
5.1	The shape, in one picture	11
5.2	Follow one click	13
5.3	Follow one trace	14
5.4	Every file	15
5.5	Who calls whom, file by file	18
5.5.1	gui.py	18
5.5.2	run.py	18
5.5.3	stmlab/main.py	18
5.5.4	stmlab/config.py	19
5.5.5	stmlab/safety.py	19
5.5.6	stmlab/instrument.py	19
5.5.7	stmlab/daq.py	19
5.5.8	stmlab/sim.py	19
5.5.9	stmlab/actuator.py	19
5.5.10	stmlab/approach.py	19
5.5.11	stmlab/trace.py	20
5.5.12	stmlab/ramps.py	20
5.5.13	stmlab/analysis.py	20
5.5.14	stmlab/storage.py	20
5.5.15	stmlab/calibrate.py	21
5.5.16	stmlab/keithley.py	21
5.5.17	stmlab/vzero.py	21
5.5.18	stmlab/echem.py	21
5.5.19	stmlab/xpiezo.py	21
5.5.20	stmlab/bringup.py	21
5.5.21	stmgui/app.py	22
5.5.22	stmgui/controller.py	22
5.5.23	stmgui/state.py	24
5.5.24	stmgui/widgets.py	24

5.5.25	stmgui/graphs.py	24
5.5.26	stmgui/panels/main_panel.py	24
5.5.27	stmgui/panels/voltage_offset.py	24
5.5.28	stmgui/panels/echem_panel.py	25
5.5.29	stmgui/panels/history.py	25
5.5.30	stmgui/panels/button_map.py	25
5.5.31	experiments/01_constant_bias/run_experiment.py	25
5.5.32	experiments/02_push_pull/run_experiment.py	25
5.5.33	experiments/03_iv_sweep/run_experiment.py	25
5.5.34	experiments/04_ac_hold/run_experiment.py	26
5.5.35	experiments/05_high_bias_hold/run_experiment.py	26
5.5.36	experiments/06_echem_gate_cv/run_experiment.py	26
5.5.37	experiments/07_lateral_monolayer/run_experiment.py	27
5.5.38	experiments/08_bias_series/run_experiment.py	27
5.6	How to use this chapter	28

II The GUI 29

40	The GUI: what it is and how to start it	30
40.1	What this folder is	30
40.2	Installing	31
40.3	Launch options	31
40.4	What opens, and why	31
40.5	The window map	32
40.6	What "simulate" swaps in	32
40.7	The menus, briefly	33
40.8	Stopping, and quitting	33
40.9	Every control says which Python it runs	33
40.10	The first five minutes on the simulator	34
40.11	Where to read next	35
41	The lab flow, button by button	36
41.1	Start Writing	36
41.2	Background Sampling (Beep is ON)	37
41.3	Start Approach	37
41.4	Find Offset (Find Zero)	38
41.5	Find Suppress	38
41.6	Start Measurement, and +1	39

41.7 Stop	40
41.8 Kill Tasks	40
41.9 A complete session on the simulator	40
41.10 Between runs	41
42 The main panel, control by control	42
42.1 The TabControl: DAQ, Inputs, Options	42
42.1.1 DAQ tab	42
42.1.2 Inputs tab	43
42.1.3 Options tab	44
42.2 Background Sampling (Beep is ON)	45
42.3 Keithley Controls	45
42.4 Actuator Controls	46
42.5 Piezo Controls	47
42.6 The readouts	47
42.7 Saved, Tip Bias, Stop #, Start Measurement, Attempts, +1	48
42.8 Controls Igor created but never showed	48
42.9 Tooltips, and the Button map	48
43 The Voltage Offset panel and V0 tracking	50
43.1 The panel	50
43.2 What Find Offset does	50
43.3 Why it is measured in contact	52
43.4 V0 check ON	52
43.5 The history is the diagnostic	53
43.6 Pitfalls	53
44 The Electrochemistry panel	55
44.1 Electrochemical Gating	55
44.2 Cyclic Voltammetry	56
44.2.1 The two buttons	56
44.2.2 The ramp and the keep mask	57
44.2.3 What the button does around the sweep	57
44.2.4 Reading the numbers	58
44.3 Between the two groups	58
45 The graph windows and the History window	59
45.1 How the windows get their data	59
45.2 HighRes	60

45.3 PullOutGvsE, PullOutLowG, AuAuConductanceLevel	60
45.4 SenseInDisplay	63
45.5 LogHistOfBlock	63
45.6 Izero and Izero_Time	64
45.7 CyclicVoltammogram	65
45.8 The History window	65
45.9 Placement and size	65
46 Menus and macros: rungo, LateralEXPT, the X piezo, config files	66
46.1 File	66
46.2 Macros	66
46.2.1 rungo(): the bias series	67
46.2.2 LateralEXPT(ZDistance, XDistance, XFreq): the monolayer walk	67
46.2.3 MoveXPiezo(nm) and MoveXPiezoToZero()	68
46.2.4 Stop (Alt)	68
46.3 Windows	68
46.4 Help	69
46.5 Keyboard	69
46.6 What the menus do not do	69
47 What the GUI writes, and how to read it back	70
47.1 Where	70
47.2 Session files	70
47.3 Reading a session file	72
47.4 Histogram files	72
47.5 CV files	72
47.6 What is not stored	73
47.7 Size	73
47.8 Comparing with an archived Igor block	73
48 How stmgui is built	74
48.1 The shape	74
48.2 state.py	74
48.3 controller.py	75
48.4 widgets.py	77
48.5 graphs.py	77
48.6 panels/	77
48.7 app.py	77

48.8 Threading rules, and what they prevent	78
48.9 Walkthrough: adding a control	78
48.10 Walkthrough: adding a button	78
48.11 Walkthrough: adding a graph	79
48.12 Tests	79
49 Deviations, limitations, troubleshooting	80
49.1 Deviations from Igor, and why	80
49.2 Limitations	81
49.3 Config warnings at Start Writing	82
49.4 Symptom, cause, fix	82
49.5 If the History shows a traceback	84
49.6 What to check before the first hardware run	84
50 GUI quick reference	86
50.1 The lab flow	86
50.2 Buttons and checkboxes	86
50.3 Events	88
50.4 Keys and menus	88
50.5 Files written	89
50.6 Commands	89
50.7 The numbers you change most	89
51 Appendix: the binding table	91
51.1 DAQ tab	91
51.2 Inputs tab	91
51.3 Options tab: Push-Pull	92
51.4 Options tab: IV	92
51.5 Options tab: AC	92
51.6 Options tab: High bias hold	93
51.7 Keithley Controls	93
51.8 Actuator and Piezo Controls	93
51.9 Bottom of the panel	93
51.10 Voltage_Offset panel	94
51.11 EChem panel	94
51.12 Readouts	94

III	The core: stmlab and the experiments	95
0	Overview	96
0.1	What this folder is	96
0.2	The experiment menu	97
0.3	Folder map	97
0.4	Quick start	98
0.5	Relationship to the neighbouring folders	98
1	Architecture	100
1.1	The module stack	100
1.2	The one-verb play() model	100
1.3	The tracked-piezo interlock	101
1.4	Clamp versus refuse	101
1.5	Raw volts in, raw volts stored	102
1.6	How ModeRamp + trace.capture generalise the validated path	102
1.7	The alignment spike, identical in every mode	102
2	Configuration reference	104
2.1	ChannelMap (cfg.channels)	104
2.2	SafetyLimits (cfg.limits)	105
2.3	RampConfig (cfg.ramp)	106
2.4	Calibration (cfg.cal)	107
2.5	ActuatorConfig (cfg.actuator)	108
2.6	Mode configs	109
2.6.1	PushPullConfig (cfg.push_pull)	109
2.6.2	IVConfig (cfg.iv)	109
2.6.3	ACHoldConfig (cfg.ac_hold)	110
2.6.4	HBHoldConfig (cfg.hb_hold)	110
2.7	VzeroConfig (cfg.vzero)	111
2.8	KeithleyConfig (cfg.keithley)	111
2.9	EChemConfig (cfg.echem)	112
2.10	XPiezoConfig (cfg.xpiezo)	112
2.11	RigConfig (top level)	112
2.12	validate(cfg)	113
3	Hardware	114
3.1	The rig as wired	114
3.2	What needs the second card	115

3.3	The Keithley 428 (GPIB address 22)	115
3.4	The NanoPZ (serial)	116
3.5	What refuses cleanly when hardware is absent	116
3.6	The bring-up ladder	117
3.7	Before an EChem or Keithley experiment	118
4	The snippets	119
4.1	Which snippet for which experiment	119
4.2	Suggested order	120
4.3	Four that are worth your time even if you skip the rest	121
4.4	What the simulator cannot tell you	122
4.5	Figures	122
4.6	Things the snippets found	122
10	Constant bias	124
10.1	What it measures	124
10.2	The loop	124
10.3	The 1 G0 ruler	125
10.4	Two Igor findings you inherit	125
10.5	Running it	125
10.6	What to watch during a run	125
10.7	See also	126
11	Push-pull	127
11.1	What it measures	127
11.2	Why the ceiling matters here and nowhere else	127
11.3	Reading the segments	128
11.4	Running it	128
11.5	What the simulator can and cannot tell you	128
11.6	Status	128
12	IV sweep	129
12.1	What it measures	129
12.2	The caps are not padding	129
12.3	Igor's sign convention, faithfully reproduced	129
12.4	Running it	130
12.5	The bias limit	130
12.6	Status	130

13 AC hold	131
13.1 What it measures	131
13.2 Why drive a junction with a sine	131
13.3 The one number to check before trusting anything	131
13.4 Running it	132
13.5 Status	132
14 High-bias hold	133
14.1 What it measures	133
14.2 The Vzero link — this is the interesting part	133
14.3 Running it	134
14.4 The bias limit	134
14.5 Status	134
15 EChem gate and cyclic voltammetry	135
15.1 The third terminal	135
15.2 Three subcommands, because Igor's module held three things	135
15.3 Two things that make this harder than it looks	135
15.4 Cyclic voltammetry	136
15.4.1 Igor's masking, kept as a mask	136
15.5 Which card	136
15.6 The gate is left on, deliberately	136
15.7 Status	137
16 Lateral monolayer mapping	138
16.1 What it measures	138
16.2 The sequence, and why the order is not negotiable	138
16.3 The X piezo	138
16.4 Two cards	139
16.5 Running it	139
16.6 Reading the result	139
16.7 Status	139
17 Bias series	140
17.1 What it measures	140
17.2 The control repeats are the point	140
17.3 The bias limit is a deliberate obstacle	141
17.4 Resume	141
17.5 Per-step calibration	141

17.6 File naming	141
17.7 Running it	142
17.8 Status	142
20 Keithley 428	143
20.1 What it is	143
20.2 Gain is the number that scales every result	143
20.3 Current suppress, and find_suppress	143
20.4 When the Keithley sources the bias	144
20.5 Simulated	144
20.6 Status	144
21 Vzero — the zero-current offset	145
21.1 The problem	145
21.2 The measurement	145
21.3 The history is the diagnostic	145
21.4 The cost	146
21.5 Where it is used	146
21.6 Status	146
30 Quick reference	147
30.1 Every command	147
30.2 Every number	147
30.2.1 Physics	147
30.2.2 Piezos	147
30.2.3 Sampling	148
30.2.4 Biases, and which need the limit raised	148
30.2.5 Support systems	148
30.3 Where to look when something is wrong	148
30.4 The trust order	149
30.5 The five safety rules, in one place	149
30.6 The one thing still unverified that can damage hardware	149

Part I

Start here: the map of the code

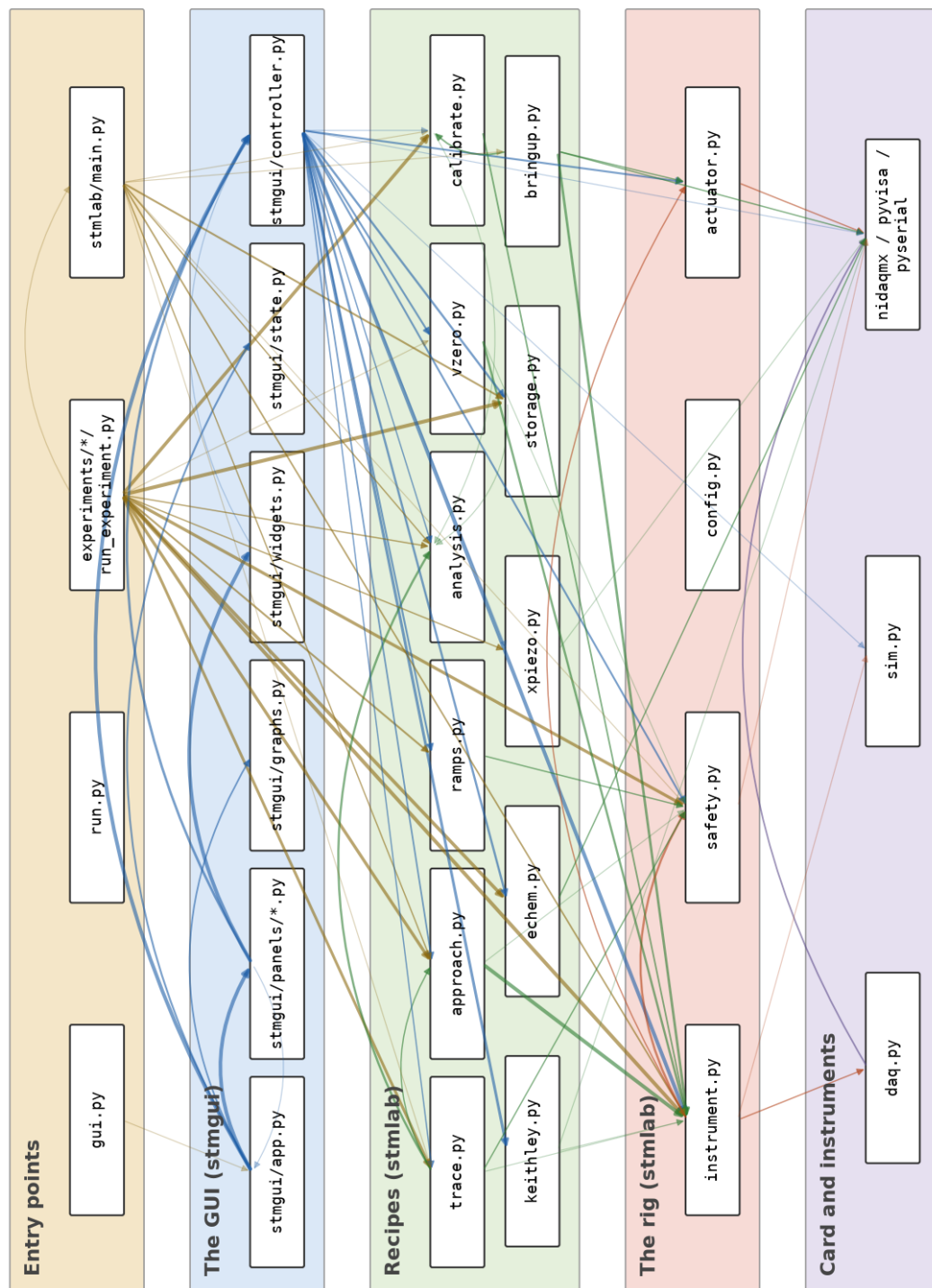
The structure map: which file calls what

Generated from the source by `manual_build/gen_structure.py` (Python's `ast`, every `def`, every `call`).
Figures: `manual/figures/structure/`. Regenerate after editing code.

Read this before chapter 00. It is the whole code on a few pages: what the files are, which file calls which function in which other file, and the two paths you will follow most — a button click, and one break-junction trace. Everything below the purpose column and the layer names is extracted from the code, not written; if a table here disagrees with a chapter, the table is right and the chapter is stale.

5.1 The shape, in one picture

Five layers. Arrows point from the file that calls to the file that is called; nothing calls upward. The GUI and the command-line experiments are alternative tops on the same `stmLab` body, and only `daq.py` / `sim.py` (and, through `keithley.py`, `actuator.py`, `echem.py`, `xpiezo.py`, the driver libraries) ever touch an instrument.



The whole code by layer, on its own page. Every arrow is at least one real call found in the source; thickness is the number of distinct functions called, colour is the calling layer. `config.py` is read by every file and its arrows are left out.

Layer	Files	What lives there
Entry points	gui.py, run.py, experiments/*/run_experiment.py, stmlab/main.py	Where a run starts. gui.py starts the panels; run.py forwards to an experiment script or to gui.py; stmlab/main.py is the constant-bias command line that experiment 01 delegates to.
The GUI	stmgui/	The panels, graphs and menus (Tk thread) and controller.py (the worker thread that owns the Rig). Every button is a RigController method.
Recipes	stmlab/trace.py, approach.py, ramps.py, analysis.py, vzero.py, calibrate.py, keithley.py, echem.py, xpiezo.py, storage.py, bringup.py	The Igor procedures, one concern per file. They know <i>what</i> to do to the junction and call the Rig to do it.
The rig	stmlab/instrument.py, safety.py, config.py, actuator.py	Rig is the only way to the card; safety is what it checks; config is every number; actuator is the coarse motor.
Card and instruments	stmlab/daq.py, stmlab/sim.py, nidaqmx, pyvisa, pyserial	DaqSession.play() is the one function that talks to the NI card; SimulatedDaqSession.play() is its stand-in.

5.2 Follow one click

Press **Find Suppress** on the panel. This is the path, file by file, top to bottom. Every button takes the same route; only the `_impl` and the recipe module change (chapter 50 lists them all, and the Button map window shows this for any control).

One click: Find Suppress

One click, from `gui.py` to the card and back to the screen.

Three things to notice. The panel never calls `stmgui` directly: it calls the controller, which queues the work. The controller never draws: it posts events, which `app.py` delivers to the panels 20 times a second. And the recipe (`keithley.find_suppress`) never touches the card: it asks the Rig, which checks the limits and calls `play()`.

5.3 Follow one trace

`python run.py constant_bias --simulate -n 100`, or Start Measurement in constant mode. The command line path is shown; the GUI's `_measure_impl` does the same steps in the same order, with its own loop instead of `trace_loop`.

One constant-bias trace: run.py constant_bias

One constant-bias trace, from the command line to the histogram.

5.4 Every file

File	Lines	Purpose	Called by
<code>gui.py</code>	37	Launcher for the panels: checks Tk, calls <code>stmgui.app.main</code>	(entry point)
<code>run.py</code>	65	Dispatcher: <code>python run.py <experiment></code> or <code>gui</code>	(entry point)
<code>stmlab/main.py</code>	213	The command line: <code>run / check / summarise / dump-config / bringup</code>	<code>experiments/01_constant_bias/run_experiment.py</code>
<code>stmlab/config.py</code>	677	Every number, as dataclasses (<code>RigConfig</code>); <code>validate()</code> refuses unsafe configs	<code>experiments/*</code> (8 scripts), <code>stmgui/app.py</code> , <code>stmgui/controller.py</code> , <code>stmlab/analysis.py</code> , <code>stmlab/approach.py</code> , <code>stmlab/bringup.py</code> , <code>stmlab/calibrate.py</code> , <code>stmlab/echem.py</code> , <code>stmlab/instrument.py</code> , <code>stmlab/main.py</code> , <code>stmlab/safety.py</code> , <code>stmlab/storage.py</code> , <code>stmlab/vzero.py</code>

File	Lines	Purpose	Called by
stmlab/safety.py	219	Limits and interlocks: check_bias, clamp_piezo, require_retracted, SafeSession, park_all_outputs	experiments/* (7 scripts), stmgui/controller.py, stmlab/approach.py, stmlab/instrument.py, stmlab/main.py, stmlab/ramps.py, stmlab/trace.py, stmlab/vzero.py
stmlab/instrument.py	248	Rig: the one funnel to the card – play(), hold(), tracked piezo position, coarse_step() behind the interlock	experiments/* (7 scripts), stmgui/controller.py, stmlab/approach.py, stmlab/bringup.py, stmlab/calibrate.py, stmlab/keithley.py, stmlab/main.py, stmlab/trace.py, stmlab/vzero.py
stmlab/daq.py	229	The real NI card: DaqSession.play() writes a waveform and reads both inputs	stmlab/instrument.py
stmlab/sim.py	214	The fake card: SimulatedDaqSession with a junction model (gold plateau, molecule, tunnelling)	stmgui/controller.py, stmlab/instrument.py
stmlab/actuator.py	218	The coarse actuator: NanoPZ over serial, simulated, or none	stmgui/controller.py, stmlab/bringup.py, stmlab/instrument.py
stmlab/approach.py	188	engage() = separate + close_in; smash(); coarse_approach(); recover_headroom()	experiments/* (7 scripts), stmgui/controller.py, stmlab/main.py, stmlab/trace.py
stmlab/trace.py	301	build_ramp() the constant pull; capture() play + align at the spike; single_trace(); trace_loop()	experiments/* (7 scripts), stmgui/controller.py, stmlab/main.py
stmlab/ramps.py	370	The four mode waveforms: build_push_pull, build_iv, build_ac_hold, build_hb_hold	experiments/* (4 scripts), stmgui/controller.py
stmlab/analysis.py	363	Volts to physics: to_conductance, select_trace (Igor TestTrace), log_histogram, peak_position	experiments/03_iv_sweep/iv_analysis.py, stmgui/controller.py, stmlab/calibrate.py, stmlab/main.py, stmlab/storage.py, stmlab/trace.py
stmlab/storage.py	346	HDF5 files: SessionWriter / Session, save_cv_cycles / CVSession, Igor .ibw readers	experiments/* (8 scripts), stmgui/controller.py, stmlab/main.py
stmlab/calibrate.py	159	measure_zero, measure_group_delay, session_calibration	experiments/* (7 scripts), stmgui/controller.py, stmlab/bringup.py, stmlab/main.py

File	Lines	Purpose	Called by
stmlab/keithley.py	233	Keithley 428 over GPIB, find_suppress (Igor TestVirtualGround)	stmgui/controller.py
stmlab/vzero.py	149	measure_offset (Igor OffsetVoltage) and VzeroTracker (SaveOffset)	experiments/05_high_bias_hold/run_experiment.py, stmgui/controller.py
stmlab/echem.py	332	CounterElectrode gate; CV ramp builders and run_cv	experiments/06_echem_gate_cv/run_experiment.py, stmgui/controller.py
stmlab/xpiezo.py	128	XPiezo: the lateral piezo on the second card	experiments/07_lateral_monolayer/run_experiment.py, stmgui/controller.py
stmlab/bringup.py	464	Hardware bring-up checks: devices, timing, hold, preamp, actuator	stmgui/main.py
stmgui/app.py	324	App: builds panels, menus, the 50 ms event pump; InitializeExperiment	gui.py, stmgui/panels/voltage_offset.py
stmgui/controller.py	1278	RigController: every button as a method; one worker thread owns the Rig; events out; COMMAND_MAP	stmgui/app.py, stmgui/panels/button_map.py, stmgui/panels/echem_panel.py, stmgui/panels/main_panel.py, stmgui/widgets.py
stmgui/state.py	337	GuiOptions (Igor globals without a config home), Param, PARAMS binding table	stmgui/app.py
stmgui/widgets.py	331	SetVariable, ValDisplay, CheckBox, button, Tooltip, Inspector binding	stmgui/panels/echem_panel.py, stmgui/panels/main_panel.py, stmgui/panels/voltage_offset.py
stmgui/graphs.py	388	The nine Igor graph windows (matplotlib in Tk) and GraphSet routing	stmgui/app.py
stmgui/panels/main_panel.py	470	The BreakJunctionMeasurement panel	stmgui/app.py
stmgui/panels/voltage_offset.py	90	The Voltage_Offset panel	stmgui/app.py
stmgui/panels/echem_panel.py	103	The EChem panel	stmgui/app.py
stmgui/panels/history.py	53	The History window	stmgui/app.py
stmgui/panels/button_map.py	110	The Button map and Inspector windows	stmgui/app.py
experiments/01_constant_bias/run_experiment.py	47	delegates to stmlab.main (the reference loop)	run.py
experiments/02_push_pull/run_experiment.py	231	loop: engage, ramps.build_push_pull, capture, save	run.py
experiments/03_iv_sweep/run_experiment.py	312	loop: engage, ramps.build_iv, capture, save; iv_analysis.py reads it back	run.py

File	Lines	Purpose	Called by
experiments/04_ac_hold/run_experiment.py	287	loop: engage, ramps.build_ac_hold, capture, save	run.py
experiments/05_high_bias_hold/run_experiment.py	281	loop: engage, ramps.build_hb_hold (optionally at Vzero), capture, save	run.py
experiments/06_echem_gate_cv/run_experiment.py	332	gate / cv / traces sub-commands over echem.py	run.py
experiments/07_lateral_monolayer/run_experiment.py	244	per site: trace_loop, withdraw, xpiezo.move_nm, approach	run.py
experiments/08_bias_series/run_experiment.py	318	Igor rungo: one trace_loop per bias, resumable	run.py

5.5 Who calls whom, file by file

For each file: the functions it defines, and for each one the functions in *other* files it calls (file: function). Calls within the same file are left out, as are calls to numpy, h5py, Tk and the standard library. A method is shown as `Class.method`; `self.x` means a method of the same class.

5.5.1 gui.py

Function	Calls (file: function)
(module level)	stmgui/app.py: main

5.5.2 run.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.3 stmlab/main.py

Function	Calls (file: function)
run (:25)	stmlab/approach.py: coarse_approach; stmlab/approach.py: recover_headroom; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: trace_loop
summarise (:74)	stmlab/analysis.py: log_histogram; stmlab/analysis.py: peak_position; stmlab/storage.py: Session; stmlab/storage.py: Session.conductances
main (:149)	stmlab/bringup.py: main; stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json; stmlab/config.py: RigConfig.to_json; stmlab/config.py: validate

5.5.4 stmlab/config.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.5 stmlab/safety.py

Function	Calls (file: function)
check_pull_headroom (:72)	stmlab/config.py: Calibration.nm_to_piezo_volts
park_all_outputs (:163)	stmlab/config.py: ChannelMap.path

5.5.6 stmlab/instrument.py

Function	Calls (file: function)
Rig.open (:41)	stmlab/actuator.py: make_actuator; stmlab/daq.py: DaqSession; stmlab/sim.py: SimulatedDaqSession
Rig.piezo_nm (:90)	stmlab/config.py: Calibration.piezo_volts_to_nm
Rig.play (:104)	stmlab/daq.py: DaqSession.play
Rig.hold (:130)	stmlab/safety.py: check_bias; stmlab/safety.py: clamp_piezo
Rig.piezo_step_nm (:152)	stmlab/config.py: Calibration.nm_to_piezo_volts
Rig.set_bias (:157)	stmlab/safety.py: check_bias
Rig.conductance_of (:189)	stmlab/config.py: Calibration.volts_to_amps
Rig.read_current_uA (:203)	stmlab/config.py: Calibration.volts_to_amps
Rig.coarse_step (:229)	stmlab/actuator.py: Actuator.step; stmlab/safety.py: check_step_budget; stmlab/safety.py: require_retracted

5.5.7 stmlab/daq.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.8 stmlab/sim.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.9 stmlab/actuator.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.10 stmlab/approach.py

Function	Calls (file: function)
_headroom_ok (:29)	stmlab/config.py: Calibration.nm_to_piezo_volts
separate (:35)	stmlab/instrument.py: Rig.piezo_step_nm; stmlab/instrument.py: Rig.probe
close_in (:55)	stmlab/instrument.py: Rig.in_contact; stmlab/instrument.py: Rig.piezo_step_nm
engage (:71)	stmlab/instrument.py: Rig.in_contact

Function	Calls (file: function)
smash (:102)	stmlab/instrument.py: Rig.piezo_step_nm
coarse_approach (:120)	stmlab/instrument.py: Rig.coarse_step; stmlab/instrument.py: Rig.read_current_uA; stmlab/safety.py: require_retracted
withdraw_coarse (:161)	stmlab/instrument.py: Rig.coarse_step
recover_headroom (:167)	stmlab/instrument.py: Rig.withdraw

5.5.11 stmlab/trace.py

Function	Calls (file: function)
TraceRecord.conductance_g0 (:60)	stmlab/analysis.py: to_conductance
TraceRecord.displacement_nm (:68)	stmlab/analysis.py: displacement_nm
build_ramp (:77)	stmlab/analysis.py: spike_indices; stmlab/safety.py: check_bias; stmlab/safety.py: check_pull_headroom
capture (:166)	stmlab/instrument.py: Rig.play
_measure_delay (:206)	stmlab/analysis.py: find_alignment_edge
trace_loop (:236)	stmlab/analysis.py: select_trace; stmlab/approach.py: engage; stmlab/approach.py: smash

5.5.12 stmlab/ramps.py

Function	Calls (file: function)
_assemble (:90)	stmlab/safety.py: SafetyViolation; stmlab/safety.py: check_bias

5.5.13 stmlab/analysis.py

Function	Calls (file: function)
to_conductance (:66)	stmlab/config.py: Calibration.volts_to_amps

5.5.14 stmlab/storage.py

Function	Calls (file: function)
SessionWriter.open (:50)	stmlab/config.py: RigConfig.to_dict
Session.__init__ (:160)	stmlab/config.py: RigConfig.from_dict
Session.conductance (:183)	stmlab/analysis.py: to_conductance
save_cv_cycles (:255)	stmlab/config.py: RigConfig.to_dict
CVSession.__init__ (:305)	stmlab/config.py: RigConfig.from_dict
CVSession.current_a (:338)	stmlab/config.py: Calibration.volts_to_amps

5.5.15 stmLab/calibrate.py

Function	Calls (file: function)
measure_zero (:28)	stmLab/config.py: Calibration.volts_to_amps; stmLab/instrument.py: Rig.hold
measure_group_delay (:57)	stmLab/analysis.py: find_alignment_edge; stmLab/instrument.py: Rig.play
measure_gain_offset (:105)	stmLab/instrument.py: Rig.hold

5.5.16 stmLab/keithley.py

Function	Calls (file: function)
find_suppress (:199)	stmLab/instrument.py: Rig.hold

5.5.17 stmLab/vzero.py

Function	Calls (file: function)
_ensure_contact (:49)	stmLab/instrument.py: Rig.in_contact; stmLab/instrument.py: Rig.piezo_step_nm; stmLab/instrument.py: Rig.set_bias; stmLab/safety.py: SafetyViolation
measure_offset (:66)	stmLab/config.py: Calibration.volts_to_amps; stmLab/instrument.py: Rig.hold; stmLab/instrument.py: Rig.set_bias

5.5.18 stmLab/echem.py

Function	Calls (file: function)
_run_cv_hardware (:283)	stmLab/config.py: ChannelMap.path

5.5.19 stmLab/xpiezo.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.20 stmLab/bringup.py

Function	Calls (file: function)
step_devices (:68)	stmLab/config.py: ChannelMap.path
step_timing (:143)	stmLab/calibrate.py: measure_group_delay; stmLab/instrument.py: Rig
step_hold (:191)	stmLab/instrument.py: Rig; stmLab/instrument.py: Rig.hold; stmLab/instrument.py: Rig.piezo_goto

Function	Calls (file: function)
step_preampl (:250)	stmlab/calibrate.py: expected_resistor_v; stmlab/calibrate.py: measure_zero; stmlab/config.py: Calibration.volts_to_amps;
step_actuator (:309)	stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.hold stmlab/actuator.py: NanoPZActuator; stmlab/actuator.py: list_serial_ports
main (:410)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.21 stmgui/app.py

Function	Calls (file: function)
App.__init__ (:41)	stmgui/controller.py: RigController; stmgui/controller.py: RigController.connect_instruments; stmgui/controller.py: RigController.request_stop; stmgui/graphs.py: GraphSet; stmgui/panels/echem_panel.py: EChemPanel; stmgui/panels/history.py: HistoryWindow; stmgui/panels/main_panel.py: MainPanel; stmgui/panels/voltage_offset.py: VoltageOffsetPanel; stmgui/state.py: GuiState; stmgui/state.py: simulated_state
App.log (:95)	stmgui/panels/history.py: HistoryWindow.append
App._build_menus (:98)	stmgui/graphs.py: GraphSet.names; stmgui/graphs.py: GraphSet.show
App._load_cfg (:156)	stmgui/controller.py: RigController.load_config
App._save_cfg (:166)	stmgui/controller.py: RigController.save_config
App._open_data (:173)	stmgui/controller.py: RigController.data_dir
App._runngo (:184)	stmgui/controller.py: RigController.runngo
App._lateral (:204)	stmgui/controller.py: RigController.lateral_expt
App._move_x (:221)	stmgui/controller.py: RigController.move_xpiezo
App.show_button_map (:227)	stmgui/panels/button_map.py: ButtonMapWindow; stmgui/panels/button_map.py: ButtonMapWindow.refresh; stmgui/panels/button_map.py: ButtonMapWindow.show
App.inspect (:237)	stmgui/panels/button_map.py: InspectorWindow; stmgui/panels/button_map.py: InspectorWindow.show_text; stmgui/panels/history.py: HistoryWindow.append
App._fanout (:258)	stmgui/panels/history.py: HistoryWindow.append
App.quit (:287)	stmgui/controller.py: RigController.shutdown
main (:299)	stmgui/state.py: GuiState; stmgui/state.py: simulated_state; stmlab/config.py: RigConfig.from_json

5.5.22 stmgui/controller.py

Function	Calls (file: function)
RigController.__init__ (:93)	stmlab/vzero.py: VzeroTracker
RigController.shutdown (:204)	stmlab/keithley.py: Keithley428.close
RigController._connect_impl (:269)	stmlab/keithley.py: make_keithley

Function	Calls (file: function)
RigController._make_rig (:292)	stmlab/actuator.py: SimulatedActuator; stmlab/instrument.py: Rig; stmlab/safety.py: park_all_outputs; stmlab/safety.py: verify_devices; stmlab/sim.py: SimulatedDaqSession
RigController._start_writing_impl (:305)	stmlab/calibrate.py: session_calibration; stmlab/config.py: Calibration.nm_to_piezo_volts; stmlab/config.py: validate; stmlab/instrument.py: Rig.hold
RigController._kill_tasks_impl (:333)	stmlab/instrument.py: Rig.close; stmlab/keithley.py: Keithley428.zero_check
RigController._reset_daq_impl (:366)	stmlab/echem.py: CounterElectrode.off; stmlab/safety.py: park_all_outputs
RigController._background_read (:399)	stmlab/config.py: Calibration.volts_to_amps; stmlab/instrument.py: Rig.hold
RigController._piezo_step_impl (:434)	stmlab/instrument.py: Rig.piezo_step_nm
RigController._piezo_goto_impl (:445)	stmlab/instrument.py: Rig.piezo_goto
RigController._set_tip_bias_impl (:460)	stmlab/instrument.py: Rig.set_bias; stmlab/keithley.py: Keithley428.set_bias_mv; stmlab/safety.py: SafetyViolation
RigController._set_gain_impl (:483)	stmlab/keithley.py: Keithley428.set_gain
RigController._find_suppress_impl (:532)	stmlab/instrument.py: Rig.set_bias; stmlab/keithley.py: find_suppress
RigController._find_offset_impl (:556)	stmlab/vzero.py: measure_offset
RigController._step_actuator_impl (:586)	stmlab/actuator.py: Actuator.close; stmlab/actuator.py: Actuator.step; stmlab/actuator.py: make_actuator; stmlab/instrument.py: Rig.coarse_step
RigController._approach_impl (:605)	stmlab/approach.py: ApproachError; stmlab/config.py: Calibration.volts_to_amps; stmlab/instrument.py: Rig.coarse_step; stmlab/instrument.py: Rig.hold
RigController._build_ramp (:681)	stmlab/ramps.py: build_ac_hold; stmlab/ramps.py: build_hb_hold; stmlab/ramps.py: build_iv; stmlab/ramps.py: build_push_pull; stmlab/trace.py: build_ramp
RigController._open_writer (:699)	stmlab/storage.py: SessionWriter
RigController._close_writer (:707)	stmlab/storage.py: SessionWriter.close; stmlab/storage.py: SessionWriter.write_summary; stmlab/vzero.py: VzeroTracker.as_arrays
RigController._measure_impl (:722)	stmlab/analysis.py: select_trace; stmlab/approach.py: recover_headroom; stmlab/approach.py: smash; stmlab/config.py: Calibration.piezo_volts_to_nm; stmlab/config.py: validate; stmlab/instrument.py: Rig.set_bias; stmlab/storage.py: SessionWriter.append; stmlab/trace.py: capture; stmlab/vzero.py: VzeroTracker.maybe_measure
RigController._histogram_block (:872)	stmlab/analysis.py: log_histogram
RigController._gate_on_impl (:897)	stmlab/echem.py: CounterElectrode.set_mv; stmlab/echem.py: make_counter_electrode
RigController._gate_set_impl (:910)	stmlab/echem.py: CounterElectrode.set_mv
RigController._cv_impl (:922)	stmlab/echem.py: run_cv; stmlab/keithley.py: Keithley428.zero_check; stmlab/storage.py: save_cv_cycles
RigController._xp (:944)	stmlab/xpiezo.py: make_xpiezo
RigController._lateral_impl (:1012)	stmlab/instrument.py: Rig.coarse_step; stmlab/instrument.py: Rig.piezo_goto; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw

Function	Calls (file: function)
RigController.save_config (:1048)	stmlab/config.py: RigConfig.to_json
RigController.load_config (:1052)	stmlab/config.py: RigConfig.from_json

5.5.23 stmgui/state.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.24 stmgui/widgets.py

Function	Calls (file: function)
tip_text (:52)	stmgui/controller.py: describe_command

5.5.25 stmgui/graphs.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.26 stmgui/panels/main_panel.py

Function	Calls (file: function)
MainPanel._sv (:47)	stmgui/widgets.py: SetVariable
MainPanel._cb (:52)	stmgui/widgets.py: CheckBox
MainPanel._build (:57)	stmgui/controller.py: RigController.piezo_step; stmgui/controller.py: RigController.start_measurement; stmgui/controller.py: RigController.step_actuator; stmgui/widgets.py: Tooltip; stmgui/widgets.py: ValDisplay; stmgui/widgets.py: button; stmgui/widgets.py: groupbox; stmgui/widgets.py: tip_text
MainPanel._build_daq_tab (:228)	stmgui/widgets.py: ValDisplay; stmgui/widgets.py: button
MainPanel._slider_release (:382)	stmgui/controller.py: RigController.piezo_goto
MainPanel.set_writing (:394)	stmgui/widgets.py: set_enabled
MainPanel.set_busy (:407)	stmgui/widgets.py: set_enabled

5.5.27 stmgui/panels/voltage_offset.py

Function	Calls (file: function)
VoltageOffsetPanel.__init__ (:26)	stmgui/widgets.py: CheckBox; stmgui/widgets.py: SetVariable; stmgui/widgets.py: ValDisplay; stmgui/widgets.py: button; stmgui/widgets.py: groupbox
VoltageOffsetPanel._toggled (:71)	stmgui/app.py: App.log

5.5.28 stmgui/panels/echem_panel.py

Function	Calls (file: function)
EChemPanel.__init__ (:27)	stmgui/controller.py: RigController.start_cv; stmgui/widgets.py: SetVariable; stmgui/widgets.py: button; stmgui/widgets.py: groupbox; stmgui/widgets.py: set_enabled
EChemPanel.set_cv_enabled (:79)	stmgui/widgets.py: set_enabled
EChemPanel.on_event (:88)	stmgui/widgets.py: set_enabled

5.5.29 stmgui/panels/history.py

Calls nothing in another file of this package (a leaf: it is called, it does not call).

5.5.30 stmgui/panels/button_map.py

Function	Calls (file: function)
ButtonMapWindow.refresh (:47)	stmgui/controller.py: command_table

5.5.31 experiments/01_constant_bias/run_experiment.py

Function	Calls (file: function)
(module level)	stmlab/main.py: main

5.5.32 experiments/02_push_pull/run_experiment.py

Function	Calls (file: function)
run (:55)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/ramps.py: build_push_pull; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.append; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: capture
main (:199)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.33 experiments/03_iv_sweep/run_experiment.py

Function	Calls (file: function)
sweep_stats (:52)	stmlab/config.py: Calibration.volts_to_amps

Function	Calls (file: function)
run (:65)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/ramps.py: build_iv; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.append; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: capture
plot_first_traces (:207)	stmlab/storage.py: Session
main (:280)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.34 experiments/04_ac_hold/run_experiment.py

Function	Calls (file: function)
lockin_summary (:57)	stmlab/config.py: Calibration.volts_to_amps
run (:82)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/ramps.py: build_ac_hold; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.append; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: capture
main (:257)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.35 experiments/05_high_bias_hold/run_experiment.py

Function	Calls (file: function)
run (:75)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/ramps.py: build_hb_hold; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.append; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: capture; stmlab/vzero.py: measure_offset
main (:252)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.36 experiments/06_chem_gate_cv/run_experiment.py

Function	Calls (file: function)
cmd_gate (:62)	stmlab/echem.py: CounterElectrode.off; stmlab/echem.py: CounterElectrode.on; stmlab/echem.py: CounterElectrode.set_mv; stmlab/echem.py: make_counter_electrode
cmd_cv (:101)	stmlab/config.py: Calibration.volts_to_amps; stmlab/echem.py: run_cv; stmlab/storage.py: save_cv_cycles
_plot_cv (:130)	stmlab/config.py: Calibration.volts_to_amps
cmd_traces (:158)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: measure_zero; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/echem.py: CounterElectrode.off; stmlab/echem.py: CounterElectrode.on; stmlab/echem.py: CounterElectrode.set_mv; stmlab/echem.py: make_counter_electrode; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: trace_loop
main (:303)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.37 experiments/07_lateral_monolayer/run_experiment.py

Function	Calls (file: function)
engage_site (:58)	stmlab/approach.py: coarse_approach; stmlab/approach.py: recover_headroom; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw
withdraw_site (:83)	stmlab/instrument.py: Rig.coarse_step; stmlab/instrument.py: Rig.withdraw
run (:96)	stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: trace_loop; stmlab/xpiezo.py: XPiezo.move_nm; stmlab/xpiezo.py: make_xpiezo
main (:216)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.5.38 experiments/08_bias_series/run_experiment.py

Function	Calls (file: function)
check_bias_limits (:106)	stmlab/safety.py: SafetyViolation
run (:127)	stmlab/approach.py: recover_headroom; stmlab/calibrate.py: measure_zero; stmlab/calibrate.py: session_calibration; stmlab/config.py: validate; stmlab/instrument.py: Rig; stmlab/instrument.py: Rig.set_bias; stmlab/instrument.py: Rig.withdraw; stmlab/safety.py: SafeSession; stmlab/storage.py: SessionWriter; stmlab/storage.py: SessionWriter.write_summary; stmlab/trace.py: trace_loop

Function	Calls (file: function)
main (:287)	stmlab/config.py: RigConfig; stmlab/config.py: RigConfig.from_json

5.6 How to use this chapter

- To find where something happens, start at the entry point you used (`gui.py` or `run.py`), find the row in *Who calls whom*, and follow the `file: function` entries downward; each one is a row in the next table. Three hops reach the card.
- To find who uses a function, search this chapter for its name: every caller lists it.
- In the GUI, hover or right-click any control: the tooltip and the Inspector give the same `file:line` chain for that control (chapter 40).
- `less -N stmlab/trace.py` then `/def` captures opens the source at the function; the `:line` after each name in the tables is where the `def` is.

Part II

The GUI

The GUI: what it is and how to start it

Igor: InitializeExperiment, Setup1_STMBJ.ipf:55; the window macros in Windows_STMBJ.ipf.
Module: stmgui/, launcher gui.py.

40.1 What this folder is

python_code_entire_igor_w_GUI/ is python_code_entire_igor/ — the complete command-line translation of the Igor procedures, with its stmlab/ package, eight experiments/, sixteen snippets/ and the core manual — copied byte for byte, plus one new package, stmgui/, that rebuilds everything the Igor experiment put on screen:

- the **Molecular Break Junction Measurement** panel (Igor: BreakJunctionMeasurement), with its DAQ / Inputs / Options tabs, the Keithley, actuator and piezo groups, the live readouts and the counters;
- the **Voltage Offset** panel (Igor: Voltage_Offset) — Find Offset, which the lab calls Find Zero;
- the **Electrochemistry** panel (Igor: EChem);
- the nine graph windows — HighRes, PullOutGvsE, PullOutLowG, AuAuConductanceLevel, Sense InDisplay, LogHistOfBlock, Izero, Izero_Time, CyclicVoltammogram;
- a **History** window standing in for Igor's history area;
- and, under a **Macros** menu, the two procedures you ran from Igor's command line, rungo() and LateralEXPT().

Nothing in stmlab/ was changed. Every button calls the same function the command-line experiment calls, so the GUI is exactly as trustworthy as the package underneath it — and no more: the hardware paths (the NI card, the Keithley over GPIB, the second card, the NanoPZ) have still never met their instruments.

There are now three ways into the same code:

Way in	Command	When
The panels	python gui.py --simulate	The lab flow you know from Igor
The dispatcher	python run.py <experiment> --simulate	Unattended runs, campaigns, scripts
The snippets	python snippets/01_numbers.py	Learning one piece at a time

python run.py gui --simulate is the same as python gui.py --simulate; the dispatcher just forwards to the launcher.

40.2 Installing

Tkinter is part of Python itself, not a pip package, and not every Python has it. The python.org installers for macOS and Windows bundle it; Debian and Ubuntu need `apt install python3-tk`; Homebrew's Python does **not** include it and needs `brew install python-tk@3.12` (matching the Python version) or, simpler, the python.org build. Check with:

```
python3 -c "import tkinter; print(tkinter.TkVersion)"
```

Create the virtual environment from an interpreter that passes that test, then install the same requirements as the command-line folder:

```
python3 -m venv .venv
./venv/bin/pip install -r requirements.txt      # numpy, h5py, matplotlib, pytest
./venv/bin/python gui.py --simulate
./venv/bin/python -m pytest tests/ -q         # 122 passed
```

On Windows replace `./venv/bin/` with `.venv\{}\Scripts\{}`. On the lab PC add the hardware drivers — `nidaqmx` for the card, `pyserial` for the NanoPZ, `pyvisa` (with NI-488.2) for the Keithley 428 — and launch without `--simulate`.

If you launch with an interpreter that has no Tk, `gui.py` does not traceback; it prints the three install lines above and exits.

40.3 Launch options

```
python gui.py [--simulate] [--config FILE.json] [-v]
```

- `--simulate` builds the state with `stmgui.state.simulated_state()`: `cfg.simulate = True`, a pretend second card (`channels.low_res_device = "Dev2"`) so the EChem and X-piezo features can be exercised, and `actuator.kind = "simulated"`. The window title gains [SIMULATED].
- `--config FILE.json` starts from a saved `RigConfig` (chapter 46 explains saving one from the File menu). With `--simulate` as well, the loaded config is forced into simulate mode.
- `-v` sets the console log level to `DEBUG`. The History window shows `INFO` and above regardless.

40.4 What opens, and why

Igor's `InitializeExperiment` (`Setup1_STMBJ.ipf:55-75`) declared the globals, opened the main panel, set up the Keithley over GPIB, initialised the EChem module and opened the Voltage Offset panel. `stmgui.app.App` does the same things in the same order:

1. `GuiState()` — the config (`RigConfig`) and the panel-only options (`GuiOptions`); chapter 48.
2. The main panel, in the root window.
3. The History window, the Voltage Offset panel and the Electrochemistry panel, as separate windows.

4. `RigController.connect_instruments()` — Igor's `SetUpGPIB_Keithley`. In simulate this is a `SimulatedKeithley` that records every command; on hardware it opens the GPIB resource, and if that fails the History says so and the Keithley controls refuse until it succeeds, while everything else works.

The graph windows are not opened at launch. Each opens itself the first time data arrives for it — the `HighRes` window on the first background read, the pull-out graphs on the first trace, `Izero` after `Find Offset`, `LogHistOfBlock` after the first histogram block, `CyclicVoltammogram` after a CV — which is what Igor's `DoWindow/F ...; if (V_flag==0) Execute "Window()"` idiom did. All of them are also under the **Windows** menu. Closing a graph or a panel only hides it; the menu brings it back.

40.5 The window map

Window title	Igor macro	Chapter
Molecular Break Junction Measurement	<code>BreakJunctionMeasurement() : Panel</code>	42
Voltage Offset	<code>Voltage_Offset() : Panel</code>	43
Electrochemistry	<code>EChem() : Panel</code>	44
HighRes	<code>HighRes() : Graph</code>	45
PullOutGvsE	<code>PullOutGvsE() : Graph</code>	45
PullOutLowG	<code>PullOutLow() : Graph</code>	45
AuAuConductanceLevel	<code>AuAuConductanceLevel() : Graph</code>	45
SenseInDisplay	<code>SenseInDisplay() : Graph</code>	45
LogHistOfBlock	<code>LogHistOfBlock() : Graph</code>	45
I(Applied Bias) fit - Izero estimate	<code>Izero() : Graph</code>	45
Izero change over time	<code>Izero_Time() : Graph</code>	45
Cyclic Voltammogram	<code>CyclicVoltammogram() : Graph</code>	45
History	Igor's history area	45

40.6 What "simulate" swaps in

With `--simulate` no driver is imported and no instrument is touched. In place of each one the package's own stand-ins run:

- `stmlab.sim.SimulatedDaqSession` for the PXI-4461: a junction model with a gold plateau, an occasional molecule, tunnelling decay, an amplifier offset and noise, and a group delay of 37 samples so the alignment spike has something to find.
- `stmlab.actuator.SimulatedActuator` for the NanoPZ, **linked to the simulated surface**: each step moves the surface 8 nm, so `Start Approach` really does walk in until current appears and `recover_headroom` really does back off when contact is too low to pull from. The command-line package's `Rig.open()` creates an unlinked actuator; the GUI's `RigController._make_rig` links it on purpose.
- `stmlab.keithley.SimulatedKeithley`: same command surface, records the command strings (`C0X`, `H6R6X`, ...) instead of sending them.

- `stmlab.echem.SimulatedCounterElectrode` and the simulated electrochemical cell inside `echem.run_cv` (a double-layer capacitance plus one reversible redox couple).
- `stmlab.xpiezo.SimulatedXPiezo`.

What the simulator cannot tell you: whether the card's clocks, the Keithley's GPIB address, the serial port or the second card's channels are right on your rig. That is bring-up, chapter 03, and it has not been done with this code.

40.7 The menus, briefly

File loads and saves the config as JSON, opens today's data folder in the file browser, and quits. **Macros** is Igor's Macros menu grown up: the three panels, `rungo()`, `LateralEXPT()`, the X piezo moves, and Stop. **Windows** lists the nine graphs and History. **Help** says where this manual is. Chapter 46 has the details.

40.8 Stopping, and quitting

Igor polled the Alt key inside every loop (`GetKeyState`, `Functions_STMBJ.ipf:1697`). Here the Alt key, the Escape key and the red **Stop (Alt)** button beside Start Measurement all set the same flag. The running command finishes the step it is on — the current attempt of a measurement, the current actuator step of an approach, the current point of a suppress sweep — and returns; nothing is interrupted half-way through a waveform.

Quitting (File → Quit, or the window's close box) runs the controller's `shutdown()`: Kill Tasks — outputs parked, the session file closed, zero check back on — then the Keithley released. Do this rather than killing the process: the card's own idle behaviour is the only thing that parks the outputs if you do not.

40.9 Every control says which Python it runs

Hover over any button and a tooltip says what the click does, which Igor procedure it mirrors, and **exactly which Python runs**: the controller method with its file and line, the worker-thread `_impl` that does the work, and the `stmlab` functions it calls, each with its file and line. For Find Suppress:

```
Button 'Find Suppress': FindSuppress, Controls_STMBJ.ipf:479. Zero the
tip bias, TestVirtualGround, restore the bias.
Python: stmgui/controller.py:527 RigController.find_suppress
       stmgui/controller.py:532 RigController._find_suppress_impl (worker thread)
Igor: FindSuppress -> TestVirtualGround
Calls: stmlab/instrument.py:157 Rig.set_bias; stmlab/keithley.py:199 find_suppress
```

Hover over any entry or checkbox and it names the Igor global it was bound to, the config attribute it writes now, and its limits.

Four ways to the same information, from the most visible:

1. The **"Which code runs this?"** button at the bottom of the main panel opens the **Button map**: one window with every control, its `RigController` method and worker-thread `_impl` (file:line), the Igor procedure, and the `stmlab` functions reached — generated from the loaded code, so the line numbers are current. The same window is under **Windows** and **Help** (on macOS the menu bar is at the top of the screen, not in the window).

2. **Right-click any control** (macOS: Control-click, or a two-finger tap) and the **Inspector** window opens with that control's text — the same as its tooltip, but it stays open and can be copied from.
3. **Hover** for half a second: the tooltip.
4. **The History window** prints, for every click, a line `-> Find Suppress: stmgui/controller.py:532 RigController._find_suppress_impl` before the command's own output, and the status line at the bottom of the main panel says `running: <command> while it is in progress`.

The orange hint line under the status line says the same thing in one sentence, so a fresh launch shows where to look.

The lab flow has not changed. **Start Writing** → **Background Sampling** → **Start Approach** → **Find Offset** → **Find Suppress** → **Start Measurement**, then **Kill Tasks** at the end. Stop is Alt, Escape, or the Stop button.

40.10 The first five minutes on the simulator

1. `./venv/bin/python gui.py --simulate`. Four windows open: the main panel, Voltage Offset, Electrochemistry, History. The History says `SIMULATED rig` and then `Keithley 428 ready (simulated)`.
2. **DAQ tab** → **Start Writing**. The History shows the config warnings (`hv_amp_gain is unknown ... is normal`), output task `ON: piezo 0.000 V (0 nm)`, bias `+100.0 mV`, and the session calibration line. Start Writing greys out, Kill Tasks stays live, the slider wakes up.
3. Tick **Background Sampling (Beep is ON)**. The `I / V / Piezo` readouts start updating four times a second and the HighRes window opens.
4. **Start Approach**. The counter beside Step Size climbs — about forty steps on the simulator — and the History prints one `Approach: actual current= ... line per step` until the current appears.
5. Voltage Offset panel → **Find Offset**. The Izero window opens with the fitted line; `V0` and `I0` fill in (near zero on the simulator, which has no bias offset).
6. **Find Suppress**. The History prints `New current suppress is ... uA` and the Suppress `I` value updates.
7. Set **Stop #** to 101 and press **Start Measurement**. Saved and Attempts count up, the three PullOut windows redraw on every attempt, and when Saved reaches 101 the LogHistOfBlock window opens with the first 100-trace histogram and the History names the file that was written.
8. **Kill Tasks**, then **File** → **Quit**.

The whole sequence takes under a minute on the simulator. Chapter 41 goes through each of those steps in depth.

40.11 Where to read next

- **41** — the lab flow, step by step, with what to expect and what can go wrong.
- **42, 43, 44** — the three panels, control by control.
- **45** — the graph windows and History.
- **46** — menus and macros: rungo, LateralEXPT, config files.
- **47** — what gets written to disk and how to read it back.
- **48** — how stmgui is built, for the day something needs changing.
- **49** — deviations from Igor, limitations, troubleshooting.
- **50, 51** — the quick reference and the complete binding table.
- Part II (chapters 00–30) is the core manual: the physics of each experiment, every config field, the hardware, and the command line.

The lab flow, button by button

```
Igor: StartHighResWritingTask, ApproachButton, FindOffset, FindSuppress, StartMeasurement,
MakeAttempt, StopWritingTasks in Controls_STMBJ.ipf; MeasureBreakJunctions, Functions_STMBJ.ipf:1664. Module: stmgui/controller.py.
```

This is the sequence you used in Igor, in the order you used it. Nothing about the order has changed; what follows is what each press does now, what you should see, and what to do when you do not see it. Every step is a method on `stmgui.controller.RigController`; every method runs on the controller's single worker thread, one at a time, in the order clicked (chapter 48 explains why that matters).

41.1 Start Writing

Igor: `StartHighResWritingTask`, `Controls_STMBJ.ipf:127`. Created the output task `G_0D1` with two channels — the piezo at `G_PiezoOffset_nm / K_ZPiezoScale` volts and the bias at `-TipBias/1000` — and started it.

Now: `RigController.start_writing`. In order:

1. `stmlab.config.validate(cfg)` runs; every warning goes to the History prefixed `config:`. On a fresh install you will see `hv_amp_gain` is unknown and, before any calibration, `current_zero_v` is 0. Both are informational. A fatal problem (bias above `limits.bias_max_v`, a pull longer than the piezo range) raises `ConfigError` and nothing opens.
2. On hardware, `safety.verify_devices` (the card is present and is the expected product type) and `safety.park_all_outputs` (both outputs to 0 V through a throw-away task). In simulate, neither.
3. `Rig(cfg).open()` — the AO/AI tasks, the actuator, and the tracked piezo position set to the park value.
4. One DC hold at **Piezo offset (nm)** and **Tip Bias**: Igor's `HighResWriteWave`.
5. Session calibration — `stmlab.calibrate.session_calibration`: the preamp zero with the bias at 0 V, and the AI/AO group delay from twenty alignment spikes. Igor never did this; `validate()` warns when it has not been done, so it is done here, once per launch, while the tip is still retracted. Turn it off with `GuiOptions.calibrate_on_start_writing = False` if you have a reason (chapter 48).

You should see: in the History, the warnings, then output task ON: piezo 0.000 V (0 nm), bias +100.0 mV, then session calibration: `current_zero_v=...`, `group_delay_samples=...`, `granted_sample_rate_hz=40000`. On the panel: Start Writing and Reset DAQ Devices grey out, Kill Tasks stays live, the DAQ-tab entries lock (Igor: `SetVariable ... disable=2`), the piezo slider and the Background Sampling box become live.

If not: ConfigError text in the History names the field. On hardware, `verify_devices` failing means the device name in `channels.device` is not what NI MAX shows, or the product type differs — set `channels.expected_product_type` to `None` to skip that check.

41.2 Background Sampling (Beep is ON)

Igor: `BkgdSamplingCheckProc` → `StartBkgdSampling`, `Functions_STMBJ.ipf:633`: a named background task every 15 ticks (0.25 s) that read `G_FastReadWaveSize` samples from both cards, put the mean junction voltage in mV, the mean current in μA and the piezo sense voltage into the three `ValDisplays`, and beeped when `|I|` exceeded $0.15\ \mu\text{A}$.

Now: the checkbox calls `set_background_sampling(on)`. The worker thread, whenever its command queue has been idle for `RigController.BKGD_PERIOD_S` (0.25 s), runs `_background_read`: one DC hold of **Fast Read Wave Size** samples at the present piezo and bias, `junction_mv = voltage_input_sign × mean(V) × 1000`, `current_ua = volts_to_amps(mean(I)) × 106`, the tracked piezo voltage, and a beep flag when `|I|` exceeds `GuiOptions.beep_current_ua` (0.15). Because it runs only while the queue is idle, sampling never interleaves with a measurement: it simply pauses while Start Measurement, Find Offset or an approach is running, and resumes after — which is what Igor's explicit `StopBkgdSampling / StartBkgdSampling` bracket around every button did.

You should see: the `I` (μA), `V` (mV) and Piezo readouts refreshing four times a second, and the HighRes window opening with the last read. On the simulator, retracted, the current is noise around zero and there is no beep; in contact it beeps every read.

If not: ticking the box before Start Writing puts Background sampling needs the output task; press Start Writing first in the History and clears the box. A read that raises (a card error on hardware) clears the box and logs background sampling stopped:

41.3 Start Approach

Igor: `ApproachButton`, `Controls_STMBJ.ipf:434` → `HighResCardApproach`, `Functions_STMBJ.ipf:1723`. Forced `G_ActuatorStepSize` to 5, then looped: read `G_FastReadWaveSize` samples, print the current, and if `|I| < 0.1\ \mu\text{A}` take one actuator step closer (`StepActuatorCloser`) and increment `G_ActuatorCounter`; otherwise stop. Alt aborted.

Now: approach. Refuses with `NotReady` before Start Writing, and with `ApproachError`: no coarse actuator configured (`actuator.kind == 'none'`); approach by hand if the config has no actuator — in which case bring the tip in by hand until the fine piezo can reach the surface. Otherwise it sets `cfg.actuator.step_size = 5` as Igor did, zeroes the counter, and loops exactly as Igor did with `actuator.stop_current_ua` ($0.1\ \mu\text{A}$) as the threshold. Each step goes through `Rig.coarse_step`, which refuses unless the fine piezo is retracted (`limits.coarse_step_max_piezo_v`, 0.1 V) and counts steps against `limits.max_coarse_steps` (2000). On hardware it waits `actuator.settle_s` (0.1 s) after each step.

You should see: the counter beside Step Size climbing, one Approach: actual current= `0.0000` uA, Voltage= `-0.1000` V line per step in the History, then a final line with the current that stopped it. About forty steps on the simulator, where the surface starts 300 nm away and each simulated step is 8 nm.

If not: if you moved the piezo up with the slider first, the interlock refuses: `SafetyViolation: refusing coarse step closer: piezo is at ... V, must be below 0.100 V`. Retract the fine piezo first. Put the slider back to 0 (Igor's `LateralEXPT` did exactly that before its

approach). Two thousand steps without current is `check_step_budget` refusing — coarse approach took 2000 steps without finding current (budget 2000). Either the tip is not where you think it is, or the current path is open.

41.4 Find Offset (Find Zero)

Igor: `FindOffset`, `Controls_STMBJ.ipf:498` → `OffsetVoltage`, `Functions_STMBJ.ipf:920`. Stopped background sampling, swept the bias through $\pm G_VoltageInterval$ in `2·G_NumPoints` steps with a contact check before every point, fitted a line, stored `G_Izero` (the intercept, μA) and `G_Vzero` ($-Izero/slope$, mV), restored the bias, opened the `Izero` graph.

Now: `find_offset` → `stmlab.vzero.measure_offset`. Chapter 43 has the algorithm line by line. The result lands in the Voltage Offset panel's **V0 (fit, mV)** and **I0 (fit, μA)** displays, in the controller's `vzero_tracker.history`, and in the `Izero` and `Izero_Time` windows.

You should see: the piezo readout jump as `_ensure_contact` steps in 2 nm at a time until contact, the `Izero` window with pink points on a blue fitted line and the `V0 / I0` box, and `Vzero = +0.000 mV` (`Izero = ...`) in the History. The simulator has no bias offset, so `V0` is a few microvolts of noise; a real amplifier gives a few millivolts.

If not: the piezo ceiling with no contact raises `SafetyViolation: vzero: piezo at the ceiling with no contact`; run the coarse approach before measuring the offset — do step 3 first. A flat fit raises `vzero: flat I(V) -- no contact`, or a railed preamp at every point.

41.5 Find Suppress

Igor: `FindSuppress`, `Controls_STMBJ.ipf:479`. Wrote zero tip bias, ran `TestVirtualGround` (`Functions_STMBJ.ipf:1097`): 21 suppress values from -1 to $+1$, a `sleep/T 5` after each because "the amplifier is slow to respond" (the SPIKE comment at line 1123), 2000 samples read, then a line fitted through the points with the two ends excluded and the suppress set to the zero crossing. Then the tip bias was restored.

Now: `find_suppress`. Refuses without the output task and without the Keithley. Sets the bias to 0 V, calls `stmlab.keithley.find_suppress(rig, amp)` with Igor's numbers (21 points, 83 ms settle, 2000 samples, endpoints excluded from the fit), and restores the bias in a finally so a failure mid-sweep cannot leave the junction at zero bias. The new value is stored back as **Suppress I value** in Igor's units — the panel shows the constant, and the amplifier receives `const × 103 - gain  $\mu A$` (`Controls_STMBJ.ipf:473`).

You should see: new current suppress: 0.3469 μA from the module and New current suppress is 0.3469 μA from the controller in the History, and the Suppress I value entry updating.

If not: `KeithleyError: suppress sweep produced a flat response`; is the amplifier connected and out of zero-check? — untick Zero Check first. On a real 428 the sweep takes $21 \times (83 \text{ ms} + 50 \text{ ms of samples})$, about three seconds.

Zero the bias yourself if you call `keithley.find_suppress` from a script: the function does not know what bias you wanted back. The button does it for you.

41.6 Start Measurement, and +1

Igor: StartMeasurement, Controls_STMBJ.ipf:9 → MeasureBreakJunctions, Functions_STMBJ.ipf:1664. Returned at once if G_PullOutNumber ≥ G_StopNumber; otherwise CreateInputs, then a loop of SmashFun and SingleBreakJunctionAttempt (contact, GenerateTrace, TestTrace, SaveOffset, SavePullOut, CreateInputs again) until Saved reached Stop #, Alt was pressed, or attempts hit 100000 ("Catastrophic failure"). MakeAttempt (+1) set G_SingleAttemptFlag so the loop ran once.

Now: start_measurement(single=False) and start_measurement(single=True) are the two buttons; both run _measure_impl. Step by step:

1. NotReady without the output task. Saved (n) >= Stop # (m): nothing to do in the History if there is nothing to do (not for +1, which always makes one attempt).
2. The mode comes from the Options tab checkboxes (GuiOptions.mode()): constant when none is ticked, else push_pull, iv, ac_hold or hb_hold. _configure_mode then does what the corresponding experiments/0N runner does before its loop: for the four ramp modes, cfg.ramp.pull_length_nm is set to initial + final pull so the headroom check in engage matches the mode's real descent, and for IV, AC hold and HB hold limits.bias_max_v is raised to $1.1 \times$ the sweep or hold amplitude with a RAISING limits.bias_max_v ... warning in the History. Constant mode changes nothing.
3. validate(cfg) again; warnings to the History.
4. A session file is opened: data/<date>/<mode>_<HHMMSS>_from<Saved>.h5 (chapter 47). writing traces to ... in the History.
5. The bias is set to Tip Bias, and the loop begins. Per attempt: Attempts increments (Igor's G_PullOutAttempt, global across presses); every **Smash Freq.** attempts approach.smash drives the tip in **Smash In** nm and back **Smash Out** nm; approach.recover_headroom engages (separate if in contact, close in 0.5 nm at a time until the threshold or a railed preamp, back the coarse actuator off up to five times if contact lands too low to pull from); the ramp is built for the mode (trace.build_ramp for constant, ramps.build_* otherwise) from the actual contact point; trace.capture plays it and cuts the trace out at the alignment spike.
6. Every generated trace is drawn in the three PullOut windows and SenseInDisplay, saved or not — Igor updated PullOutConductance inside GenerateTrace before TestTrace judged it.
7. Constant mode only: analysis.select_trace, Igor's TestTrace — ended in tunnelling below **Zero Cutoff**, and has a gold plateau. The four ramp modes keep every aligned trace, as their runners do and as Igor did (it applied TestTrace to constant pulls only).
8. An accepted trace: with **V0 check ON** in constant mode, VzeroTracker.maybe_measure re-measures the offset every **Frequency (traces)** saved traces (Igor's SaveOffset); the trace is appended to the file; Saved increments; the conductance joins the current histogram block; when the block reaches 100, analysis.log_histogram runs on it and the LogHistOfBlock window redraws (Igor's LogHistFromBlocks on every saved block).
9. The loop ends when Saved reaches Stop #, when Stop is pressed (stopping: Stop pressed), when engage fails and the coarse actuator cannot help (cannot make contact: ..., Igor's return -1), or at max_attempts (Catastrophic failure: n attempts). The partial histogram block is drawn, the file is closed with its summary, and the History prints constant: 100 saved / 102 attempts this run (Saved now 101).

You should see: Attempts and Saved climbing; the status line reading running: Start Measurement; Start Measurement, +1 and Start Approach greyed while it runs; the PullOut windows redrawing per attempt; the histogram after each 100 saved.

What V0 check ON changes: in constant mode every trace is applied at Tip Bias + V0 (trace. build_ramp(bias_offset_v=...), Igor's $-(\text{TipBias} + \text{Vzero})/1000$ at Functions_STMBJ.ipf:1611, with the alignment spike still at +Tip Bias, line 1631) and V0 is re-measured periodically. In high-bias-hold mode the hold bias *becomes* V0 (Igor lines 1595-1599). In push-pull, IV and AC hold it has no effect, as in Igor. Chapter 43.

Rejections are counted by reason (engage, alignment, and the TestTrace reasons) and written into the file summary; they are not individually reported in the History, which would drown it.

41.7 Stop

Igor: GetKeyState(0) == 2, the Alt key, polled between attempts in MeasureBreakJunctions (line 1697) and between steps in HighResCardApproach (line 1751).

Now: request_stop sets a threading.Event that the measurement loop, the approach loop, rungo and LateralEXPT all check between steps. It is bound to the Alt keys, to Escape, to the **Stop (Alt)** button and to Macros → Stop. The History prints stop requested; finishing the current step. The step in progress completes — a pull is never cut short, so the piezo always ends where the waveform said it would and the tracker stays truthful.

Kill Tasks also sets the flag first, so a Kill pressed during a run stops the run and then parks.

41.8 Kill Tasks

Igor: StopWritingTasks, Controls_STMBJ.ipf:172. Stopped background sampling, sent C1X (zero check on), wrote the piezo back to zero with the bias still applied, stopped and cleared the output task, reset the second card, re-enabled the EChem gate button and disabled the CV buttons, and re-enabled the DAQ-tab controls.

Now: kill_tasks: the stop flag; background sampling off; zero check on (and the checkbox re-ticked); the session file closed if a run was interrupted; Rig.close() — which withdraws (piezo to park, bias to 0 V) and closes the tasks, after which the card's idle behaviour holds the outputs at 0 V; the counter electrode and the X piezo released; the panel back to its pre-Start state. output task OFF; outputs parked in the History.

Igor left the bias applied while zeroing the piezo and only then cleared the task. Rig.withdraw drops both, deliberately: 0 V on both channels is the one state that is safe regardless of what the tip is near.

41.9 A complete session on the simulator

The buttons, and the History lines that should follow each one (numbers will differ):

[launch]	Molecular Break Junction Measurement -- SIMULATED rig Igor flow: Start Writing -> (Background Sampling) -> ... Keithley 428 ready (simulated)
[Start Writing]	config: hv_amp_gain is unknown; piezo limits are ... output task ON: piezo 0.000 V (0 nm), bias +100.0 mV preamp zero ... uV, noise ... uV rms session calibration: current_zero_v=..., group_delay_samples=37, ...
[Background on]	(readouts update; no History line)

```

[Start Approach]  Approach: actual current= 0.0000 uA, Voltage= -0.1000 V   (x ~40)
                  Approach: actual current= 10.0000 uA, Voltage= -0.1000 V
[Find Offset]     Vzero = +0.000 mV (Izero = -5.782e-10 A)
[Find Suppress]   new current suppress: 0.3469 uA
                  New current suppress is 0.3469 uA

[Stop # = 101]
[Start Measurement] writing traces to ../data/20260902/constant_152830_from1.h5
                   no headroom for the pull; backing the coarse actuator off one step (1/5)
                   smashing tip: +30 nm then -40 nm                               (every 50 attempts)
                   wrote loghist_152901_upto101.csv (100 traces)
                   constant: 100 saved / 102 attempts this run (Saved now 101)

[Kill Tasks]      output task OFF; outputs parked

```

The no headroom line is normal on the simulator: the approach stops with the surface within reach of the retracted piezo, contact is made too low to pull 5 nm from, and `recover_headroom` backs the actuator off one step and tries again. On a real rig this is the same mechanism that keeps an overnight run alive as the contact point drifts down the piezo range.

41.10 Between runs

Saved and Attempts persist across presses within a launch, as Igor's globals did across Start Measurement calls. Each press opens a new session file whose name carries the Saved number it started from, so a session is the set of files in one dated folder, in Saved order. Edit Saved by hand if you want a fresh numbering; edit Stop # to extend a run. The mode can be changed between presses (each file records its own); the bias can be changed between presses (Tip Bias applies immediately and every file records `bias_v` per trace).

The main panel, control by control

Igor: BreakJunctionMeasurement() panel macro, Windows_STMBJ.ipf:133-375; the control procedures in Controls_STMBJ.ipf. Module: stmgui/panels/main_panel.py.

Igor's panel was one tall window of absolutely positioned controls. The rebuild keeps every group, every control and every title, in the same top-to-bottom order, laid out with a grid instead of pixel coordinates. This chapter goes through them in Igor's groups. For each control the table gives the Igor control name (what ControlInfo would have called it), its title on the panel, the procedure Igor attached to it, and what it is bound to or does now. The complete list of limits, formats and defaults is the generated table in chapter 51; here the columns are kept to what you need at the panel.

Two words about **bindings**. Every entry and checkbox is bound to one row of stmgui.state.PARAMS (chapter 48), which names a dotted attribute of the GuiState: `cfg...` is a field of the Rig Config that every data file carries, `opts...` is a field of GuiOptions, the panel-only state. Editing the control writes that attribute at once — on Return, on leaving the field, or on a spinbox arrow — and clamps to Igor's limits. Whether the new value *does* anything at once depends on the control: "immediate" below means the hardware or the running state changes now, "next Start" means it is read the next time Start Measurement builds its ramps.

42.1 The TabControl: DAQ, Inputs, Options

Igor: TabControl TAB, tabs DAQ / Inputs / Options, with MainWindowTabProc (Controls_STMBJ.ipf:39) showing the controls whose names end in `_DAQ`, `_Inputs` or `_Ramps`. The Options tab held a second TabControl RampSubTabs_Ramps with Push-Pull / IV / AC / High bias hold, handled by SubWindowTabProc (line 74). Here both are ttk.Notebooks.

42.1.1 DAQ tab

Igor name	Title	Igor proc	Bound to / does
ZPiezoScale_DAQ	Z piezo scale (nm/V)	ValDisplay of K_ZPiezoScale	shows <code>cfg.cal.piezo_nm_per_volt</code> (62)
ZSenseScale_DAQ	Z Sense scale (nm/V)	ValDisplay of K_SenseScale	shows <code>opts.sense_scale_nm_per_v</code> (314)
AcqRate_DAQ	Acquisition Rate	—	<code>cfg.ramp.sample_rate_hz</code> ; next Start Writing
WriteBufferSize_DAQ	Write Buffer Size	—	<code>cfg.ramp.settle_samples</code> ; immediate (every DC hold)
ReadWaveSize_DAQ	Fast Read Wave Size	—	<code>opts.fast_read_wave_size</code> ; immediate (background reads, approach)

Igor name	Title	Igor proc	Bound to / does
OutputRangeHighRes_DAQ	HighRes Output Range (V)	—	cfg.channels.bias_ao_range_v; next Start Writing
(new)	Piezo offset (nm)	—	opts.piezo_offset_nm; where Start Writing puts the piezo
Start_DAQ	Start Writing	StartHighResWriting Task	RigController.start_writing
Stop_DAQ	Kill Tasks	StopWritingTasks	RigController.kill_tasks
Reset_DAQ	Reset DAQ Devices	ECheMResetDAQDevices	RigController.reset_daq

The two scale ValDisplays are constants in Igor (Constant K_ZPiezoScale = 62, Setup1_STMBJ.ipf:26-28) and read-only here too; change cal.piezo_nm_per_volt in a config file if you re-calibrate the piezo.

Piezo offset (nm) is the one control on this tab Igor did not have. G_PiezoOffset_nm existed (Functions_STMBJ.ipf:44, with the comment “MUST BE LARGER THAN EXCURSION SIZE when used with unipolar piezos”) but was only settable from the command line. Start Writing parks the piezo at this height; on a unipolar piezo it is the number that gives a pull somewhere to go, so it belongs on the panel.

Enable rules (Igor’s disable= lists at Controls_STMBJ.ipf:159-166 and 210-217): while the output task is running, Start Writing and Reset DAQ Devices are greyed, the five entries are locked, the slider and the Background Sampling box are live. Kill Tasks reverses all of it. Reset DAQ Devices refuses while the task is running (Kill Tasks before resetting the DAQ devices), since resetting a card under an open task would invalidate the tracked piezo position.

42.1.2 Inputs tab

Igor name	Title	Bound to / does
PullRate_Inputs	Pull Rate (nm/s)	cfg.ramp.pull_rate_nm_per_s; next Start
Excursion_Inputs	Excursion (nm)	cfg.ramp.pull_length_nm; next Start (constant mode; the ramp modes overwrite it, chapter 41)
EngageG_Inputs	Contact Threshold (G0)	cfg.ramp.engage_g0; next engage
EngageStep_Inputs	Contact step size (nm)	cfg.ramp.approach_step_nm; next engage
SeriesR_Inputs	Series R (Ohm)	cfg.keithley.series_resistance_ohm; used only with Keithley-sourced bias (chapter 20)
ZerCutoff_Inputs	Zero Cutoff (G0)	cfg.ramp.break_g0; next trace (selection and histogram floor)
SmashFreq_Inputs	Smash Freq.	cfg.ramp.smash_every; next attempt (0 disables)
SmashIn_Inputs	Smash In	cfg.ramp.smash_in_nm
SmashOut_Inputs	Smash Out	cfg.ramp.smash_out_nm
SaveBias_Inputs	Save Bias	opts.save_bias; recorded in the file header
SaveCurrent_Inputs	Save Current	opts.save_current; recorded

Igor name	Title	Bound to / does
SavePiezoWave_Inputs	Save Piezo Wave	opts.save_piezo_wave; recorded
SaveSense_Inputs	Save Sense	opts.save_sense; recorded
SaveHist_Inputs	Save Hist	opts.save_hist; writes the histogram CSV per block
VoltageRead_Inputs	VoltageRead	opts.voltage_read; recorded

Igor's default Contact Threshold was 5 G0 (`G_ConductanceThreshold = 5`), which at gain 10^6 and 100 mV is 38.7 V at the ADC and unreachable; the package's default is 0.5 G0 and a railed preamp also counts as contact. Chapter 02 argues this out. If you type 5 here you get what Igor got: the approach loop climbs to the piezo ceiling and refuses.

The five Save boxes did real work in Igor's SavePullOut — each selected a wave to write into the .ibw block. The HDF5 session file always stores both raw channels for every accepted trace, so nothing is lost by leaving them unticked; they are written into the file header so an archived run says what Igor would have kept. Save Hist is the exception: it controls whether each histogram block is also written as a CSV (chapter 47).

Two controls that Igor created on this tab are not reproduced: CounterElectrodeCheck_EChem and CounterElectrodeSetVar_EChem (Windows_STMBJ.ipf:334-338). They were created `disable=1` and no tab procedure ever enabled them — their names end in _EChem, which none of the ControlNameList filters matched — so they were invisible in Igor. The live counter-electrode controls are on the Electrochemistry panel, chapter 44.

42.1.3 Options tab

Four sub-tabs, each with one checkbox and its numbers. **The four checkboxes are mutually exclusive:** ticking one clears the other three, as RampOptionsCheckProc (Controls_STMBJ.ipf:343) did, and with none ticked the next Start Measurement is a constant-bias run. The selection is read at Start, not before.

Sub-tab	Igor name	Title	Bound to
Push-Pull	PushPullCheck_PushPullRamps	Push-Pull	opts.push_pull_check
	NoPPCyclesSetvar_PushPullRamps	Number of cycles	cfg.push_pull.cycles
	PPInitialPull_PushPullRamps	Initial Pull (nm)	cfg.push_pull.initial_pull_nm
	PPFinalPull_PushPullRamps	Final Pull (nm)	cfg.push_pull.final_pull_nm
	PushPullLength_PushPullRamps	Push-Pull length (nm)	cfg.push_pull.push_pull_nm
	HoldLength_PushPullRamps	Hold length (nm)	cfg.push_pull.hold_nm
IV	IVCheck_IVRamps	IV	opts.iv_check
	IVSignPopup_IVRamps	Sign: (+ / - or - / +)	cfg.iv.positive_first (item 2 = True)
	MaxBias_IVRamps	MaxBias (V)	cfg.iv.max_bias_v
	IVInitPull_IVRamps	Initial Pull (nm)	cfg.iv.init_pull_nm
	IVIFinPull_IVRamps	Final Pull (nm)	cfg.iv.final_pull_nm
	IVCapLength_IVRamps	Cap length (nm)	cfg.iv.cap_nm
	IVRampLength_IVRamps	Ramp length (nm)	cfg.iv.ramp_nm

Sub-tab	Igor name	Title	Bound to
AC	ACHoldCheck_ACHold Ramps	AC Hold	opts.ac_hold_check
	ACamp_ACHoldRamps	Amp (V)	cfg.ac_hold.amp_v
	ACFreq_ACHoldRamps	Freq (kHz)	cfg.ac_hold.freq_khz
	ACInitPull_ACHold Ramps	Initial Pull (nm)	cfg.ac_hold.init_pull_nm
	ACHoldLength_ACHold Ramps	Hold length (nm)	cfg.ac_hold.hold_nm
	ACFinalPull_ACHold Ramps	Final Pull (nm)	cfg.ac_hold.final_pull_nm
	ACFinalPull_ACHold Ramps	Final Pull (nm)	cfg.ac_hold.final_pull_nm
High bias hold	HBHoldCheck_HBHold Ramps	High Bias Hold	opts.hb_hold_check
	HBBias_HBHoldRamps	Hold bias (V)	cfg.hb_hold.hold_bias_v
	HBInitPull_HBHold Ramps	Initial Pull (nm)	cfg.hb_hold.init_pull_nm
	HBFinalPull_HBHold Ramps	Final Pull (nm)	cfg.hb_hold.final_pull_nm
	HBFinalPull_HBHold Ramps	Final Pull (nm)	cfg.hb_hold.final_pull_nm
	HBHoldLength_HBHold Ramps	Hold length (nm)	cfg.hb_hold.hold_nm
	HBHoldLength_HBHold Ramps	Hold length (nm)	cfg.hb_hold.hold_nm
	HBCapIN_HBHoldRamps	Cap Length IN (nm)	cfg.hb_hold.cap_in_nm
	HBCapFIN_HBHoldRamps	Cap Length FIN (nm)	cfg.hb_hold.cap_fin_nm

The Sign popup is Igor's IVSignPopup (Controls_STMBJ.ipf:445): the first item set `G_IVSignFlag = 0`, the second = 1. The package names the flag for what it does, `positive_first`, because Igor negated the whole bias wave before applying the flag (chapter 12): the default + / - sweeps to the *negative* apex first. Igor's push-pull globals had no defaults (they were set from the panel); the values shown are the package's working defaults, and `validate()` warns if the push-pull length does not exceed the initial pull, which would mean the junction is never re-formed.

The lengths of the AC cap (`cfg.ac_hold.cap_nm`, Igor `G_ACCapLength`) have no panel control in Igor either; edit the config file.

42.2 Background Sampling (Beep is ON)

Igor: `CheckBox OnCheck, BkgdSamplingCheckProc`. **Now:** bound to `opts.bkgd_sampling`, calls `set_background_sampling`. Live only while the output task runs. Reads **Fast Read Wave Size** samples every 0.25 s, feeds the three readouts and the HighRes window, and rings the terminal bell when $|I|$ exceeds $0.15 \mu\text{A}$ (`GuiOptions.beep_current_ua`). The read runs only when the command queue is idle, so it never collides with a measurement. Chapter 41, step 2.

42.3 Keithley Controls

Igor: `GroupBox box10105`. Every control here talks to the Keithley 428 over GPIB; in simulate the commands are recorded, on hardware they go to `cfg.keithley.resource` (`GPIB0::22::INSTR`, Igor's `ibdev={0,22,...}`). If the GPIB open failed at launch, each of these refuses with the Keithley 428 is not connected and the rest of the panel is unaffected.

Igor name	Title	Igor proc	Does now
KeithleyBias	Bias	KeithleyBias	B1X / B0X; cfg.keithley.enabled; Tip Bias then quantised to 5 mV
ZeroCheckBox	Zero Check	ZeroCheckProc	C1X / C0X; opts.zero_check; re-ticked by Kill Tasks
ZeroCorrectButton	Zero Correct	ZeroCorrectProc	C2X
SuppressCheckBox	Suppress I	CurrentSuppressCheck Proc	N1X / N0X; opts.suppress_on
FindSuppressButton	Find Suppress	FindSuppress	bias to 0, keithley. find_suppress, bias back; chapter 41 step 5
setvar43	Gain (log(V/A))	SetGain	H6R<g>X; cfg.keithley. gain_exponent and cfg.cal.preamp_gain_ v_per_a = 10^g
SuppressSetVar	Suppress I value	SetCurrentSuppress	H8S<amps>, 0X with amps = const × 10^(3-g) μA; cfg.keithley. suppress_const

Gain deserves a sentence. Igor's SetGain (Controls_STMBJ.ipf:290) sent the gain to the box and recomputed $G_{\text{CurrentVoltConversion}} = 10^{(6-\text{gain})}$ for the readouts. The package keeps the box's exponent and the number every conductance divides by as two separate fields, and validate() warns when they disagree because a mismatch scales every histogram by a power of ten. The panel's Gain entry therefore writes both at once, so they cannot disagree from here. The same entry appears on the Electrochemistry panel (Igor had it in both places) and both write the same fields.

The Suppress I value is stored and shown as Igor's *constant*; what the amplifier receives is $\text{const} \times 10^{(3 - \text{gain})} \mu\text{A}$, Igor's $G_{\text{CurrentSuppress}}$ (Controls_STMBJ.ipf:473). Find Suppress writes the result back in the same units, and changing Gain re-sends the suppress at its rescaled value, as SetGain did (lines 306-310).

42.4 Actuator Controls

Igor: GroupBox ActuatorGroupBox; the two step buttons live in NanoPZ_Actuator_Functions_STM.ipf:6-23 and send 0M0 then 0PR±n over COM1 at 19200 baud.

Igor name	Title	Igor proc	Does now
StepSizeSetVar	Step Size	—	cfg.actuator.step_ size; Start Approach forces it to 5
ActuatorCounter_ Valdisp	(counter)	ValDisplay of G_ActuatorCounter	opts.actuator_ counter, steps of the last approach
StartApproachButton	Start Approach	ApproachButton	RigController. approach; chapter 41 step 3

Igor name	Title	Igor proc	Does now
StepTogether	Step closer	StepActuatorCloser	one step in
StepOpen	Step apart	StepActuatorApart	one step out

With the output task running, a step goes through `Rig.coarse_step`, which refuses unless the fine piezo is retracted below `limits.coarse_step_max_piezo_v` (0.1 V, 6 nm) — the interlock that stops a coarse step driving an extended tip into the sample — and counts against `limits.max_coarse_steps`. With the output task off (no piezo energised) the step goes straight to a fresh `make_actuator(cfg)`, as Igor’s buttons did at any time. Igor had no interlock at all.

42.5 Piezo Controls

Igor: GroupBox PiezoGroupbox.

Igor name	Title	Igor proc	Does now
MoveZIn	Step closer	PiezoStepCloser	<code>piezo_step(+Z)</code> — extend by <code>Z nm</code>
MoveZStepSize	Z (nm)	—	<code>opts.piezo_step_nm</code> , the step size
MoveZout	Step apart	PiezoStepApart	<code>piezo_step(-Z)</code> — retract by <code>Z nm</code>
SliderPos_Z	(slider)	<code>SetPiezoBiasFromSlider</code>	<code>piezo_goto(volts)</code> on release

The slider’s value is the **voltage at the DAQ output**, not a distance — Igor’s own comment above `SetPiezoBiasFromSlider` (Controls_STMBJ.ipf:376) says so. Its range is `limits.piezo_ao_min_v .. piezo_ao_max_v`, 0 to 10 V on this unipolar piezo, where Igor’s was -10 to $+10$ for a bipolar one (chapter 02 on why the difference is load-bearing). It applies on mouse release (Igor’s `live=0`), and every fine-piezo move made anywhere — a step, an approach, a pull — moves the slider to follow, so it always shows where the piezo is. Closer is $+nm$: extension is the positive direction on this rig and a pull descends.

42.6 The readouts

Igor name	Title	Igor variable	Shows
CurrentReadOut	I (μA)	<code>G_CurrentSamplingReadoutVal</code>	mean current of the last background read, in μA
JunctionVoltageReadout	V (mV)	<code>G_JunctionVoltage</code>	mean junction voltage, sign-corrected, in mV
PiezoVoltReadout	Piezo	<code>G_PiezoBiasSamplingReadoutVal</code>	the commanded piezo voltage

Igor’s Piezo readout came from the second card’s sense input (`LowResPiezoIn`); this rig has one card and no sense line, so the readout is the tracked command voltage — the same number the interlock trusts. Chapter 45 says the same about `SenseInDisplay`.

42.7 Saved, Tip Bias, Stop #, Start Measurement, Attempts, +1

Igor name	Title	Igor proc / variable	Does now
Saved_setVar	Saved	G_PullOutNumber	opts.pull_out_number; editable
TipBias	Tip Bias (mV)	SetTipBiasVoltage	set_tip_bias: cfg.ramp.bias_v (V), applied immediately
StopNo_setVar	Stop #	G_StopNumber	opts.stop_number; editable
StartMeasurement Button	Start Measurement	StartMeasurement	start_ measurement(single=False)
setvar17	Attempts	G_PullOutAttempt	opts.pull_out_ attempt; editable
(new) SingleAttemptButton	Stop (Alt) +1	the Alt key MakeAttempt	request_stop start_ measurement(single=True)

Tip Bias is stored in volts (`cfg.ramp.bias_v`) and shown in millivolts. With **Bias** ticked in Keithley Controls it is quantised down to a multiple of 5 mV and sent to the box as well (`SetTipBiasVoltage`, `Controls_STMBJ.ipf:425-427`). A value beyond `limits.bias_max_v` (0.5 V by default) is refused with tip bias +900.0 mV exceeds the limit of +/-500 mV (`limits.bias_max_v`); raise it deliberately in the config if the experiment needs it — Igor had no software bias limit, and rungo's sweep to -1.1 V needs the limit raised on purpose (chapter 46). A second `TipBias_Inputs` entry existed on Igor's Inputs tab bound to the same global; one is enough.

The status line under this group is new: idle, or running: `<command>` while the worker is busy, during which Start Measurement, +1, Start Approach, Find Suppress, Start Writing and Reset DAQ Devices are greyed. Everything else — Stop, Kill Tasks, the entries — stays live.

42.8 Controls Igor created but never showed

Besides the two `_EChem` controls, the Igor panel macro creates `check08`, `setvar15`, `setvar5` and `setvar7` (`Windows_STMBJ.ipf:234-239`), all `disable=1`, untitled and unbound: leftovers of an earlier layout. They are not reproduced.

42.9 Tooltips, and the Button map

Hover over any of the above. A button's tooltip is built by `stmgui.controller.describe_command` from the method it calls: the docstring (what happens, which Igor procedure), then Python: with the controller method's file and line, the worker-thread `_impl` that does the work, Igor: the procedure, and Calls: every `stm`lab function reached, each as `file:line name`. Entries and checkboxes show the Igor global, the attribute they write, the display scale if any, and the limits. Igor only had a `help=` string on Start Writing.

The same information for every control at once is the **Button map** — the “Which code runs this?” button beside the status line at the bottom of the panel, or **Windows** → **Button map**: one row per command in `controller.COMMAND_MAP`, with the line numbers taken from the loaded code (press Refresh after editing). It is the table in chapter 50, kept alive. **Right-click** any control (macOS: Control-click or two-finger tap) and the **Inspector** window opens with that control's

text, staying open where a tooltip would vanish. The orange hint line under the status line says so.

The Voltage Offset panel and V0 tracking

Igor: Voltage_Offset() panel, Windows_STMBJ.ipf:378-405; FindOffset, Controls_STMBJ.ipf:498; OffsetVoltage / SaveOffset, Functions_STMBJ.ipf:920 / 993. Modules: stmgui/panels/voltage_offset.py, stmlab/vzero.py.

The lab calls this button **Find Zero**, and it is the first of the two calibrations pressed before every Igor run. What it finds is the applied bias at which no current flows — the amplifier chain’s input offset, expressed as an equivalent bias — and, optionally, keeps finding it as the run goes on. Chapter 21 is the physics; this chapter is the panel.

43.1 The panel

Igor’s panel was a single group box titled “Voltage Offset” with seven controls. Same seven here.

Igor name	Title	Bound to / does
TipBiasInterval	Tip Bias interval (mV)	cfg.vzero.interval_mv: the sweep runs from $-\text{interval}$ to $+\text{interval}$ (default 5)
NumPoints	Number of points	cfg.vzero.n_points: points <i>per polarity</i> ; the sweep has $2N$ (default 5, so 10)
NumPoints1	Frequency (traces)	cfg.vzero.every_n_traces: re-measure every N saved traces when V0 check is ON (default 50)
VzeroCheckBox	V0 check ON	opts.vzero_check
FindOffset	Find Offset	RigController.find_offset
Vzero	V0 (fit, mV)	display of opts.vzero_mv (Igor G_Vzero)
Vzero1	I0 (fit, uA)	display of opts.izero_ua (Igor G_Izero)

Igor’s V0 and I0 were editable SetVariables bound to the globals; here they are read-only displays, because a hand-typed V0 would be fed into the applied bias with nothing to say where it came from.

43.2 What Find Offset does

Igor (OffsetVoltage, Functions_STMBJ.ipf:920-982): redimension the waves to $2 \cdot \text{NumPts}$; build a ramp from $-|\text{Bound}|$ upward in steps of $|\text{Bound}| / \text{NumPts}$; for each point call HighResContact() (step in at 100 mV until above the conductance threshold), write the low bias, Sleep/

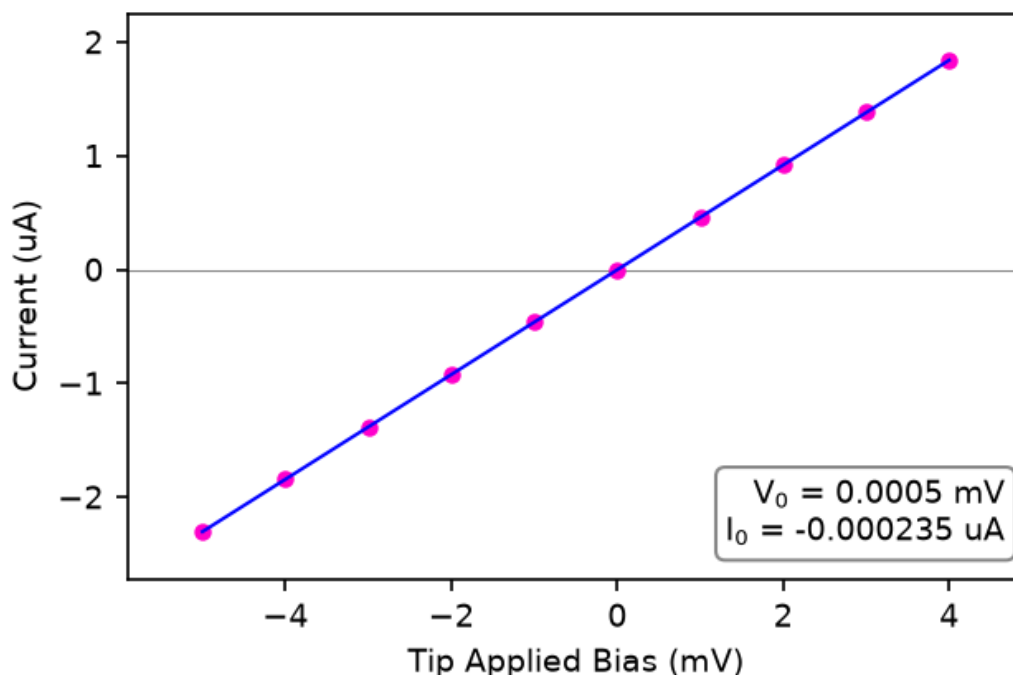
T 3 (3 ticks, 50 ms), ReadCurrentAndVoltage() (5000 samples); then CurveFit line of current against applied bias; $I_{\text{zero}} = W_{\text{coef}}[0]$ (the intercept, in μA), $V_{\text{zero}} = -I_{\text{zero}} / W_{\text{coef}}[1]$ (mV); finally write the original tip bias back.

Now (stmllib.vzero.measure_offset, called by find_offset on the worker thread):

1. Remember the original bias, `cfg.ramp.bias_v`.
2. Build the bias points: `bias_mv = -|interval| + (|interval|/N) × k` for $k = 0 \dots 2N-1$ — Igor's line 951, so the sweep ends one increment short of +interval, as Igor's did.
3. For each point: `_ensure_contact` (Igor's HighResContact) sets 100 mV and steps the piezo in 2 nm at a time until `Rig.in_contact()` — above `cfg.ramp.engage_g0` or a railed preamp. It refuses at the piezo ceiling with `vzero`: piezo at the ceiling with no contact; run the coarse approach before measuring the offset; Igor's loop had no exit on that side. Then the bias point is applied, `cfg.vzero.settle_s` (50 ms) elapses, and 5000 samples are read, the last 4200 averaged (the `settle_discard` tail rule every DC read uses).
4. The original bias is restored (Igor line 980).
5. `numpy.polyfit(bias_mv, current_a, 1)`: slope in A/mV, intercept `izero` in A; `vzero_mv = -izero / slope`. A zero slope raises `vzero`: flat $I(V)$ — no contact, or a railed preamp at every point.

The controller then records the result (`_record_vzero`): `opts.vzero_mv` and `opts.izero_ua = izero_a × 106` for the two displays; the result appended to `vzero_tracker.history`; (Saved, I0) appended to `izero_history`; and three events — `vzero` for the panel, `izero` for the Izero graph, `izero_time` for the Izero_Time graph. The piezo readout and slider follow the position `_ensure_contact` left the tip at.

You should see the History line $V_{\text{zero}} = +0.0005 \text{ mV}$ ($I_{\text{zero}} = -5.782\text{e-}10 \text{ A}$) (the simulator's numbers; a real amplifier gives millivolts and nanoamps), the two displays filling in, and the Izero window opening.



The Izero window after Find Offset on the simulator: the measured current at each applied bias (Igor's CurrentWave, pink markers), the fitted line (fit_CurrentWave, blue), and the V0 / I0 box Igor drew with a TextBox.

43.3 Why it is measured in contact

A line can only be fitted through a current that is measurable. Out of contact, 5 mV across a tunnelling gap gives picoamps of current and a slope made of noise. In contact the junction is a few k Ω , the current at 5 mV is microamps, and the intercept is clean. That is why every point re-checks contact first — a junction that breaks halfway through the sweep would otherwise fit a line through two regimes — and why the routine has to restore your bias afterwards: it has been driving the junction at voltages you did not ask for.

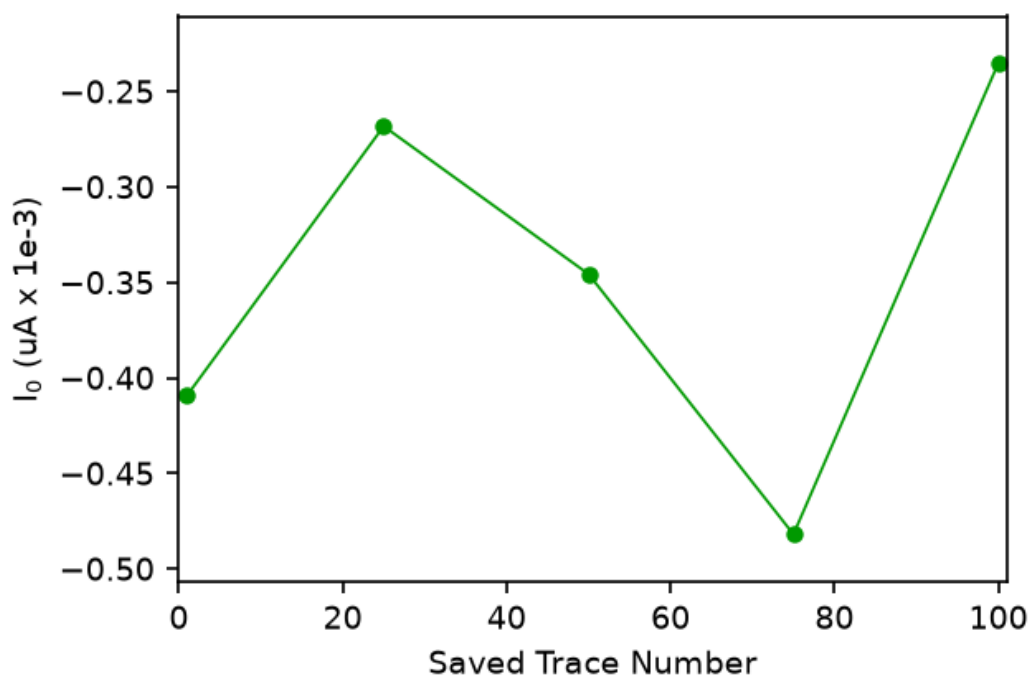
It is also why the button belongs *after* Start Approach in the lab sequence and before the run: the tip must be able to reach the surface, and the number is only worth having if it is applied to what follows.

43.4 V0 check ON

The checkbox is Igor's VzeroCheckBox, read in two places by CreateInputs and in one by SingleBreakJunctionAttempt:

- **Constant-bias mode** (Functions_STMBJ.ipf:1609-1614): with the box ticked the applied bias wave became $-(\text{TipBias} + \text{Vzero})/1000$ instead of $-(\text{TipBias})/1000$. Now: `_measure_impl` passes `vzero_tracker.offset_v` (V0 in volts, 0 if nothing has been measured) into `trace.build_ramp(bias_offset_v=...)`, which adds it to the DC baseline only. The alignment spike stays at +Tip Bias, as Igor's line 1631 wrote it, so trace alignment does not depend on the offset. The bias recorded per trace in the session file is `bias_v + offset`, so a later reader knows what was actually applied.
- **High-bias-hold mode** (lines 1595-1599): with the box ticked the hold bias *became* the offset, `HBBias = Vzero/1000` — the zero-field control for the high-field runs. Now: `_build_ramp` passes `opts.vzero_mv` into `ramps.build_hb_hold(vzero_mv=...)`, whose `meta["from_vzero"]` records that it did.
- **Push-pull, IV, AC hold**: no effect, in Igor or here.
- **After every saved trace** (SaveOffset, line 993, called from SingleBreakJunctionAttempt at 1649): Igor re-measured the offset every `G_VzeroFrequency` saved traces and appended the result to `OffsetWave`, `IzeroWave` and `PullOutNumberWave`, then called `CreateInputs` again so the new V0 was in the next trace's bias. Now: `VzeroTracker.maybe_measure (rig, Saved)` runs after each accepted constant-mode trace, measures when `Saved % every_n_traces == 0`, and the next `build_ramp` picks up `tracker.offset_v`. Each measurement goes through the same `_record_vzero`, so the panel, the Izero window and the Izero_Time window all update mid-run.

The tracker's history is written into the session file's summary as `vzero_history` (arrays of `vzero_mv`, `izero_a`, `timestamp`; chapter 47), which is the equivalent of Igor's three waves.



Izero_Time on the simulator with V0 check ON and Frequency set to 25 during a 100-trace run: one Find Offset before the run and four re-measurements, plotted against the Saved number at which each was taken. Igor's prescaleExp(left)=3 put the axis in $\mu\text{A} \times 10^{-3}$, kept here.

43.5 The history is the diagnostic

A single V0 is a correction. A *series* of them is a measurement of the rig: an offset that drifts steadily over a session says the amplifier is warming, the electrolyte is changing, or the tip is ageing — long before the histogram shows it. Igor plotted IzeroWave against PullOutNumber Wave for exactly this reason, and the Izero_Time window is that plot. Leave V0 check ON with a Frequency of 50 for any run that cares about the low-bias region, and look at the window before trusting the day's data.

43.6 Pitfalls

- **Pressing Find Offset before Start Writing** refuses with the output task is not running -- press Start Writing first.
- **Before Start Approach**, on a real rig, the fine piezo cannot reach the surface; _ensure_contact climbs to the ceiling and refuses (message above). On the simulator the surface happens to be within reach from park, so this step is optional there.
- **At the piezo floor.** The routine steps *in*; it does not step out. If the tip is already in hard contact it fits a line through a metallic contact, which is fine — a few $\text{k}\Omega$ is exactly what it wants.
- **Simulator numbers.** The simulated card has a current offset but no bias offset, so V0 comes out at microvolts of noise. Do not read the simulator's V0 as a test of the fit; read the Izero window's straight line as one.

- **Zero check on the Keithley** must be off (Keithley Controls) for the current to be real; Igor ticked it on at every Kill Tasks and left it to you to untick.
- **The offset is not the preamp zero.** `calibrate.measure_zero` (run at Start Writing) measures the amplifier's *output* at zero bias out of contact and stores it as `cal.current_zero_v`, which every conductance subtracts. `V0` is the *input* offset expressed as a bias, measured in contact, and is added to the applied bias. Both are needed; they correct different things.

The Electrochemistry panel

Igor: EChem() panel, Windows_STMBJ.ipf:436-473; EChem_Module.ipf. Modules: stmgui/panels/echem_panel.py, stmlab/echem.py.

Igor's EChem module did two unrelated things through one panel: it held a DC potential on a counter electrode while break-junction traces were taken (electrochemical gating), and it ran cyclic voltammetry by driving a triangle wave out of one card and recording the tip current. The panel has the same two group boxes — **Electrochemical Gating** and **Cyclic Voltammetry** — with Igor's controls in each. Chapter 15 is the physics and the command-line version (experiments/06_echem_gate_cv); this chapter is the panel.

Nothing in this panel has ever driven a real second card or a real cell from this code. The simulator exercises every line; the hardware paths are transcriptions of Igor's calls. Bring up on the scope first.

44.1 Electrochemical Gating

Igor name	Title	Igor proc	Does now
StartEChemWriteButton	Counter Electrode ON	CounterElectrodeOn → StartEChemWriting	RigController. counter_electrode_on
CounterElectrodeSet Var	Counter Electrode Bias (mV)	CounterElectrodeSet Var → WriteToCounter Electrode	set_counter_ electrode(mv); cfg.echem.gate_mv

Igor created an output task on the low-res card's ao1 (LowResCounterElectrodeOut = "dev2/ao1", Setup1_STMBJ.ipf:11) with a ± 5 V range, wrote the current G_CounterElectrodeBias, then greyed the button and enabled the entry (EChem_Module.ipf:41-48). Typing a new bias wrote it at once. StopWritingTasks and EChemResetDAQDevices reversed the enable states and reset the card.

Now, Counter Electrode ON calls stmlab.echem.make_counter_electrode (cfg).on() — a SimulatedCounterElectrode in simulate, otherwise a CounterElectrode that opens an nidaqmx task on {channels.low_res_device}/{echem.counter_electrode_channel} with min_val=-5.0, max_val=5.0, as Igor's MXCreateA0VoltageChan(-5,5) — and then writes the bias currently in the entry. If channels.low_res_device is None, which is this one-card rig's default, it refuses with EChemError: no low-res device configured (channels.low_res_device is None); the counter electrode lives on the second card. —simulate pretends a Dev2 so the panel can be exercised.

After ON the button greys, the entry wakes, and every committed value goes to CounterElectrode.set_mv, which writes mv/1000 volts and stores the value in **cfg.echem.gate_mv** — so

the gate potential travels in the `config.json` of every session file written while it is applied. That is the point of doing it through the config rather than a local variable: a trace file taken under gate says so.

The entry is refused before ON with press Counter Electrode ON first. **Kill Tasks** releases the gate (`CounterElectrode.off()`, which writes 0 V, closes the task and zeroes `cfg.echem.gate_mv`) and puts the panel back to its starting state; **Reset DAQ Devices** does the same after resetting the cards, as Igor's `EChemResetDAQDevices` set `CounterElectrodeBias = 0` (`Controls_STMBJ.ipf:228`). Both emit a gate event with `None`, which is what the panel listens for.

A gate that should outlive the process — set from a script, then left standing while the command-line experiment runs — is `off(zero=False)` in the module; the GUI never does that, because the panel is the process.

44.2 Cyclic Voltammetry

Igor name	Title	Bound to
SetGain	Gain ($\log(V/A)$)	<code>cfg.keithley.gain_exponent</code> and <code>cfg.cal.preamp_gain_v_per_a</code> (the same entry as on the main panel)
CVAcquisitionRateSetVar	CV Acquisition Rate (samps/s)	<code>cfg.echem.cv_rate_hz</code> (50 ... 40000, default 1000)
NumCVCyclesSetVar	Number of Cycles	<code>cfg.echem.cycles</code> (default 1)
ScanRateSetVar	Scan Rate (mV/s)	<code>cfg.echem.scan_rate_mv_per_s</code> (0 ... 2000 in steps of 25, default 100)
VoltagePeakOneSetVar	Voltage Peak 1 (V)	<code>cfg.echem.peak_one_v</code> (−2.5 ... 0 in steps of 0.25, default −1)
VoltagePeakTwoSetVar	Voltage Peak 2 (V)	<code>cfg.echem.peak_two_v</code> (0 ... 2.5 in steps of 0.25, default 1)
StartCVHighResButton	CV HighRes	<code>start_cv("highres")</code>
StartCVLowResButton	CV LowRes	<code>start_cv("lowres")</code>

Igor's five `SetVariable` procedures (`VoltagePeakOneSetVar` and friends, `EChem_Module.ipf:414-466`) each copied the typed value into its global; the bindings do that directly.

44.2.1 The two buttons

Igor enabled both CV buttons only after `SetGain` had run (`Controls_STMBJ.ipf:314-315`) and disabled them again at `StopWritingTasks` (lines 204-205). The reason is worth keeping: a voltammogram is a current, and the current is the amplifier's output divided by a gain the analysis has to know. So the buttons are greyed until the Gain entry (either copy) has been committed once in this launch, and greyed again by `Kill Tasks`.

CV HighRes (`CVcurveHighRes`, `EChem_Module.ipf:106`) drove the triangle out of the *high-res card's bias channel* — `dev1/ao1`, the same output the break-junction bias uses — with the tip as working electrode, and read both `ai0` (the electrode voltage) and `ai1` (the tip current). **CV LowRes** (`CVcurveLowRes`, line 234) drove it out of the low-res card's counter-electrode channel with a ± 10 V range and read the tip current only. `stmlab.echem.run_cv(cfg, kind)` does the same two things:

- `kind="highres"`: `build_cv_ramp_highres` builds the wave (below); on hardware `_run_cv_` hardware opens an AO task on `channels.path(ao_bias)` with a range wide enough for both

peaks, an AI task on both input channels, both finite at `cv_rate_hz`, and — one deliberate improvement — starts the AI on the AO's start trigger, where Igor started the two tasks back to back and lived with the skew. Each cycle returns a `CVCycle` with `applied_v`, `tip_current_v` (raw preamp volts), `we_voltage_mv` (the measured electrode voltage) and `keep`.

- `kind="lowres"`: `build_cv_ramp_lowres` (5000 samples of 0 V first, then the triangle with a longer repeated tail, Igor lines 251-286); the AO task on the second card's counter-electrode channel at ± 10 V; tip current only, `we_voltage_mv` is `None`.

In `simulate`, `_run_cv_simulated` replaces the cell with a model: a 1 μ F double-layer capacitance whose current flips sign with the scan direction (the rectangular part of the loop) and one reversible couple with anodic and cathodic peaks 60 mV apart around +0.2 V, plus noise. Not chemistry — shape, enough to test every line of the analysis and the plot.

44.2.2 The ramp and the keep mask

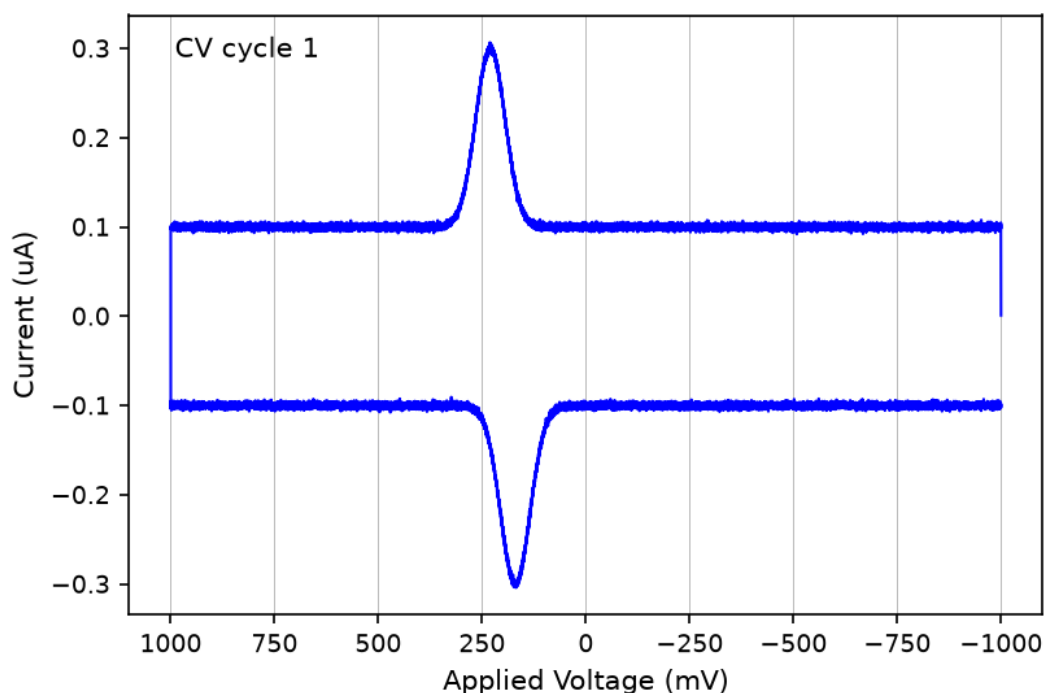
Igor's high-res wave (lines 127-158) was $0 \rightarrow \text{Peak 1} \rightarrow 0 \rightarrow \text{Peak 2} \rightarrow 0$, then the first half again, and it NaN-ed the first quarter (the switch-on transient) and the repeated half so that what remained was one closed cycle starting and ending at the same potential. `build_cv_ramp_highres` builds exactly that wave, with the number of samples per leg from $2 \times |\text{peak}| / \text{scan_rate} \times \text{cv_rate_hz}$, and returns beside it a boolean `keep` that is `False` where Igor wrote NaN. Keeping the mask *beside* the complete record instead of destroying the samples means a later reader can second-guess the choice. A ramp with fewer than four samples per leg is refused: CV ramp too short; slow the scan rate or raise `cv_rate_hz`.

44.2.3 What the button does around the sweep

`StartCVHighResButton` (`EChem_Module.ipf:364-387`) checked the main panel's Zero Check box, sent `C0X` to lift zero check if it was ticked, ran the sweep, and sent `C1X` to put it back. `RigController._cv_impl` does the same: if `opts.zero_check` is set and the Keithley is connected, zero check is lifted, the sweep runs inside a `try`, and zero check is restored in the `finally`.

One refusal Igor did not have: on hardware, **CV HighRes while the output task is running** is refused with CV HighRes drives the junction-bias channel; Kill Tasks first (Igor reset the card afterwards). Two tasks cannot own `dev1/ao1` at once; Igor let them collide and then `MXResetDevice`'d the card. In `simulate` there is nothing to collide with and the refusal is not raised.

After the sweep, `storage.save_cv_cycles` writes `data/<date>/cv_<kind>_<HHMMSS>.h5` (layout in chapter 47), the cycles are kept as `controller.last_cv`, and a `cv` event opens or redraws the Cyclic Voltammogram window.



The Cyclic Voltammogram window after CV HighRes on the simulator: tip current in μA against applied potential in mV, x axis reversed as Igor's SetAxis/A/R bottom had it, the masked switch-on quarter and repeated tail left out. The rectangle is the double-layer current; the pair of peaks is the model couple.

44.2.4 Reading the numbers

The plotted current is $\text{tip_current_v} / \text{cal.preamp_gain_v_per_a}$, in μA . With the default gain of 10^6 V/A , $0.1 \mu\text{A}$ of double-layer current is 0.1 V at the ADC; a real cell with a large electrode can put out far more, and a railed channel shows as a flat top at $\pm 10 \text{ V}$. Lower the gain (the Gain entry on this panel) before a CV on a real cell, and remember that the same entry sets the gain the break-junction conductance divides by — put it back before the next Start Measurement, or `validate()` will tell you about the mismatch in the History.

44.3 Between the two groups

Igor's gating and CV were independent — you could gate while measuring traces and never run a CV, or run CVs with no gate. The same holds here, with one practical link: on hardware a CV HighRes needs the output task off, so the sequence for a gated voltammogram on this rig is Counter Electrode ON, set the bias, Kill Tasks if the task was running, CV HighRes, then Start Writing again for traces.

The graph windows and the History window

Igor: the Graph macros in Windows_STMBJ.ipf:33-131, 407-434 and 475-489; the history area. Modules: stmgui/graphs.py, stmgui/panels/history.py.

Igor's experiment showed its data in nine small graph windows, each a macro that displayed one or two waves with fixed axes, and printed its commentary into the history area. All ten are here. Each graph is a matplotlib figure in its own Tk window (GraphWindow), created the first time it is needed, hidden rather than destroyed when closed, and listed under the **Windows** menu. Every figure in this chapter was produced by the window itself, on the simulator, by `manual_build/make_figures.py`.

45.1 How the windows get their data

The controller posts events; `GraphSet.on_event` routes them. The table is `GraphSet.FEEDS`:

Event	Windows fed	Fired by
highres trace	HighRes PullOutLowG, PullOutGvsE, AuAuConductanceLevel, SenseInDisplay	every background read every generated trace, saved or not
hist	LogHistOfBlock	every 100 saved traces, and the partial block at the end of a run
izero	Izero	Find Offset, and each periodic re-measure
izero_time cv	Izero_Time CyclicVoltammogram	the same each CV run

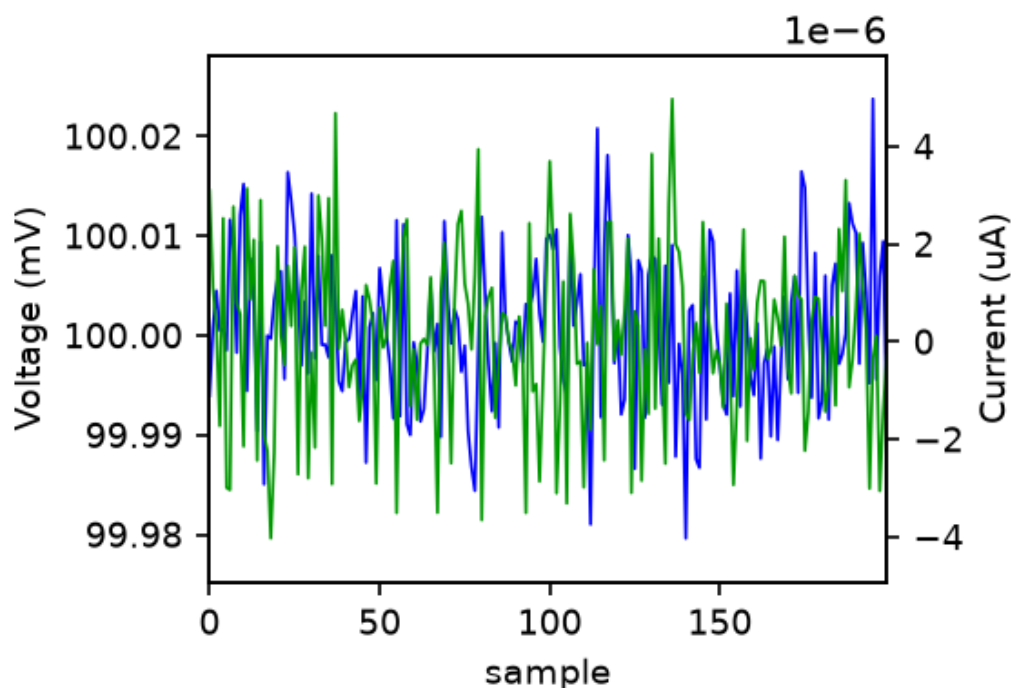
The first time an event reaches a window that does not yet exist, the window is created and — for the first-listed window of each event, and always for `Izero`, `LogHistOfBlock` and `CyclicVoltammogram` — shown. That is Igor's `DoWindow/F` name; `if (V_flag==0) Execute "name()"` idiom, which opened a graph on first use and thereafter just brought it forward. The three secondary trace windows (`PullOutGvsE`, `AuAuConductanceLevel`, `SenseInDisplay`) are created and kept up to date but not raised, to keep the first run from covering the screen; open them from the **Windows** menu.

Updates happen on the Tk thread, at most every 50 ms, from the event pump (chapter 48). A run on the simulator generates traces faster than that, so the pull-out windows show every trace but not every one is rendered before the next arrives; on hardware, at one trace per second or so, every one is.

45.2 HighRes

Igor: HighRes(), Windows_STMBJ.ipf:33 — VoltageIn (blue) and VoltageInLowRes (orange) against the left axis in mV, CurrentInLowRes (pink) and CurrentIn (green) against the right axis in μA ; the last background read from both cards.

Now: the last background read from the one card: junction voltage in mV ($\text{voltage_input_sign} \times V \times 1000$) on the left axis in Igor's blue, tip current in μA ($\text{volts_to_amps}(I) \times 10^6$) on the right axis in Igor's green, against sample number. **Fast Read Wave Size** sets the length. Axes autoscale to the data with a 10 % margin and never use matplotlib's offset notation, so 100.02 mV reads as 100.02, not +0.02 over $1e2$. There is no low-res pair because there is no low-res card.



HighRes on the simulator, in contact after Start Approach: 200 samples of junction voltage around -100 mV (the bias is applied as $-\text{TipBias}$, so the measured, sign-corrected voltage is -100 mV) and a current of about $10 \mu\text{A}$. Retracted, the current trace is noise around zero.

45.3 PullOutGvsE, PullOutLowG, AuAuConductanceLevel

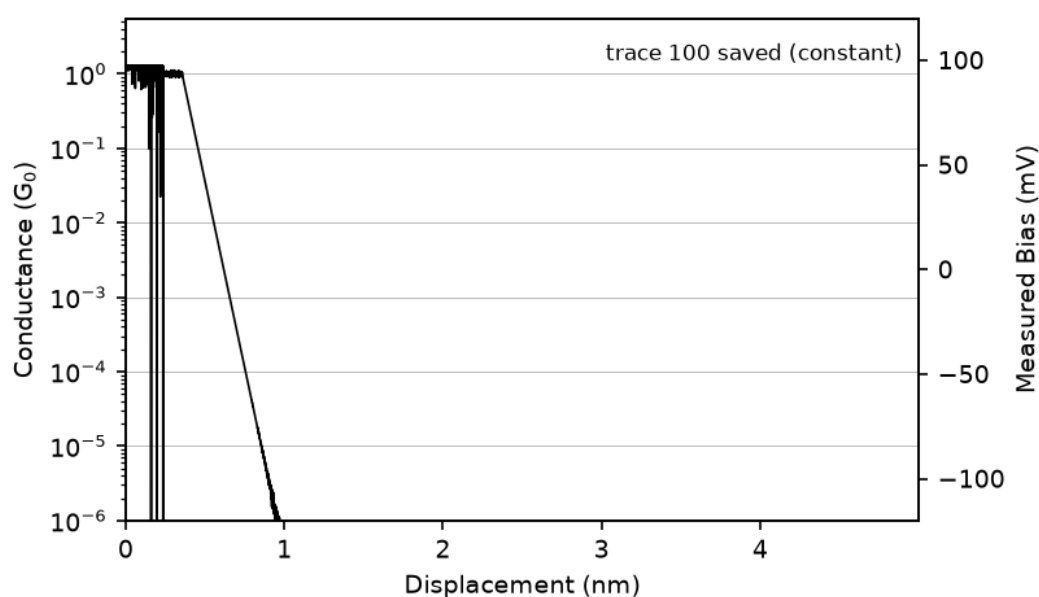
Three views of the same wave. Igor's PullOutConductance was the conductance of the last generated trace in units of G_0 against displacement in nm; GenerateTrace filled it before TestTrace judged the trace, so rejected traces were shown too. The same here: every trace event carries g_0 (from TraceRecord.conductance_g0, i.e. the measured current over the measured voltage), disp_nm (the commanded piezo retraction from the contact point, $\text{piezo_volts_to_nm}(\text{start} - \text{piezo})$), bias_mv (the measured junction voltage), piezo_nm (the absolute commanded position), the trace number, the mode, and whether it was saved. Each window prints trace N saved (constant) or trace N rejected in its corner.

Window	Igor macro	Axis	Igor range
PullOutGvsE	PullOutGvsE(), line 52	linear G0	from 0 upward, autoscaled
PullOutLowG	PullOutLow(), line 66	log G0	1e-6 to 5.58676
AuAuConductanceLevel	AuAuConductanceLevel(), line 86	linear G0	1e-5 to 5

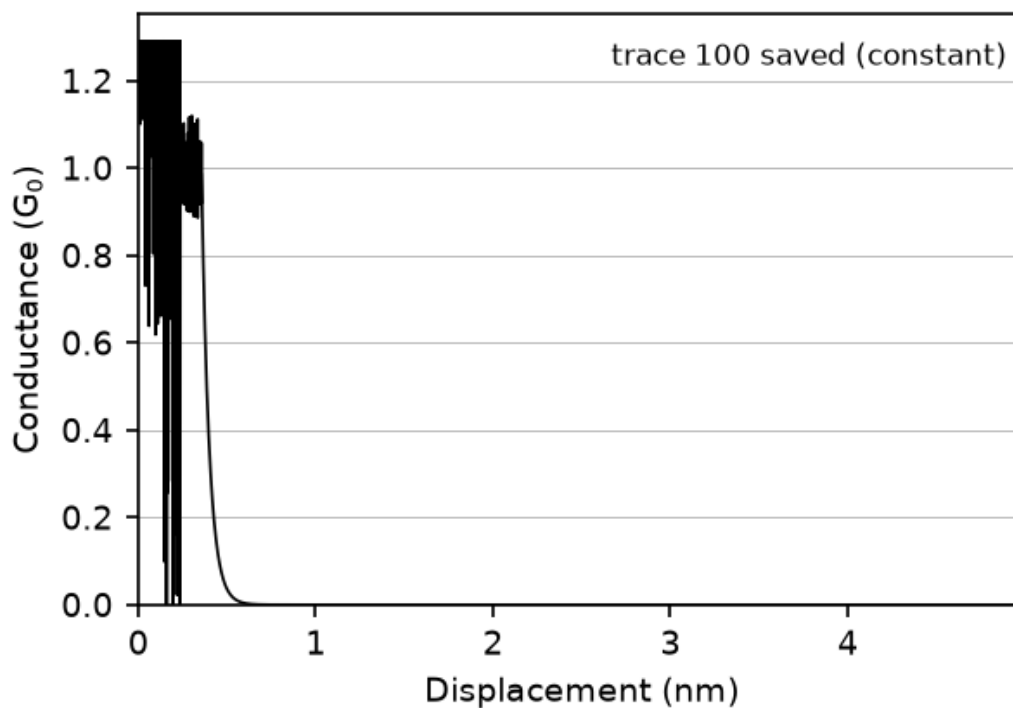
PullOutLowG is the one you watch. The log axis spans the whole trace from metallic contact through the molecular plateau to tunnelling; the odd upper limit is Igor's own `SetAxis left 1e-06, 5.58676`. Igor also appended **PullOutVoltage** — the measured bias in mV — on a right axis and hid it (`hideTrace(PullOutVoltage)=1`); the same trace is here, hidden, with a **show PullOutVoltage** checkbox under the plot to reveal it. Igor's window was titled "PullOutLowG"; so is this one.

PullOutGvsE is the linear view of the plateau region, y from 0 to a little above the trace's maximum (Igor: `SetAxis left 0, *`).

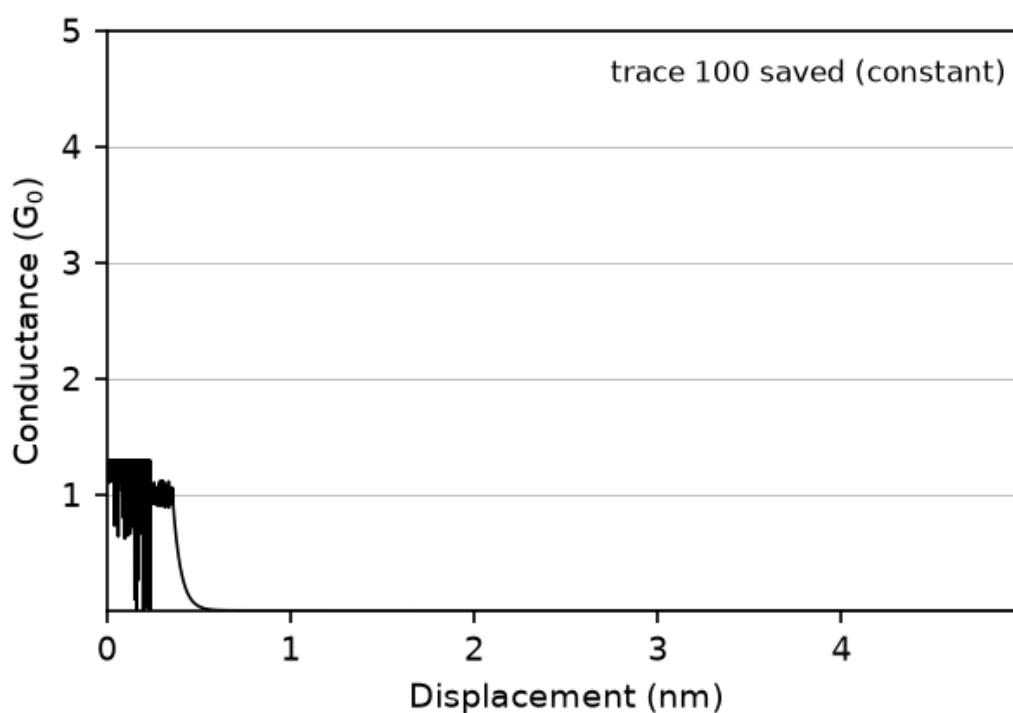
AuAuConductanceLevel is Igor's third view, linear from 1e-5 to 5 — in practice the 1 G0 gold plateau at the left edge of the trace, which is what the name says.



PullOutLowG on the simulator: the last saved trace of a 100-trace run. Metallic contact at the left, the gold plateau at 1 G0, then the exponential tunnelling descent to the noise floor; this particular trace caught no molecule, and one that does shows a second plateau three to four decades down. The measured-bias trace on the right axis is hidden, as in Igor.



PullOutGvsE, the same trace on a linear axis from 0.



AuAuConductanceLevel, the same trace between $1e-5$ and 5 G_0 .

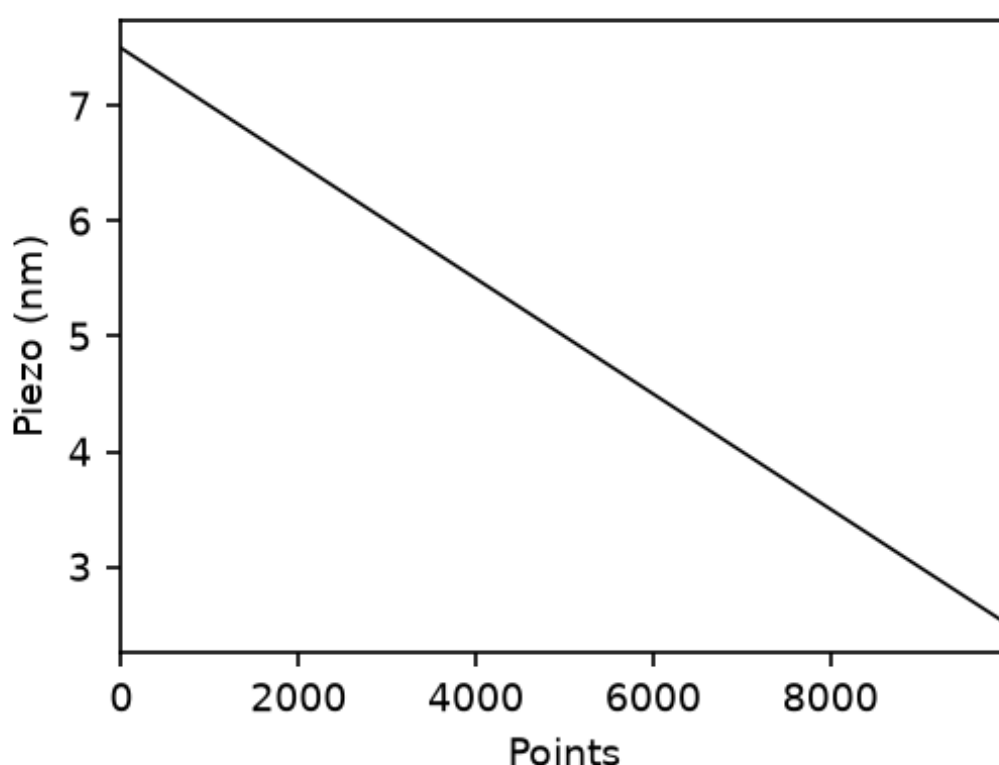
For the four ramp modes the displacement axis is still the commanded piezo position relative to contact, which for a push-pull cycle goes negative and comes back, and for a hold stays flat while the bias does the work. `TraceRecord.displacement_nm`, which assumes a constant pull

rate, is *not* used for these windows for exactly that reason; the ramp's own waveform is.

45.4 SenseInDisplay

Igor: `SenseInDisplay()`, line 100 — `SenseIn`, the piezo position in nm read back from the low-res card's sense input, against points.

Now: the *commanded* piezo position in nm for the last trace, against sample number. This rig has one card and no sense line; the number shown is the same tracked command every safety check trusts. A straight descending line is a constant pull; a push-pull cycle shows its excursions; a hold shows a flat section. If you ever add a sense input, `Rig.play` returns the record and this window is where the readback belongs.



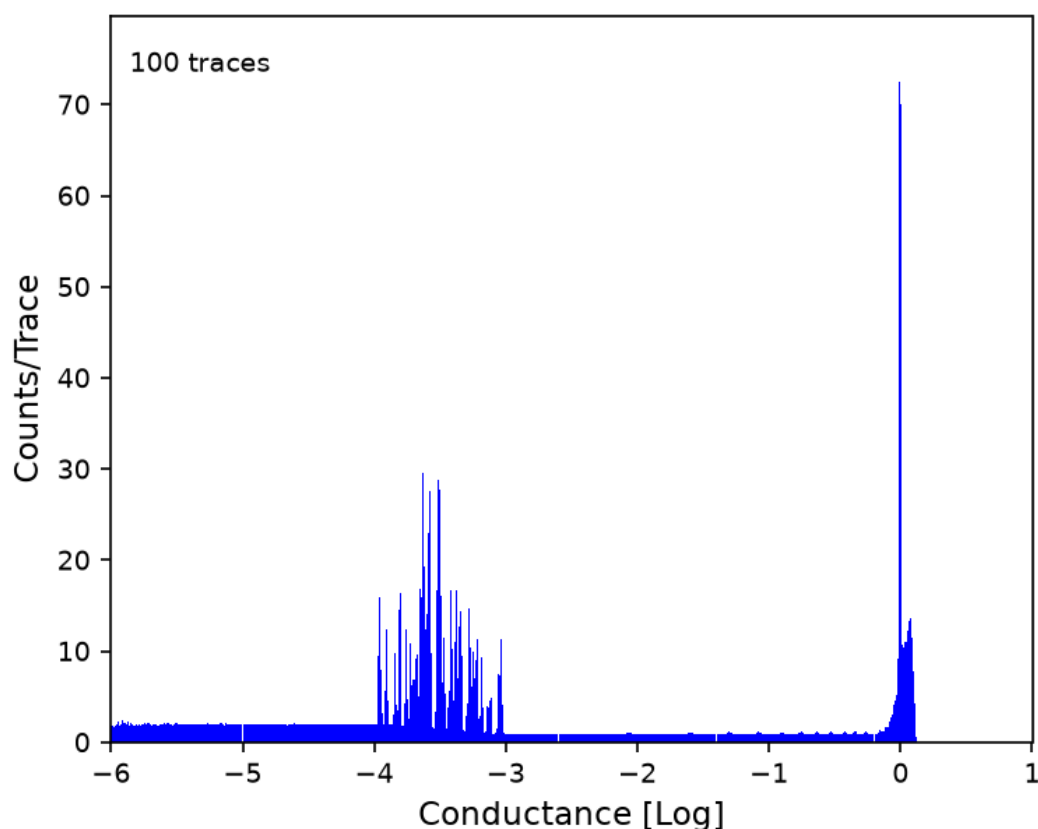
SenseInDisplay for a constant pull: the commanded piezo descending at 20 nm/s from the contact point, 10 000 points at 40 kHz for a 5 nm excursion.

45.5 LogHistOfBlock

Igor: `LogHistOfBlock()`, line 113 — `LogHistWave` as bars (`mode=7`, `hbFill=5`, blue), "Counts/-Trace" against "Conductance [Log]", axes $-6 \dots 1$ and $0 \dots 40$. `LogHistFromBlocks` (`Functions_STMBJ.ipf:785`) built it from each 100-trace `ConductanceBlock` as it was saved: keep 95 % of each trace, smooth, estimate the floor from the 95-96 % window, drop the trace if the floor is above the zero cutoff, subtract, smooth, $\log_{10}$, histogram into 1000 bins from -8 to 2 , divide by 100.

Now: `analysis.log_histogram` on the block, called by the controller when the block reaches 100 saved traces (`HIST_BLOCK`) and again for whatever is left when a run ends (that second call is

a GUI addition; Igor showed nothing until a block was complete). The zero cutoff is `cfg.ramp.break_g0`, Igor's `G_EndOfTraceNoiseThreshold`. For the ramp modes the hold or push-pull section is removed first — Igor's `Deletepoints StartDelete, NumberToDelete` — using the ramp's segment table: everything between the end of the initial pull and the start of the final pull. The bars are Igor's blue; x is fixed at $-6 \dots 1$; y starts at $0 \dots 40$ and grows if a peak exceeds it. The corner text says how many traces the block holds and whether it is partial. With **Save Hist** ticked, each block is also written as a CSV (chapter 47).



LogHistOfBlock after the first 100 saved traces on the simulator: the 1 G0 gold peak at $\log G = 0$, the simulator's molecular peak around -3.5 , and the tunnelling background.

The histogram is the measurement. One trace proves nothing; a peak that recurs across a hundred does. Chapter 10 says why the floor estimate and the cutoff gate matter, and what a missing gold peak means (usually the gain).

45.6 Izero and Izero_Time

Igor: `Izero()`, line 407 — CurrentWave against TipBiasRamp as markers (pink) with `fit_Current Wave` (blue), "Current (μA)" against "Tip Applied Bias (mV)", a zero line, and a TextBox with `G_Vzero` and `G_Izero`. `Izero_Time()`, line 421 — `IzeroWave` against `PullOutNumberWave` as connected markers (green), " $I0$ ($\mu\text{A} \times 10^{-3}$)" against "Saved Trace Number".

Now: `Izero` draws each `VzeroResult` from Find Offset or a periodic re-measure — points at `(bias_mv, current_a  $\times 10^6$ )`, a `numpy.polyfit` line through them, and the `V0 / I0` box. `Izero_Time` draws the controller's `izero_history`, one (Saved, `I0`) pair per measurement, in $\mu\text{A} \times 10^{-3}$

as Igor's `prescaleExp(left)=3` had it. Chapter 43 shows both figures and explains what to read in them.

45.7 Cyclic Voltammogram

Igor: `CyclicVoltammogram()`, line 475 — TipCurrent against CVAppliedRamp, "Current (μA)" against "Applied Voltage (mV)", x reversed (SetAxis/A/R bottom), a "CV cycle N" text box.

Now: every cycle of the last CV, current `tip_current_v / cal.preamp_gain_v_per_a` in μA against applied potential in mV, masked samples left out, earlier cycles in a lighter blue and the last cycle in Igor's blue, x reversed, the cycle number in the corner. Chapter 44 has the figure.

45.8 The History window

Igor printed to its history area — every `Print`, every command echo — and that record was the run's log. The **History** window is that: a scrolling text with one line per log record from the `stmlab` and `stmgui` loggers at INFO and above, routed there by `controller.QueueLogHandler` as log events, plus every error event. Each line is stamped with the wall-clock time, then the level letter and the logger name, then the message:

```
15:28:30 -> Start Measurement: stmgui/controller.py:722 RigController._measure_impl
15:28:30 I stmlab.storage: writing to ../constant_152830_from1.h5
15:28:30 W stmgui.controller: config: hv_amp_gain is unknown; ...
15:28:31 ERROR Start Measurement: the output task is not running -- press Start Writing first
```

The `->` line is written by the worker as each command starts: the button name, then the file, line and function of the Python that is about to run — so the History is also the answer to "which function did that button call". W lines are amber, E and ERROR lines red. The buffer keeps the last 5000 lines. Everything the chapters of this manual quote as "in the History" is a line here; if you want it on disk too, launch with `-v` and redirect the console, or read the session file's `run_stats.json` (chapter 47), which keeps the counts.

45.9 Placement and size

Igor positioned each window with its `Display /W=(left, top, right, bottom); GraphSet.ORDER` keeps those origins, scaled to today's screens, and each window's `igor_geometry` is a size in the same spirit — HighRes small, LogHistOfBlock large. Drag them where you like; positions are not remembered between launches, as Igor's were in the experiment file. The figures resize with their windows.

Menus and macros: rungo, LateralEXPT, the X piezo, config files

Igor: the Macros menu, Setup1_STMBJ.ipf:50; rungo(), :138; LateralEXPT(), :91; SetupXPiezo / Move XPiezo / MoveXPiezoToZero, NanoPZ_Actuator_Functions_STM.ipf:25-91. Module: stmgui/app.py (menus), stmgui/controller.py (the macros).

Igor's menu bar had one item of its own — **Macros** → **STM Break Junction Measurement**, which ran `InitializeExperiment` — and two procedures that were only ever typed into the command line: `rungo()`, the bias series, and `LateralEXPT(...)`, the monolayer walk. The rebuild has four menus. This chapter is what is in them.

46.1 File

Item	Does
Load config JSON...	<code>RigConfig.from_json</code> into the live config; every panel refreshes
Save config JSON...	<code>RigConfig.to_json</code> of the live config
Open data folder	opens <code>data/<today>/</code> in the file browser
Quit	<code>controller.shutdown()</code> — Kill Tasks, release the Keithley — then closes

A saved config is the complete `RigConfig` — channels, limits, ramp, calibration, actuator, the four mode configs, `vzero`, `keithley`, `echem`, `xpiezo`, the `simulate` flag and notes — as indented JSON. It is the same file `run.py <experiment> --config my.json` reads and the same dictionary every session file carries as `config_json`, so a config tuned at the panel runs unchanged from the command line, and a command-line config loads unchanged into the panel. What it does *not* contain is `GuiOptions`: `Saved`, `Stop #`, the checkbox states, Piezo offset, Fast Read Wave Size. Those are panel state, not rig description, and start fresh each launch (chapter 48 lists them).

Loading replaces every section of the live config in place, so the controller, the open Rig and the tracker all see the new values at once. Load before Start Writing; a config loaded under a running task takes effect at the next command that reads each field, which is consistent but rarely what you meant.

46.2 Macros

Item	Igor	Does
STM Break Junction Measurement	<code>InitializeExperiment</code>	raises the main panel

Item	Igor	Does
Voltage Offset panel	Voltage_Offset()	raises it (chapter 43)
Electrochemistry panel	EChem()	raises it (chapter 44)
rungo() bias series...	rungo()	the bias campaign, below
LateralEXPT(Z, X, XFreq)...	LateralEXPT	the monolayer walk, below
MoveXPiezo(nm)...	MoveXPiezo	relative lateral move
MoveXPiezoToZero()	MoveXPiezoToZero	lateral piezo to 0 V
Stop (Alt)	GetKeyState	request_stop

46.2.1 rungo(): the bias series

Igor (Setup1_STMBJ.ipf:138-179) built three waves of fourteen entries: bias = -100, -200, ... -900, then -100, -900, -1000, -1100, -100 mV; StopNumWave[i] = (i+1) × 1000 + 1; StartNumWave = StopNumWave - 1000. It then found where the current G_PullOutNumber fell in StartNumWave (FindLevel/EDGE=1) and, from that index on, set G_TipBias and G_StopNumber and called StartMeasurement — so a campaign interrupted after 3400 saved traces resumed at step 3, bias -400 mV, and ran it to

1. Two PiezoStepApart calls and a FindSuppress per step were commented out. Alt between steps aborted. At the end: SaveExperiment and StopWritingTasks.

Now, Macros → rungo() asks two questions — the bias list in mV, comma separated, pre-filled with Igor's fourteen values, and the number of traces per bias, pre-filled with 1000 — and calls RigController.rungo(biases_mv, counts_each). _rungo_impl computes the same stop_nums and start_nums, resumes from the last step whose start number is at or below the current Saved (the FindLevel logic), and for each remaining step logs rungo step i: tip bias ±... mV, stop # n, sets Tip Bias through the same _set_tip_bias_impl the panel entry uses, sets Stop # (the panel entry follows), and runs _measure_impl — a full Start Measurement, with its own session file per step. Stop ends the campaign after the current step's attempt (rungo aborted at step i); otherwise rungo complete.

Two things to set first:

- **The bias limit.** Igor's list reaches -1100 mV; the package's limits.bias_max_v is 0.5 V and set_tip_bias refuses beyond it (chapter 42). Raise it deliberately in a config file ("bias_max_v": 1.2) before a full rungo, or give a shorter list. The campaign stops with the refusal in the History at the first step it cannot apply.
- **Saved.** The resume logic reads it. To start a fresh campaign, set Saved to 1; to resume, leave it.

experiments/08_bias_series is the command-line version of the same campaign, with one file per bias and a --resume flag that reads the files instead of a counter; chapter 17. The GUI's rungo does not read those files and they do not read the GUI's.

46.2.2 LateralEXPT(ZDistance, XDistance, XFreq): the monolayer walk

Igor (Setup1_STMBJ.ipf:91-136) looped until G_StopNumber reached 12201: add XFreq to G_StopNumber; print the X piezo voltage; Delay(500); StartMeasurement (so XFreq traces at

this site); three StepActuatorApart calls with Delay(100) after each (withdraw in Z); MoveXPiezo(XDistance) and Delay(100); set G_ActuatorStepSize = 5, SetPiezoBiasFromSlider("", 0, 0) (fine piezo to 0 V), ApproachButton (coarse approach back into current), Delay(100). ZDistance set the actuator step size for the withdraw. Alt broke the loop.

Now, Macros → LateralEXPT asks for ZDistance (actuator steps per withdraw, pre-filled 10), XDistance (nm per site, pre-filled 200) and XFreq (traces per site, pre-filled 250), and calls lateral_expt(z, x, x_freq) with Igor's final_stop_number of 12201. _lateral_impl needs the output task and mirrors the loop: Stop # = Saved + XFreq; log the X piezo voltage; _measure_impl; Rig.withdraw() (fine piezo to park — the interlock needs it before a coarse step, which Igor's SetPiezoBiasFromSlider(0) only did *after* the withdraw); three Rig.coarse_step(closer=False) with 0.1 s pauses on hardware; the X piezo move; step size 5; fine piezo to 0 V; bias re-applied (coarse_approach watches the current, so the bias must be on); _approach_impl; step size back to ZDistance. Stop ends it after the current site. The final History line is LateralEXPT done at Saved = n.

Every site's traces go into that site's session file (one per _measure_impl call), whose igor_globals.json records the Stop # of the site, but not the X position — experiments/07_lateral_monolayer, the command-line version (chapter 16), writes x_position_nm into each site file and is the better tool for a map you intend to analyse by position. The macro is here because Igor had it.

46.2.3 MoveXPiezo(nm) and MoveXPiezoToZero()

Igor (NanoPZ_Actuator_Functions_STM.ipf:25-91): SetupXPiezo created an output task on dev2/ao0 with a 0 ... 10 V range and drove it to 0; MoveXPiezo(XDistance) added XDistance to G_XPiezoPosition and wrote position / K_XPiezoScale volts (522 nm/V); MoveXPiezoToZero wrote 0. Setup1_STMBJ.ipf:86–88 called setup, stop and zero at load. No range check.

Now: the first lateral command creates stmlab.xpiezo.make_xpiezo(cfg).setup() — SimulatedXPiezo in simulate, otherwise an nidaqmx task on {channels.low_res_device}/{xpiezo.channel} over xpiezo.min_v .. max_v (0 ... 10 V) driven to 0 — and keeps it until Kill Tasks. MoveXPiezo asks for a relative distance in nm and calls XPiezo.move_nm, which computes the target voltage at xpiezo.nm_per_volt (522) and **refuses** a move outside the channel's range with lateral move to ... nm needs ... V, outside [0.0, 10.0] V. Re-centre the sample or zero the X piezo. — Igor did not check, and 12 200 accumulated 200 nm steps is 4.7 V of a 10 V range, so the headroom is real but finite. MoveXPiezoToZero calls XPiezo.zero. Both post an xpiezo event with the new position; the History logs X piezo at ... nm (... V). Without a second card configured, XPiezoError: no low-res device configured (channels.low_res_device is None); the X piezo lives on the second card.

46.2.4 Stop (Alt)

The same request_stop as the Stop button, the Alt keys and Escape (chapter 41). It is in the menu so it can be reached when the main panel is behind a graph.

46.3 Windows

The nine graph windows by their Igor names — HighRes, PullOutGvsE, PullOutLowG, AuAu-ConductanceLevel, SenseInDisplay, LogHistOfBlock, Izero, Izero_Time, CyclicVoltammogram — then History, then **Button map**. Selecting one creates it if it has never been shown and raises

it. Chapter 45 for the graphs and History.

Button map (control → Python → Igor) is one window listing every command: the panel control, the RigController method with its file and line, the worker-thread `_impl` with its file and line, the Igor procedure it mirrors, and each stmlab function it reaches with file and line. It is generated from `stmgui.controller.COMMAND_MAP` and the loaded code when it opens; **Refresh** regenerates it. Hovering a control on a panel shows the same entry as a tooltip.

46.4 Help

Button map — the same window, for people who look in Help first. **About / where the manual is** shows the package path, the manual (`STMLAB_GUI_Manual.pdf`, source `manual/4x_gui_*.md`), and the data path.

46.5 Keyboard

Key	Does
Alt (left or right)	Stop
Escape	Stop
Return, in an entry	commit the value
Tab / click elsewhere	commit the value (focus-out)
Up / Down, in a spinbox entry	step by Igor's increment

There are no other accelerators. Igor's "Alt" was `GetKeyState(0) == 2` polled inside the loops; here it is a key binding on every window that sets the same flag the loops poll.

46.6 What the menus do not do

There is no "save experiment" — Igor's `SaveExperiment` wrote the whole `.pxp`; here every result is already on disk in `data/` as it is taken (chapter 47), and the config can be saved from File. There is no "load data" — analysis happens outside the GUI, with `storage.Session` and the experiments' analysis scripts (chapters 10-17). And there is no preferences dialog: everything adjustable is a config field or a panel control, and both round-trip through the JSON.

What the GUI writes, and how to read it back

Igor: MakePath, Functions_STMBJ.ipf:200; SavePullOut, :438; LogHistFromBlocks, :785. Modules: stmgui/controller.py (naming, headers), stmlab/storage.py (formats).

Igor wrote its data as .ibw blocks — one two-dimensional wave per hundred saved traces, each column a trace with twenty parameters stacked on top of the conductance — into a dated folder under D:Experiments: (DataDir, MakePath). The GUI writes HDF5 session files, CSV histograms and HDF5 voltammograms into a dated folder under data/. This chapter is the exact layout of each, how to read them, and how they relate to what Igor kept.

47.1 Where

GuiOptions.data_dir (default data, relative to the package folder) plus a folder named for the day, YYYYMMDD — RigController.data_dir(), Igor's G_PathDate. Created on first use. **File** → **Open data folder** opens today's. Set data_dir to an absolute path in a script or a custom launcher to write elsewhere; there is no panel control for it.

47.2 Session files

Name: <mode>_<HHMMSS>_from<Saved>.h5, where mode is constant, push_pull, iv, ac_hold or hb_hold, the time is when Start Measurement (or +1) was pressed, and Saved is the Saved counter at that moment — the number the first trace in the file will have. One file per press. A run of three presses at Saved 1, 101 and 201 leaves constant_152830_from1.h5, constant_153102_from101.h5, constant_153340_from201.h5, and the numbering carries through them.

Igor's unit was the 100-trace block, written by SavePullOut whenever PullOutNumber crossed a multiple of 100; a run stopped at 250 left two full blocks and 50 traces in memory. Here the file is flushed every 32 traces and closed properly at the end of every run, including a stopped one, so nothing is left in memory.

Written by stmlab.storage.SessionWriter. The layout:

Dataset	Shape	Meaning
voltage_v	(n, samples) float64	raw ADC volts, junction-voltage channel, one row per accepted trace
current_v	(n, samples) float64	raw ADC volts, preamp output
delay_samples	(n,) int32	the AI/ AO lag recovered from the alignment spike
start_piezo_v	(n,) float64	the piezo voltage at which the pull started (the contact point)

Dataset	Shape	Meaning
bias_v	(n,) float64	the applied DC bias, in volts, including V0 when V0 check was ON
sample_rate_hz	(n,) float64	the granted rate
timestamp	(n,) float64	Unix time of capture
attempt	(n,) int64	the global attempt number (Attempts on the panel)
end_conductance_g0	(n,) float64	from TestTrace; NaN for the ramp modes
start_conductance_g0	(n,) float64	from TestTrace; NaN for the ramp modes

samples is the number of points in the pull proper — 10 000 for a 5 nm pull at 20 nm/s and 40 kHz — or the full length of a mode ramp. A ramp parameter changed mid-file is refused (a ramp parameter changed mid-session); that cannot happen from the panel, because each press opens a new file, but it is why the file is per press.

Header attributes:

Attribute	Written by	Content
format_version	storage	1
created_unix, created_iso	storage	when the file was opened
config_json	storage	the complete RigConfig as JSON, the same document File → Save writes
units	storage	the sentence "voltage_v and current_v are raw volts at the ADC. Conductance is derived, not stored."
n_traces, closed_unix	storage, at close	the row count and when the file was closed
mode	the GUI	the mode string
igor_globals_json	the GUI	every panel value, keyed by its Igor global name
segments_json	the GUI, ramp modes only	the ramp's segment table: name → (start, stop) sample indices
run_stats_json	the GUI, at close	saved_upto, attempts_total, rejections by reason, vzero_history

Two of these are the GUI's own. `igor_globals_json` is what the panel showed when the file was opened — `G_TipBias: 100.0`, `G_PseudoTotalLength_nm: 5.0`, `ZeroCheckBox: false`, all seventy-odd rows of chapter 51 — so a file can be read the way an Igor block was, by the names in the panel, and so the non-config state (Stop #, the save flags, the mode checkboxes) is on record. `run_stats_json` is the run's bookkeeping: `rejections` is a dictionary of reason → count (engage, alignment, and select_trace's reasons), `vzero_history` holds the arrays `vzero_mv`, `izero_a` and `timestamp` of every offset measurement so far in the launch — Igor's `OffsetWave`, `IzeroWave`.

Conductance is **not** stored. It is `current / voltage / G0` with the calibration in `config_json`, recomputed on every read, so that a corrected gain six months later re-analyses the raw data instead of invalidating it. That is the reason the raw volts are always kept and the Save Bias / Save Current boxes are records rather than switches.

47.3 Reading a session file

```
from stmlab import analysis, storage
from stmlab.config import RigConfig

with storage.Session("data/20260902/constant_152830_from1.h5") as s:
    n = len(s)
    v, i = s.raw(0)                # raw volts, first trace
    g0 = s.conductance(0)           # measured V and I -> G/G0
    starts = s.scalar("start_piezo_v") # any per-trace scalar
    centres, counts = analysis.log_histogram(
        s.conductances(), zero_cutoff=5e-4)
```

`Session.conductance(i, cal=None, use_measured_voltage=True)` uses the calibration stored in the file unless you pass another; with `use_measured_voltage=False` it divides by the recorded `bias_v` instead of the measured junction voltage, which is what to do when the voltage channel was not trustworthy. `conductances()` is a generator over all traces, which is what `log_histogram` wants. The header attributes are on `s._h5.attrs`; `json.loads(s._h5.attrs["igor_globals_json"])` gives the panel back as a dictionary, `RigConfig.from_dict(json.loads(s._h5.attrs["config_json"]))` the config.

For the ramp modes, cut the segment you want with `segments_json`:

```
import json
seg = json.loads(s._h5.attrs["segments_json"]) # e.g. {"iv_ramp": [6000, 10000], ...}
a, b = seg["iv_ramp"]
v_sweep, i_sweep = (x[a:b] for x in s.raw(0))
```

`experiments/03_iv_sweep/iv_analysis.py` and the other analysis scripts do this for each mode; chapters 11-14.

47.4 Histogram files

Name: `loghist_<HHMMSS>_upto<Saved>.csv`, written when a block of 100 saved traces completes and, at the end of a run, for the partial block — only when **Save Hist** is ticked (`G_HistSaveCheck`, on by default). Two columns, `log10(G/G0)` and `counts_per_trace`, 1000 rows from -8 to 2 , a one-line header. It is `analysis.log_histogram` of that block with the zero cutoff `cfg.ramp.break_g0`, exactly what the `LogHistOfBlock` window showed. Saved in the name is the counter *after* the block, so `upto101` is traces 1-100.

The CSV is a convenience; the session file is the record. Recomputing the histogram from the file, as above, gives the same numbers, or different ones if you choose a different cutoff — which is the point of keeping the raw data.

47.5 CV files

Name: `cv_<kind>_<HHMMSS>.h5`, `kind = highres or lowres`. Written by `storage.save_cv_cycles`:

Item	Content
<code>applied_v</code>	(samples,) the commanded potential in volts, shared by every cycle
<code>keep</code>	(samples,) bool, False where Igor NaN-ed (switch-on quarter, repeated tail)

Item	Content
tip_current_v	(cycles, samples) raw preamp volts
we_voltage_mv	(cycles, samples) measured electrode voltage — highres only
attrs	format_version, record_kind = "cv", cv_variant, created_*, config_json, n_cycles, rate_hz, gate_mv, units

gate_mv is the counter-electrode potential in force when the CV ran. Read with storage.CVSession: applied_v, keep, current_a(i) (raw volts over the stored gain), we_voltage_mv(i).

47.6 What is not stored

- Rejected traces. Igor did not keep them either. The count per reason is in run_stats_json; if you need the traces, lower the selection thresholds or use a ramp mode, which keeps everything.
- Background reads, approach reads, the Find Suppress sweep and the Find Offset points. The last of each lives in the controller (last_suppress, vzero_tracker.history) for the duration of the launch, and the offset history is written into every session file's summary.
- The History text. Launch with -v and redirect the console if you want it on disk.

47.7 Size

A constant-bias trace is two channels $\times$ 10 000 samples $\times$ 8 bytes, 160 kB raw; gzip on the chunked datasets brings a hundred traces to a few megabytes. A 5000-trace day is well under a gigabyte. Igor's single-precision .ibw blocks were smaller; the difference is the price of raw volts on both channels at full resolution.

47.8 Comparing with an archived Igor block

storage.load_ibw(path) reads an Igor binary wave; storage.split_igor_block(block, n_parameters=20) separates the twenty-row parameter header SavePullOut stacked above each column from the conductance below it. Igor stored *conductance*, not raw volts, so the comparison is between Igor's conductance columns and Session.conductance(i) here — same displacement axis, same bias, same gain. The 1 G0 peak position of the two histograms is the whole-chain check (chapter 10).

That comparison has not been done. No .ibw file from this rig was available when the package was written, and the 1 G0 peak has only been reproduced from the simulator. Until an archived block has been read and matched, the analysis is self-consistent, not validated.

How stmgui is built

Module: stmgui/ — state.py, controller.py, widgets.py, graphs.py, panels/, app.py. Tests: tests/test_gui_controller.py, tests/test_gui_app.py.

This chapter is for the day something needs changing: a new control, a new graph, a different data path, a hardware quirk. It describes the design, the one rule that holds it together, and three walkthroughs.

48.1 The shape

stmgui/app.py	App: builds everything, the menus, the event pump
stmgui/panels/	MainPanel, VoltageOffsetPanel, EChemPanel, HistoryWindow
stmgui/graphs.py	GraphSet and the nine GraphWindow classes
stmgui/widgets.py	SetVariable, ValDisplay, CheckBox, button, groupbox, Tooltip
	Tk thread: reads state, queues commands, drains events
v	
stmgui/controller.py	RigController: one worker thread, owns the Rig, posts events
stmgui/state.py	GuiState = RigConfig + GuiOptions; PARAMS, the binding table
v	
stmlab/	the translated procedures; unchanged

The one rule: the controller never imports Tk, and the panels never touch the Rig. A click becomes a command queued to the controller’s worker thread; the worker posts (kind, payload) events; the Tk thread drains them on a timer and fans them out to the panels and graphs. Every consequence in this chapter follows from that rule.

Why: Tk is single-threaded and a DAQ call blocks for the length of a waveform. Igor ran everything on its one GUI thread and polled the Alt key to get a word in edgeways; a Python GUI that did the same would freeze for the duration of every pull and could not draw a trace while the next was being taken. Putting the rig on its own thread fixes that, and putting it on exactly *one* thread with a queue means two buttons can never reach the card at once — there is no lock to forget, because there is nothing to contend.

48.2 state.py

GuiOptions is a dataclass of the Igor globals that have no home in RigConfig — the things a panel needs that do not describe the rig:

- DAQ tab: fast_read_wave_size, piezo_offset_nm, sense_scale_nm_per_v; piezo piezo_step_nm.
- Counters: pull_out_number (Saved), stop_number, pull_out_attempt, actuator_counter.
- Save flags: save_bias, save_current, save_hist, save_sense, save_piezo_wave, voltage_read.

- Keithley checkboxes: `zero_check`, `suppress_on`.
- Mode checkboxes: `push_pull_check`, `iv_check`, `ac_hold_check`, `hb_hold_check`, with `mode()` and `select_mode(name, on)` implementing `RampOptionsCheckProc`'s exclusivity.
- Voltage Offset: `vzero_check`, `vzero_mv`, `izero_ua`.
- Background sampling: `bkgd_sampling`, `beep_current_ua`.
- Readouts: `current_readout_ua`, `junction_voltage_mv`, `piezo_readout_v`.
- `data_dir`, and `calibrate_on_start_writing`.

GuiState is a `RigConfig` and a `GuiOptions` together. Everything the panels read or write is reachable from it by a dotted path.

Param is one panel control's binding: the Igor name, the dotted path, the kind (`float`, `int` or `bool`), the panel title, Igor's limits triple, a `printf` fmt, and a display scale (Tip Bias is stored in volts, shown $\times 1000$). `get(state)` walks the path and applies the scale; `set(state, value)` coerces, clamps to the limits, divides by the scale, stores, and returns what was stored in display units so a widget can show the clamped value.

PARAMS is the dictionary of every binding, keyed by Igor name. It is the single place a control's meaning is defined, and chapter 51 is generated from it. **`simulated_state()`** is the `--simulate` starting point.

48.3 controller.py

`RigController(state, events=None, start_worker=True)` holds the Rig (None until Start Writing), the Keithley (amp), the counter electrode (gate), the X piezo (xp), the VzeroTracker, the histogram block, and the bookkeeping (`izero_history`, `last_suppress`, `last_cv`, `last_trace`, `session_files`, `rejections`). It starts a daemon thread running `_worker`.

Commands. Every public method — `start_writing`, `kill_tasks`, `reset_daq`, `piezo_step`, `piezo_goto`, `set_tip_bias`, `set_gain`, `set_suppress_const`, `zero_check`, `zero_correct`, `suppress_enable`, `keithley_bias`, `find_suppress`, `find_offset`, `step_actuator`, `approach`, `start_measurement`, `counter_electrode_on`, `set_counter_electrode`, `start_cv`, `move_xpiezo`, `zero_xpiezo`, `rungo`, `lateral_expt`, `connect_instruments` — is a one-liner that queues its `_impl` through `_run(name, fn, *args, wait=False)`. With `wait=False` (the panels) it returns at once; with `wait=True` (the tests and the macros) it blocks until the command has run and re-raises the command's exception in the caller. `wait_idle()` queues a no-op and waits for it, which drains everything ahead of it.

The worker loops on the queue with a timeout of `BKGD_PERIOD_S` (0.25 s). A command sets busy, posts busy with its name, runs, and in finally clears busy and posts idle. The expected exceptions — `SafetyViolation`, `ApproachError`, `ConfigError`, `NotReady`, `KeithleyError`, `EChemError`, `XPiezoError` — are logged and posted as error events with the command name; anything else is logged with a traceback and posted the same way, so a bug in one command never takes the thread down. When the queue times out with nothing to do and `opts.bkgd_sampling` is set and a rig is open, `_background_read` runs once — which is why sampling pauses during commands and resumes after without any explicit start/stop.

Stop is one threading.Event. `request_stop` sets it; `_measure_impl`, `_approach_impl`, `_rungo_impl` and `_lateral_impl` clear it when they begin and check it between steps. `kill_tasks` sets it before queueing itself so that a run ahead of it in the queue ends first.

Guards. `_require_rig()` raises `NotReady` with the “press Start Writing first” message; `_require_amp()` likewise for the Keithley. Every `_impl` that needs one calls it first, so refusals are uniform and happen before any hardware is touched.

Events. `_emit(kind, payload)` puts a tuple on events. The kinds and payloads:

Kind	Payload
log	a formatted log line (from <code>QueueLogHandler</code>)
error	"<command>: <message>"
busy, idle	the command name
readout	{current_ua, junction_mv, piezo_v, beep}
highres	{voltage_mv, current_ua, piezo_nm} arrays
piezo	{volts, nm} after any fine-piezo move
writing	True / False
bkgd	True / False (the checkbox state after <code>set_background_sampling</code>)
trace	{g0, disp_nm, bias_mv, piezo_nm, accepted, mode, number}
hist	{centres, counts, n, partial}
izero	a <code>VzeroResult</code>
izero_time	(saved_numbers, izero_ua) arrays
vzero	(vzero_mv, izero_ua)
suppress	(new_ua, sweep, readings)
cv	list of <code>CVCycle</code>
counter	the actuator step count
saved, attempts, stop_number	the counter value
bias	Tip Bias in mV after <code>set_tip_bias</code>
gain	the exponent after <code>set_gain</code>
gate	counter-electrode mV, or None when released
xpiezo	position in nm
config	None, after a config load

QueueLogHandler is a `logging.Handler` attached to the `stmlab` and `stmgui` loggers at `INFO`; every log record becomes a log event. That is how the History window sees the package’s own messages without the package knowing about the GUI.

_make_rig. In `simulate`, builds the `SimulatedDaqSession` and a `SimulatedActuator` *linked to it* (`rig_sim=session`), so coarse steps move the simulated surface — `Rig.open()` alone would build an unlinked actuator and `Start Approach` would never find current. On hardware, `verify_devices`, `park_all_outputs`, then `Rig(cfg).open()`.

_measure_impl is chapter 41 step 6 in code; read them side by side. The helpers: `_configure_mode` (the per-mode `pull_length_nm` and bias limit), `_build_ramp` (constant → `trace.build_ramp`, else `ramps.build_*`), `_open_writer` / `_close_writer` (the session file and its summary), `_reject` (the reason counts), `_histogram_input` (removing the hold section of a mode trace) and `_histogram_block` (the histogram, the hist event, the CSV).

shutdown sets `stop`, runs `_kill_tasks_impl` with `wait=True`, closes the Keithley, stops the thread loop and detaches the log handler.

The button map. At the bottom of `controller.py`, `COMMAND_MAP` is a dictionary from each public method name to (panel control, Igor procedure, `_impl` name, the `stmlab` functions it calls). Three helpers read it: `locate(obj)` gives `file:line` for any function or method via `inspect`; `where("stmlab.keithley.find_suppress")` imports and locates a dotted name; `describe_command(method)` builds the tooltip text (docstring, Python `file:line` of the method and its `_impl`, Igor procedure, calls); `command_table()` builds the rows the Button map window

shows. The worker also posts a `-> <name>: file:line function log` line as each command starts. A test (`test_every_public_command_is_in_the_button_map`) fails if a public method is added without a `COMMAND_MAP` row or if a listed call cannot be resolved, so the map cannot drift from the code.

48.4 widgets.py

`SetVariable` is label + entry (or spinbox when the Igor increment is non-zero) bound to a `Param`; it commits on `Return`, `KP_Enter`, `FocusOut` and the spinbox arrows, rejects non-numeric text by restoring the display, clamps through `Param.set`, and calls `on_commit(stored)` when the value changed (or unconditionally when called programmatically with no event). `refresh()` re-reads the state. `ValDisplay` is a read-only entry with a format. `CheckBox` is a `ttk.Checkbutton` bound to a bool `Param` with `on_toggle`. `button()` is a `tk.Button` with Igor's text colour — macOS Aqua ignores background colours on native buttons, so the text colour carries the meaning there. `groupbox()` is a `LabelFrame`. `set_enabled()` works on any of them. `Tooltip` and `param_tip` give every control its hover text: buttons show the docstring passed as `tip=`, entries and checkboxes show the Igor name, the path and the limits.

48.5 graphs.py

`GraphWindow` is a `Toplevel` with a `matplotlib Figure`, one axes, a `FigureCanvasTkAgg`, and a `decorate()` hook for labels and ranges; it starts withdrawn, `show()` raises it, closing withdraws it. The nine subclasses each implement `update(payload)`. `GraphSet` creates them lazily from `ORDER(name, class, Igor position)`, routes events by `FEEDS`, and applies the first-data auto-open rule (chapter 45).

48.6 panels/

`MainPanel` is a `ttk.Frame` in the root window. Its `_sv(parent, igor, on_commit, **kw)` and `_cb(...)` helpers build a `SetVariable` or `CheckBox` from a `PARAMS` row and register it in `refreshable`, so a config load refreshes every control with one loop. `_build_daq_tab`, `_build_inputs_tab` and `_build_options_tab` are the three tabs. `set_writing(on)` and `set_busy(name)` are the enable/disable rules (chapter 42). `on_event(kind, payload)` is one `if/elif` per event kind. `VoltageOffsetPanel` and `EChePanel` are `Toplevels` with the same pattern; `HistoryWindow` is a `Text` with a scrollbar and two tags.

48.7 app.py

`App(state=None, simulate=False, root=None)` builds, in the order `InitializeExperiment` did: styles, the controller, the main panel, the History, the Voltage Offset panel, the EChem panel, the `GraphSet`, the menus; binds `Alt` and `Escape` to `request_stop`; logs the two banner lines; queues `connect_instruments`; and schedules `_pump`. `_pump` drains up to 200 events from the controller's queue, fans each out (`_fanout`) to the main panel, the two `Toplevels`, the History and the graphs — catching and logging any exception a handler raises, so a plotting error cannot stop the pump — and reschedules itself every `PUMP_MS` (50 ms). `pump_once()` drains synchronously, for tests. `quit()` shuts the controller down and destroys the root. `main(argv)` is the `gui.py` entry point.

48.8 Threading rules, and what they prevent

- Only the worker thread calls anything in `stmLab` that touches hardware. Prevents two DAQ calls at once (a card error at best, an untracked piezo position at worst).
- Only the Tk thread touches widgets. `_impl` methods communicate by events. Prevents the Tk “calling from another thread” crashes.
- The panels read state freely and write it through `Param.set` on the Tk thread; the worker reads it when a command runs. A value changed during a command is picked up at that command’s next read of it, which is why chapter 42 says “next Start” for most fields. There is no lock; the fields are plain attributes and Python’s assignment is atomic.
- `stop` is the only cross-thread signal that interrupts anything, and it is polled, never forced. A waveform in progress always completes.

48.9 Walkthrough: adding a control

Suppose you want Igor’s `G_ACCapLength` (the AC-hold cap length, which Igor never put on the panel) on the AC sub-tab.

1. In `state.py`, add one row to `PARAMS`: `Param("G_ACCapLength", "cfg.ac_hold.cap_nm", float, "Cap length (nm)", fmt="%3.1f")`.
2. In `panels/main_panel.py`, in `_build_options_tab` under the AC sub-tab, add `self._sv(ac, "G_ACCapLength", width=6).grid(row=3, column=1, sticky="w", padx=8)`.
3. Run `manual_build/gen_binding_table.py` — the generator refuses if a `PARAMS` row is not in one of its groups, so add the name to the “Options tab: AC” list there too.

Nothing else: the value is read by `ramps.build_ac_hold` at the next Start, the tooltip is automatic, `test_every_param_round_trips` covers it. If the control must *act* at once (like Gain), give it an `on_commit` that calls a controller method, and write that method as a `_run` one-liner plus an `_impl`.

48.10 Walkthrough: adding a button

Say a “Zero X piezo” button on the panel instead of only in the menu.

1. Controller: the method exists (`zero_xpiezo`). A new action would be `def frob(self, wait=False): return self._run("Frob", self._frob_impl, wait=wait)` and `def _frob_impl(self): rig = self._require_rig(); ...; self._emit("something", payload)`.
2. Panel: `button(parent, "Zero X", self.ctl.zero_xpiezo, tip=self.ctl.zero_xpiezo.__doc__)` and a grid call. If it must be greyed while busy, add it to the tuple in `set_busy`.
3. Test: in `test_gui_controller.py`, call it with `wait=True` against the simulator and assert on `drain(ctl)`.

48.11 Walkthrough: adding a graph

1. Subclass `GraphWindow` in `graphs.py` with `title_text`, `igor_geometry`, `decorate()` and `update(payload)`.
2. Add it to `GraphSet.ORDER` and, under the event that feeds it, to `FEEDS`. The Windows menu picks it up from `ORDER` automatically.
3. Post the event from the controller where the data is produced.

48.12 Tests

`tests/test_gui_controller.py` (23 tests) drives the controller headlessly against the simulator: the binding table round-trips; Tip Bias is in mV; the mode checkboxes are exclusive; buttons refuse before Start Writing; Start Writing then Kill Tasks; background sampling updates the readouts; piezo step and slider; the approach steps until current and forces step size 5; the standalone actuator when the task is off; the Keithley controls send Igor's command strings; Tip Bias is quantised with the Keithley bias on and refused beyond the limit; Find Offset then Find Suppress with the bias restored; +1 then a run to Stop #, with the partial-block histogram and the file's trace count; V0 check re-measures mid-run; each of the four ramp modes; Stop ends a run; counter electrode and CV; X piezo and LateralEXPT; rungo resumes from Saved; config save and load. `tests/test_gui_app.py` (6 tests) builds the real widgets with a withdrawn root: every panel and graph constructs; a `SetVariable` writes through and rejects bad text; the mode checkboxes; a simulated run reaches the graphs and the enable states; a CV reaches the voltammogram; every button has a tooltip. They skip where Tk has no display.

```
./venv/bin/python -m pytest tests/ -q # all 122
./venv/bin/python -m pytest tests/test_gui_controller.py -q
```

The controller tests need no display because the controller has no Tk in it. That is the practical payoff of the one rule: the entire lab flow can be exercised on a build server, and a change to the flow is caught before anyone opens a window.

Deviations, limitations, troubleshooting

Igor: Controls_STMBJ.ipf and Functions_STMBJ.ipf throughout. Modules: stmgui/controller.py, stmlab/safety.py, stmlab/approach.py.

49.1 Deviations from Igor, and why

Every one of these is deliberate and is called out in a docstring at the point it happens. The list is complete as far as a line-by-line comparison of controller.py with the Igor control procedures goes.

What	Igor	Here	Why
Stop	polled the Alt key inside loops	a Stop button, plus Alt and Escape	a mouse-only operator; same flag, same semantics
Session calibration	none	preamp zero and group delay measured at Start Writing	validate() warns without them; the tip is retracted at that moment, which is when they are valid
Piezo offset	G_PiezoOffset_nm, command line only	an entry on the DAQ tab	on a unipolar piezo it decides whether a pull has room
Slider range	-10 ... +10 V	limits.piezo_ao_min_v .. max_v, 0 ... 10 V	the piezo is unipolar; Igor's own defaults would drive it 10 V the wrong way (chapter 02)
Contact threshold default	5 G0	0.5 G0	5 G0 is 38.7 V at the ADC at gain 10 ⁶ and unreachable; a railed preamp also counts as contact
Coarse steps	no interlock	refused unless the fine piezo is retracted; step budget	a coarse step with the piezo extended drives the tip into the sample
Kill Tasks	zeroed the piezo, kept the bias, cleared the task	withdraws both to 0 V	0 V on both channels is safe regardless of what the tip is near
Histogram	complete 100-trace blocks only	also the partial block when a run ends	you see what a short run gave you
Data files	one .ibw block per 100 traces, conductance only	one HDF5 file per Start press, raw volts on both channels	re-analysis with a corrected calibration is the whole reason to keep raw volts

What	Igor	Here	Why
Find Offset V0 / I0	editable SetVariables	read-only displays	a hand-typed V0 would be applied with no record of where it came from
Find Suppress	restored the bias after the sweep	restores it in a finally	a failed sweep cannot leave the junction at zero bias
CV HighRes on hardware	ran with the output task open, then reset the card	refuses while the task is open	two tasks cannot own dev1/ao1; Igor let them collide
CV timing	AI and AO started back to back	AI triggered by the AO start	removes the skew for free
X piezo	no range check	refuses a move outside 0 ... 10 V	the headroom is finite
Mode ramps	bias refused nowhere	limits.bias_max_v raised to $1.1 \times$ the sweep/hold with a warning	the limit exists to protect the constant-bias modes, and lifting it should be visible
LateralEXPT	slider to 0 <i>after</i> the actuator steps	Rig.withdraw() <i>before</i> them	the interlock needs the fine piezo parked before a coarse step
Readout "Piezo" and SenseInDisplay	the second card's sense input	the commanded piezo voltage	one card, no sense line
Inputs-tab _EChem controls	created disabled, never enabled	not reproduced	they were invisible in Igor
check08, setvar15, setvar5, setvar7	disabled placeholders	not reproduced	untitled and unbound
Thermocouple sampling	commented out	absent	as in Igor
Suppress sweep units	SetCurrent Suppress(CurSup) with CurSup in the constant's units	keithley.find_suppress sweeps ± 1 μA and the constant is derived back	the module translated the sweep in μA ; the panel shows Igor's constant either way

49.2 Limitations

- **Hardware has never met this code.** Every nidaqmx call in stmllib/daq.py, echem.py and xpiezo.py, the Keithley over pyvisa, the NanoPZ over pyserial and the second card are transcriptions of Igor's calls, reviewed but unexercised. The GUI adds no hardware code of its own; it inherits this status whole. Chapter 03 is the bring-up plan.
- **One command at a time.** The worker thread runs commands in order. A click during a long command queues behind it, and the status line says running: ... until then. Only Stop and Kill Tasks act on a running command.
- **macOS button colours.** Aqua ignores background colours on native buttons; the text colour carries the meaning. On Windows and Linux the buttons look like Igor's.
- **No panel screenshots in this manual.** The graph figures are real; the panels are described, not pictured, because the build machine did not permit screen capture. Launch --simulate to see them.
- **Window positions** are not remembered between launches.

- The tooltip on a spinbox arrow may not appear on every platform; hover the entry itself.

49.3 Config warnings at Start Writing

`validate()` runs at Start Writing and at every Start Measurement; the History shows each warning prefixed `config:.` What they mean:

Warning begins	Meaning	Do
<code>hv_amp_gain</code> is unknown	the piezo driver gain is not recorded; limits are enforced at the DAQ output	nothing, if <code>cal.piezo_nm_per_volt</code> is an end-to-end number (it is, 62 nm/V from Igor)
<code>current_zero_v</code> is 0	the preamp zero has not been measured	it will be, at Start Writing; if you turned that off, run <code>calibrate.measure_zero</code>
1 G0 produces ... V, above the preamp saturation limit	the metallic-contact region will read railed	normal at gain 10 ⁶ and 100 mV; the science is below 1 G0
engage threshold ... needs ... V at the ADC but the input saturates	contact can only be detected by saturation	lower Contact Threshold, or the gain
<code>keithley.gain_exponent</code> = g programs ... but <code>cal.preamp_gain_v_per_a</code> is ...	the box and the arithmetic disagree	set Gain from the panel, which writes both
push-pull: <code>push_pull_nm</code> ... does not exceed <code>initial_pull_nm</code>	every cycle holds below the contact point	raise Push-Pull length above Initial Pull
a ... nm pull uses ...% of the usable piezo range	contact must be made high in the range	raise Piezo offset, or approach higher
requested ... Hz but the card granted ... Hz	delta-sigma rate rounding	nothing; every time axis uses the granted rate
RAISING	a mode's sweep or hold	expected for IV / AC / HB hold;
<code>limits.bias_max_v</code> from ... to ...	exceeds the everyday limit	check the number is what you meant

A `ConfigError` (not a warning) stops the command; its text names the field: bias beyond the limit, bias of 0 V, a pull longer than the piezo range, an engage threshold below the break threshold, a spike that does not fit in the pull, a sample rate that gives fewer than 100 points.

49.4 Symptom, cause, fix

Symptom	Cause	Fix
<code>gui.py</code> prints three install lines and exits	this Python has no Tk	use the python.org build or brew install <code>python-tk@3.12</code> , recreate <code>.venv</code>
<code>ModuleNotFoundError: matplotlib</code>	wrong interpreter	run with <code>./venv/bin/python</code> , not <code>python</code>
Keithley 428 not available: ... at launch	pyvisa missing, no GPIB interface, wrong address	install pyvisa and NI-488.2; check <code>keithley.resource</code> ; the rest works meanwhile

Symptom	Cause	Fix
the output task is not running -- press Start Writing first	a button that needs the rig, before Start Writing	Start Writing
the Keithley 428 is not connected	see above	connect, relaunch
Background sampling needs the output task	ticked before Start Writing	Start Writing first
background sampling stopped: ...	a read raised (card error)	check the History for the underlying error; Kill Tasks, Start Writing
Kill Tasks before resetting the DAQ devices	Reset with the task open	Kill Tasks first
no coarse actuator configured (actuator.kind == 'none')	one-card rig with no NanoPZ in the config	set actuator.kind to nanopz with the port, or approach by hand
refusing coarse step ...: piezo is at ... V, must be below 0.100 V	fine piezo extended	slider to 0 (or Step apart) before any coarse step
coarse approach took 2000 steps without finding current	actuator direction, wiring, or the tip is far away	check the NanoPZ direction and the current path
bias ... V exceeds limit +/-... V or tip bias ... exceeds the limit	above limits.bias_max_v	raise the limit in a config file, deliberately
piezo command ... V clamped to ... V (warning)	a move beyond the piezo range	the move was clamped; expected during smash near the floor
refusing pull: contact at ... V, a ... nm pull needs ... V	contact too low in the range for the excursion	recover_headroom backs the actuator off automatically up to five times; if it still fails, raise Piezo offset or approach higher
could not separate contacts: piezo is at the ... V floor	in contact at the bottom of the range	back the coarse actuator off (Step apart)
could not make contact: piezo is at the ... V ceiling	surface out of reach of the fine piezo	Start Approach
cannot make contact: ... ends a run	engage failed and the actuator could not help	see the two rows above; the file is closed cleanly
vzero: piezo at the ceiling with no contact	Find Offset before the approach	Start Approach first
vzero: flat I(V) -- no contact, or a railed preamp	no measurable current, or every point railed	check contact and gain
suppress sweep produced a flat response	Keithley in zero check, or disconnected	untick Zero Check
no low-res device configured (channels.low_res_device is None)	EChem or X piezo on a one-card config	set channels.low_res_device if you have the card; --simulate pretends one
press Counter Electrode ON first	typed a gate bias before ON	ON, then the bias

Symptom	Cause	Fix
CV HighRes drives the junction-bias channel; Kill Tasks first	CV HighRes with the task open, on hardware	Kill Tasks, CV, Start Writing
CV ramp too short; slow the scan rate or raise <code>cv_rate_hz</code>	fewer than four samples per leg	as it says
lateral move to ... nm needs ... V, outside [0.0, 10.0] V	X piezo range	MoveXPiezoToZero, re-centre
Saved (n) >= Stop # (m): nothing to do	Stop # not above Saved	raise Stop #
Catastrophic failure: 100000 attempts a run with Attempts climbing and Saved not	Igor's own guard: the loop never accepted a trace every trace rejected by TestTrace	something is wrong with contact or selection; look at PullOutLowG watch PullOutLowG: no plateau (tip needs a smash, or the gain is wrong) or no tunnelling tail (Zero Cutoff too low)
constant beeping	current above 0.15 μ A during background sampling	expected in contact; retract, or untick sampling
the GUI "hangs"	a long command is running; the status line says which	wait, or Stop; the window is never actually blocked, only greyed
Start Measurement greyed	a command is running, or the task is off	see the status line
CV buttons greyed	Gain not yet committed this launch	type the Gain (either panel) and press Return
no histogram after 60 traces	blocks are 100 traces	finish the run (partial block) or reach 100
a ramp parameter changed mid-session	a mode config edited during a run	cannot happen from the panel; from a script, open a new writer

49.5 If the History shows a traceback

An unexpected exception in a command is logged with its traceback and posted as error; the worker thread survives and the next command runs. Copy the traceback from the History (select and copy works in the text widget) — it names the `stmlab` function. Nothing in the GUI needs a restart after an error; Kill Tasks and Start Writing re-establish a known output state.

49.6 What to check before the first hardware run

1. `python run.py 01 bringup` — chapter 03, with the card, the scope and a resistor, before any tip is near a sample.
2. `keithley.resource` matches the GPIB address, and the gain on the box's front panel matches the Gain entry.
3. `actuator.kind`, port and baud match the NanoPZ, and Step apart moves it *apart*.
4. `limits.piezo_ao_max_v` and `cal.piezo_nm_per_volt` match the driver box and the piezo; Piezo offset larger than Excursion.
5. Start Writing with the tip far from the sample; watch the HighRes window read noise around zero; Step closer and apart on the piezo and watch the readout follow.

6. Only then Start Approach.

GUI quick reference

Every button, event, key and file on two pages. Chapter 51 is the complete binding table; chapter 30 is the command-line quick reference.

50.1 The lab flow

Step	Press	Needs	Ends with
1	DAQ tab → Start Writing	a valid config	output task ON ... in the History
2	Background Sampling (Beep is ON)	step 1	readouts updating, HighRes open
3	Start Approach	step 1, an actuator	Approach: actual current= ... above 0.1 μ A
4	Voltage Offset → Find Offset	step 1 (and 3 on hardware)	V0, I0, the Izero window
5	Find Suppress	step 1, the Keithley, zero check off	New current suppress is ...
6	Start Measurement (or +1)	step 1, Stop # above Saved	<mode>: n saved / m attempts
7	Stop (Alt / Escape)	—	stopping: Stop pressed
8	Kill Tasks	—	output task OFF; outputs parked

50.2 Buttons and checkboxes

Control	RigController method	Igor procedure	stmlab
Start Writing	start_writing	StartHighResWriting Task	Rig.open, Rig.hold, calibrate.session_calibration
Kill Tasks	kill_tasks	StopWritingTasks	Keithley428.zero_check, Rig.close
Reset DAQ Devices	reset_daq	EChemResetDAQDevices	nidaqmx.system.Device.reset_device, safety.park_all_outputs
Background Sampling	set_background_sampling	BkgdSamplingCheckProc	Rig.hold every 0.25 s
Bias	keithley_bias	KeithleyBias	Keithley428.bias_enable (B1X/B0X)

Control	RigController method	Igor procedure	stmlab
Zero Check	zero_check	ZeroCheckProc	Keithley428.zero_check (C1X/C0X)
Zero Correct	zero_correct	ZeroCorrectProc	Keithley428.zero_correct (C2X)
Suppress I	suppress_enable	CurrentSuppressCheckProc	Keithley428.suppress_enable (N1X/N0X)
Find Suppress	find_suppress	FindSuppress → TestVirtualGround	Rig.set_bias(0), keithley.find_suppress, Rig.set_bias
Gain	set_gain	SetGain	Keithley428.set_gain (H6R<g>X)
Suppress I value	set_suppress_const	SetCurrentSuppress	Keithley428.set_suppress_ua (H8S...X)
Start Approach	approach	ApproachButton → HighResCardApproach	Rig.hold, Rig.coarse_step
Step closer / apart (actuator)	step_actuator	StepActuatorCloser / Apart	Rig.coarse_step or Actuator.step
Step closer / apart (piezo)	piezo_step	PiezoStepCloser / Apart	Rig.piezo_step_nm
Slider	piezo_goto	SetPiezoBiasFrom Slider	Rig.piezo_goto
Tip Bias (mV)	set_tip_bias	SetTipBiasVoltage	Rig.set_bias, Keithley428.set_bias_mv
Start Measurement	start_measurement(False)	StartMeasurement → MeasureBreakJunctions	approach.recover_headroom, trace.build_ramp / ramps.build_*, trace.capture, analysis.select_trace, storage.SessionWriter, analysis.log_histogram
+1	start_measurement(True)	MakeAttempt	as above, once
Stop (Alt)	request_stop	GetKeyState	—
Find Offset	find_offset	FindOffset → OffsetVoltage	vzero.measure_offset
V0 check ON	(read at Start)	VzeroCheckBox	trace.build_ramp(bias_offset_v), VzeroTracker.maybe_measure, ramps.build_hb_hold(vzero_mv)
Counter Electrode ON	counter_electrode_on	CounterElectrodeOn	echem.make_counter_electrode(...).on()
Counter Electrode Bias	set_counter_electrode	CounterElectrodeSetVar	CounterElectrode.set_mv
CV HighRes / LowRes	start_cv(kind)	StartCVHighResButton / LowRes	echem.run_cv, storage.save_cv_cycles
Macros → rungo	rungo	rungo()	_set_tip_bias_impl, _measure_impl per bias

Control	RigController method	Igor procedure	stmlab
Macros → LateralEXPT	<code>lateral_expt</code>	<code>LateralEXPT()</code>	<code>_measure_impl</code> , <code>Rig.withdraw</code> , <code>Rig.coarse_step</code> , <code>XPiezo.move_nm</code> , <code>_approach_impl</code>
Macros → MoveXPiezo / ToZero	<code>move_xpiezo</code> , <code>zero_xpiezo</code>	<code>MoveXPiezo</code> , <code>MoveXPiezoToZero</code>	<code>XPiezo.move_nm</code> , <code>XPiezo.zero</code>
File → Load / Save config	<code>load_config</code> , <code>save_config</code>	—	<code>RigConfig.from_json</code> , <code>to_json</code>
Quit	<code>shutdown</code>	—	<code>_kill_tasks_impl</code> , <code>Keithley428.close</code>

50.3 Events

Kind	Payload	Consumed by
log, error	text	History (and the status via error)
busy, idle	command name	main panel status line, button greying; Voltage Offset button
readout	<code>current_ua</code> , <code>junction_mv</code> , <code>piezo_v</code> , beep	readouts; the bell
highres	arrays	HighRes
piezo	volts, nm	Piezo readout, slider
writing	bool	enable rules on all three panels
bkgd	bool	the Background Sampling box
trace	<code>g0</code> , <code>disp_nm</code> , <code>bias_mv</code> , <code>piezo_nm</code> , <code>accepted</code> , <code>mode</code> , number	the three PullOut windows, <code>SenseInDisplay</code>
hist	<code>centres</code> , <code>counts</code> , <code>n</code> , <code>partial</code>	<code>LogHistOfBlock</code>
<code>izero</code> , <code>izero_time</code> , <code>vzero</code>	<code>result</code> / arrays / (mV, μ A)	<code>Izero</code> , <code>Izero_Time</code> , the V0 and I0 displays
suppress	<code>new_ua</code> , <code>sweep</code> , <code>readings</code>	Suppress I value
cv	<code>cycles</code>	Cyclic Voltammogram
<code>counter</code> , <code>saved</code> , <code>attempts</code> , <code>stop_number</code>	<code>ints</code>	the counters
<code>bias</code> , <code>gain</code>	mV / exponent	Tip Bias, Gain (and the CV buttons' enable)
<code>gate</code> , <code>xpiezo</code>	mV or None / nm	EChem panel
config	—	every panel refreshes

50.4 Keys and menus

Key or item	Does
Alt, Escape, Stop (Alt), Macros →	<code>request_stop</code>
Stop	
Return / focus-out in an entry	<code>commit</code>
File	Load config JSON, Save config JSON, Open data folder, Quit
Macros	the three panels; <code>runo()</code> ; <code>LateralEXPT(...)</code> ; <code>MoveXPiezo(nm)</code> ; <code>MoveXPiezoToZero()</code> ; Stop

Key or item	Does
Windows	HighRes, PullOutGvsE, PullOutLowG, AuAuConductanceLevel, SenseInDisplay, LogHistOfBlock, Izero, Izero_Time, CyclicVoltammogram, History, Button map
Help	Button map (which Python does each button run?), About / where the manual is
Hover any control	tooltip: what it does, Igor procedure, Python file:line of the method and its <code>_impl</code> , the stmlab functions called
Right-click any control (macOS: Control-click)	the Inspector window with the same text, kept open
"Which code runs this?" (bottom of the main panel)	the Button map window, all controls at once

50.5 Files written

File	When	Content
data/<date>/<mode>_<HHMMSS>_from<Saved>.h5	each Start Measurement / +1	raw traces, config, panel state, run summary (chapter 47)
data/<date>/loghist_<HHMMSS>_upto<Saved>.csv	each 100-trace block and the final partial block, with Save Hist on	log10(G/G0), counts per trace
data/<date>/cv_<kind>_<HHMMSS>.h5	each CV	applied potential, keep mask, tip current per cycle

50.6 Commands

Do	Command
Launch, simulated	<code>./.venv/bin/python gui.py --simulate</code>
Launch, hardware	<code>./.venv/bin/python gui.py</code>
Launch from a saved config	<code>./.venv/bin/python gui.py --config my.json</code>
Same through the dispatcher	<code>./.venv/bin/python run.py gui --simulate</code>
All tests	<code>./.venv/bin/python -m pytest tests/ -q</code>
GUI tests only	<code>./.venv/bin/python -m pytest tests/test_gui_controller.py tests/test_gui_app.py -q</code>
Rebuild this manual	<code>manual_build/build.sh</code> (add <code>--figures</code> to regenerate the graph figures from a simulated run)
Regenerate the binding table	<code>./.venv/bin/python manual_build/gen_binding_table.py</code>

50.7 The numbers you change most

Panel control	Default	Lives in
Tip Bias (mV)	100	<code>cfg.ramp.bias_v (V)</code>
Excursion (nm)	5	<code>cfg.ramp.pull_length_nm</code>
Pull Rate (nm/s)	20	<code>cfg.ramp.pull_rate_nm_per_s</code>
Contact Threshold (G0)	0.5	<code>cfg.ramp.engage_g0</code>
Contact step size (nm)	0.5	<code>cfg.ramp.approach_step_nm</code>

Panel control	Default	Lives in
Zero Cutoff (G0)	0.0005	cfg.ramp.break_g0
Smash Freq. / In / Out	50 / 30 / -40	cfg.ramp.smash_every, smash_in_nm, smash_out_nm
Stop #	1001	opts.stop_number
Saved	1	opts.pull_out_number
Piezo offset (nm)	0	opts.piezo_offset_nm
Fast Read Wave Size	200	opts.fast_read_wave_size
Z (nm), piezo step	5	opts.piezo_step_nm
Step Size, actuator	5	cfg.actuator.step_size
Gain (log(V / A))	6	cfg.keithley.gain_exponent and cfg.cal.preamp_gain_v_per_a
Tip Bias interval (mV) / Number of points / Frequency (traces)	5 / 5 / 50	cfg.vzero.interval_mv, n_points, every_n_traces
CV rate / cycles / scan rate / peaks	1000 / 1 / 100 / -1, +1	cfg.echem.*
Bias limit (config file only)	0.5 V	cfg.limits.bias_max_v

Everything else: chapter 51.

Appendix: the binding table

Generated from `stmgui/state.py` by `manual_build/gen_binding_table.py`. Igor: `Declare_STMBJ_Variables, Functions_STMBJ.ipf:5-198`, and the `value=` bindings in `Windows_STMBJ.ipf`.

Every panel control is bound to one row of this table. **Igor global** is the name the control was bound to in Igor; **lives in** is the attribute it writes now, dotted from a `GuiState` (`cfg.` is the `RigConfig` that every data file carries, `opts.` is `GuiOptions`, the panel-only state); **shown as** is the display unit where it differs from the stored one (Tip Bias is stored in volts, shown in mV); **limits** are Igor's `limits={lo, hi, increment}` (an increment gives the control arrow buttons); **default** is what a fresh panel shows.

Bool rows are checkboxes. Rows with no limits are display-only or unbounded, as in Igor.

51.1 DAQ tab

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_AcquisitionRate	Acquisition Rate	<code>cfg.ramp.sample_rate_hz</code>	float	100, 200000, 0	40000
G_WriteBufferSize	Write Buffer Size	<code>cfg.ramp.settle_samples</code>	int	25, 2000, 0	1000
G_FastReadWaveSize	Fast Read Wave Size	<code>opts.fast_read_wave_size</code>	int	25, 2000, 0	200
G_HighResOutputRange	HighRes Output Range (V)	<code>cfg.channels.bias_ao_range_v</code>	float	0, 10, 0	2.5
K_ZPiezoScale	Z piezo scale (nm/V)	<code>cfg.cal.piezo_nm_per_volt</code>	float		62
K_SenseScale	Z Sense scale (nm/V)	<code>opts.sense_scale_nm_per_v</code>	float		314
G_PiezoOffset_nm	Piezo offset (nm)	<code>opts.piezo_offset_nm</code>	float	0, 620, 0	0

51.2 Inputs tab

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_PullOutRate	Pull Rate (nm/s)	<code>cfg.ramp.pull_rate_nm_per_s</code>	float	0, 1000, 0	20
G_PseudoTotalLength_nm	Excursion (nm)	<code>cfg.ramp.pull_length_nm</code>	float	0.1, 500, 0	5
G_ConductanceThreshold	Contact Threshold (G0)	<code>cfg.ramp.engage_g0</code>	float	0, 10000, 0	0.5
G_MakeContactApproachStepSize	Contact step size (nm)	<code>cfg.ramp.approach_step_nm</code>	float	0, 50, 0	0.5
G_SeriesResistance	Series R (Ohm)	<code>cfg.keithley.series_resistance_ohm</code>	float		106130

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_EndOfTraceNoiseThreshold	Zero Cutoff (G0)	cfg.ramp.break_g0	float	0, 0.5, 0	0.0005
G_SmashFrequency	Smash Freq.	cfg.ramp.smash_every	int	0, inf, 0	50
G_SmashInSteps	Smash In	cfg.ramp.smash_in_nm	float		30
G_SmashOutSteps	Smash Out	cfg.ramp.smash_out_nm	float		-40
G_BiasSaveCheck	Save Bias	opts.save_bias	bool		off
G_CurrentSaveCheck	Save Current	opts.save_current	bool		off
G_PiezoWaveSaveCheck	Save Piezo Wave	opts.save_piezo_wave	bool		off
G_SenseSaveCheck	Save Sense	opts.save_sense	bool		off
G_HistSaveCheck	Save Hist	opts.save_hist	bool		on
G_VoltageRead	VoltageRead	opts.voltage_read	bool		off

51.3 Options tab: Push-Pull

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_PushPullCheck	Push-Pull	opts.push_pull_check	bool		off
G_NumPushPullCycles	Number of cycles	cfg.push_pull.cycles	int	1, inf, 1	2
G_InitialPullLength	Initial Pull (nm)	cfg.push_pull.initial_pull_nm	float		1
G_FinalPullLength	Final Pull (nm)	cfg.push_pull.final_pull_nm	float		4
G_PushPullLength	Push-Pull length (nm)	cfg.push_pull.push_pull_nm	float		1.5
G_HoldLength	Hold length (nm)	cfg.push_pull.hold_nm	float		1

51.4 Options tab: IV

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_IVCheck	IV	opts.iv_check	bool		off
G_IVMaxBias	MaxBias (V)	cfg.iv.max_bias_v	float		1
G_IVSignFlag	Sign:	cfg.iv.positive_first	bool		off
G_IVInitPull	Initial Pull (nm)	cfg.iv.init_pull_nm	float		3
G_IVFinPull	Final Pull (nm)	cfg.iv.final_pull_nm	float		4
G_IVCapLength	Cap length (nm)	cfg.iv.cap_nm	float		0.5
G_IVRampLength	Ramp length (nm)	cfg.iv.ramp_nm	float		2

51.5 Options tab: AC

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_ACHoldCheck	AC Hold	opts.ac_hold_check	bool		off
G_ACamp	Amp (V)	cfg.ac_hold.amp_v	float		0.8
G_ACFreq	Freq (kHz)	cfg.ac_hold.freq_khz	float		10
G_ACInitPullLength	Initial Pull (nm)	cfg.ac_hold.init_pull_nm	float		3
G_ACHoldLength	Hold length (nm)	cfg.ac_hold.hold_nm	float		3

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_ACFinPullLength	Final Pull (nm)	cfg.ac_hold.final_pull_nm	float		4

51.6 Options tab: High bias hold

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_HBHoldCheck	High Bias Hold	opts.hb_hold_check	bool		off
G_HBBias	Hold bias (V)	cfg.hb_hold.hold_bias_v	float		0.8
G_HBInitPullLength	Initial Pull (nm)	cfg.hb_hold.init_pull_nm	float		3
G_HBFinPullLength	Final Pull (nm)	cfg.hb_hold.final_pull_nm	float		4
G_HBHoldLength	Hold length (nm)	cfg.hb_hold.hold_nm	float		3
G_HBCapLengthIN	Cap Length IN (nm)	cfg.hb_hold.cap_in_nm	float		0.1
G_HBCapLengthFIN	Cap Length FIN (nm)	cfg.hb_hold.cap_fin_nm	float		0.1

51.7 Keithley Controls

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_CurrentVoltGain	Gain (log(V / A))	cfg.keithley.gain_exponent	int	3, 10, 1	6
G_CurrentSuppress Const	Suppress I value	cfg.keithley.suppress_const	float	-1000, 1000, 0	0
G_KeithleyBiasEnabled	Bias	cfg.keithley.enabled	bool		off
ZeroCheckBox	Zero Check	opts.zero_check	bool		on
SuppressCheckBox	Suppress I	opts.suppress_on	bool		off

51.8 Actuator and Piezo Controls

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_ActuatorStepSize	Step Size	cfg.actuator.step_size	int	1, inf, 1	5
G_ActuatorCounter	(counter)	opts.actuator_counter	int		0
G_PiezoDeltaZ_nm	Z (nm)	opts.piezo_step_nm	float	0, 100, 1	5

51.9 Bottom of the panel

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_TipBias	Tip Bias (mV)	cfg.ramp.bias_v	float (x1000 for display)	-5000, 5000, 0	100
G_PullOutNumber	Saved	opts.pull_out_number	int	0, inf, 0	1
G_StopNumber	Stop #	opts.stop_number	int	0, inf, 0	1001
G_PullOutAttempt	Attempts	opts.pull_out_attempt	int	0, inf, 0	0
OnCheck	Background Sampling (Beep is ON)	opts.bkgd_sampling	bool		off

51.10 Voltage_Offset panel

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_VoltageInterval	Tip Bias interval (mV)	cfg.vzero.interval_mv	float	0.1, 1000, 0	5
G_NumPoints	Number of points	cfg.vzero.n_points	int	3, 1000, 1	5
G_VzeroFrequency	Frequency (traces)	cfg.vzero.every_n_traces	int	1, 5000, 10	50
VzeroCheckBox	V0 check ON	opts.vzero_check	bool		off
G_Vzero	V0 (fit, mV)	opts.vzero_mv	float		0
G_Izero	I0 (fit, uA)	opts.izero_ua	float		0

51.11 EChem panel

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_CounterElectrode Bias	Counter Electrode Bias (mV)	cfg.echem.gate_mv	float		0
G_CVAcquisitionRate	CV Acquisition Rate (samps/s)	cfg.echem.cv_rate_hz	float	50, 40000, 0	1000
G_NumCVCycles	Number of Cycles	cfg.echem.cycles	int	0, 1000, 1	1
G_ScanRate	Scan Rate (mV/s)	cfg.echem.scan_rate_mv_per_s	float	0, 2000, 25	100
G_VoltagePeakOne	Voltage Peak 1 (V)	cfg.echem.peak_one_v	float	-2.5, 0, 0.25	-1
G_VoltagePeakTwo	Voltage Peak 2 (V)	cfg.echem.peak_two_v	float	0, 2.5, 0.25	1

51.12 Readouts

Igor global	Panel title	Lives in	Kind	Limits (lo, hi, inc)	Default
G_CurrentSampling ReadoutVal	I (uA)	opts.current_readout_ua	float		0
G_JunctionVoltage	V (mV)	opts.junction_voltage_mv	float		0
G_PiezoBiasSampling ReadoutVal	Piezo	opts.piezo_readout_v	float		0

Part III

The core: stmlab and the experiments

Overview

0.1 What this folder is

`python_code_entire_igor/` is the **complete** Igor STM break-junction setup, translated to Python: everything the procedures in `../igor_code/` could run, as one package (`stmlab/`) plus one folder per experiment (`experiments/`).

It is a strict superset of `../python_code/stmbj`, the minimal package that translated **only** the constant-bias experiment. That reference core was copied in unchanged as the first thirteen modules of `stmlab/` (`config`, `safety`, `daq`, `sim`, `instrument`, `approach`, `trace`, `analysis`, `storage`, `calibrate`, `actuator`, `bringup`, `main`) and extended with five new modules:

Module	Igor origin	What it adds
<code>stmlab/ramps.py</code>	CreateInputs, Functions_STMBJ.ipf:1398-1633	The four non-constant trajectories (push-pull, IV, AC hold, HB hold) as pure functions returning a ModeRamp
<code>stmlab/keithley.py</code>	SetUpGPIB_Keithley.ipf; SetGain, Controls_STMBJ.ipf:290	The Keithley 428 current amplifier over GPIB, the suppress-current calibration (TestVirtualGround, Functions_STMBJ.ipf:1097), and the series-resistance-corrected conductance
<code>stmlab/vzero.py</code>	OffsetVoltage/SaveOffset, Functions_STMBJ.ipf:920/993	The zero-current bias offset: measure it in contact, re-measure every N traces, feed it back into the applied bias
<code>stmlab/echem.py</code>	EChem_Module.ipf	The counter-electrode gate and both cyclic-voltammetry ramp builders, with Igor's cycle masking and a simulated electrochemical cell
<code>stmlab/xpiezo.py</code>	SetupXPiezo/MoveXPiezo, NanoPZ_Actuator_Functions_STMBJ.ipf:100	The lateral X piezo at 522 nm/V, with checked position and out-of-range refusal

One design decision carries the whole extension: every trajectory – constant-bias pull, push-pull cycle, IV sweep, AC or high-bias hold – is a (2, N) array of volts played through the same `Rig.play()`, captured and aligned by the same `trace.capture()`, and stored as the same raw-volt records. A mode is a different waveform plus a segments dictionary saying where its pieces live. Chapter 01 explains why this works.

0.2 The experiment menu

01_constant_bias – the reference measurement, and the one path that carries the 1 G0 self-check. A gold tip is crashed into and pulled out of a gold surface thousands of times at a fixed 100 mV bias; each pull thins the contact to a single atom ($1\text{ G0} = 2e^2/h$), sometimes catches a molecule, and ends in tunnelling. Accepted traces accumulate into a histogram of $\log_{10}(G/G_0)$; molecular conductance is a peak in that histogram, and the 1 G0 peak position is a whole-chain calibration check. Igor: MeasureBreakJunctions, Functions_STMBJ.ipf:1664.

02_push_pull – instead of breaking the junction once, re-form it: an initial pull, then per cycle {hold, push back in, hold, pull out}, then a final pull. The result is re-binding statistics for the same junction, cycle by cycle. Bias stays at the DC baseline throughout. Igor: CreateInputs push-pull branch, Functions_STMBJ.ipf:1461-1495.

03_iv_sweep – transport spectroscopy: pull to a held junction, sweep the bias in a triangle (0 to +max, through -max, back to 0, up to 1 V), with DC caps either side so the junction settles. One IV curve per held junction. Igor: IV branch, Functions_STMBJ.ipf:1497-1537.

04_ac_hold – hold the junction while the bias is a pure sine (default 0.8 V amplitude at 10 kHz, replacing the DC baseline entirely during the hold), for lock-in-style conductance analysis. Igor: AC-hold branch, Functions_STMBJ.ipf:1539-1564.

05_high_bias_hold – junction stability under field: hold at an elevated DC bias (default 0.8 V) between an initial and final pull. The hold bias can optionally be the measured Vzero offset, reproducing Igor's Voltage_Offset link (Functions_STMBJ.ipf:1595-1599). Igor: HB-hold branch, Functions_STMBJ.ipf:1566-1599.

06_echem_gate_cv – electrochemistry. A DC gate on the counter electrode shifts molecular levels relative to tip and substrate; a cyclic voltammogram sweeps the electrode potential in a triangle and records tip current. Two CV variants: high-res (drives the junction-bias channel; the tip is the working electrode) and low-res (drives the counter electrode on the second card). Igor: EChem_Module.ipf.

07_lateral_monolayer – walk the tip across a monolayer with the X piezo (522 nm/V on the second card), taking a batch of traces at each site, with a full withdraw before every lateral move so the tip is never dragged in contact. Igor: the LateralEXPT campaign, Setup1_STMBJ.ipf:91.

08_bias_series – Igor's rungo scripted campaign (Setup1_STMBJ.ipf:138): the constant-bias experiment repeated over a series of biases, one file per bias, resumable. Biases above the 0.5 V default limit are raised deliberately and loudly, never silently.

0.3 Folder map

python_code_entire_igor/	
run.py	dispatcher: python run.py list / <name> / <NN>
requirements.txt	numpy, h5py, matplotlib, pytest
README.md	
stmlab/	the package (reference core + 5 new modules)
config.py	every constant, as dataclasses -> chapter 02
safety.py	limits, interlocks, output parking
daq.py	DAQmx; the one verb play()
sim.py	the fake card that behaves like a junction
instrument.py	Rig: open tasks + tracked piezo position
approach.py	engage, coarse approach, smash, recover_headroom
trace.py	build_ramp, capture, trace_loop, TraceRecord
ramps.py	build_push_pull / build_iv / build_ac_hold / build_hb_hold
analysis.py	conductance, selection, log histogram, peak position
storage.py	HDF5 SessionWriter / Session, .ibw import
calibrate.py	zero, group delay, gain sweep, session_calibration

actuator.py	NanoPZ coarse actuator over serial
keithley.py	Keithley 428 over GPIB
vzero.py	zero-current offset workflow
echem.py	counter electrode + CV
xpiezo.py	lateral piezo
bringup.py	first-contact hardware checks 1-5
main.py	the constant-bias CLI (run/check/summarise/...)
experiments/	
01_constant_bias/ ... 08_bias_series/	
	run_experiment.py + README.md each
snippets/	16 runnable scripts (chapter 04)
tests/	93 tests; all pass without hardware
data/	session .h5 files land here by default
manual/	this manual

0.4 Quick start

From `python_code_entire_igor/`:

```
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt

.venv/Scripts/python run.py list           # what can I run?
.venv/Scripts/python run.py constant_bias --simulate -n 200
.venv/Scripts/python -m pytest tests/ -q    # 93 passed
```

Rules of the road:

1. **Simulate first.** Every experiment runs end to end with `--simulate` on any machine – no hardware, no NI drivers. The simulated card (`sim.py`) produces traces with a 1 G0 plateau, a molecular plateau, a tunnelling tail, an AI/AO group delay, and a noise floor, so the entire pipeline including alignment and selection is exercised, not stubbed.
2. `python run.py list` prints one line per experiment; `python run.py <name>` and `python experiments/<NN>_<name>/run_experiment.py` are exactly equivalent (`run.py` is a dispatcher, nothing more).
3. On the rig PC, additionally install `nidaqmx` (and `pyserial` for the NanoPZ, `pyvisa` for the Keithley) and drop `--simulate`. Run the bring-up ladder (chapter 03) before the first real trace.
4. Validate before touching hardware: `python run.py constant_bias check --config rig.json`.

0.5 Relationship to the neighbouring folders

`../igor_code/` is the original: `Setup1_STMBJ.ipf`, `Functions_STMBJ.ipf`, `Controls_STMBJ.ipf`, `Windows_STMBJ.ipf`, `EChem_Module.ipf`, `SetUpGPIB_Keithley.ipf`, `NanoPZ_Actuator_Functions_STM.ipf`. Every function translated here cites its Igor source and line numbers in its docstring, and Igor's defaults were kept wherever they were safe so an archived Igor run and a Python run are directly comparable. `igor_code/Source_Files/` (an AFM, an older Setup 3) is reference material and was not translated. The Igor GUI, the 0.25 s background readout, and the thermocouple were deliberately not translated; the reasons are in the top-level README.

`../python_code/` is the constant-bias reference: the `stmbj` package, built first, reviewed line by line against the Igor source, and covered by 38 tests. It stays frozen as the ground truth. This

folder's `stmLab/` began as a byte-for-byte copy of it, so the constant-bias path here is that same code, and those 38 tests are part of this folder's 93. When in doubt about the core behaviour, the reference package and `../STMBJ_Python_Manual.pdf` are the deep story.

The Igor comparison has NOT been done. The intended milestone — push archived `.ibw` blocks through `storage.split_igor_block` → `analysis.to_conductance` → `log_histogram` and check the 1 G0 peak agrees with Igor's to within 0.05 decade — is still outstanding, because no archived `.ibw` file was available. The 1 G0 peak *has* been reproduced to +0.02 decade, but from the **simulator**, which proves the analysis arithmetic is self-consistent and says nothing about whether it matches Igor. Until a real `.ibw` goes through it, "validated" is the wrong word for any path here. `storage.load_ibw` is written and untested for the same reason.

Trust order.

1. **Reviewed and unit-tested, self-consistent on synthetic data** — the constant-bias path. The strongest claim currently available.
2. **Index-for-index translations with geometry tests, exercised end to end on the simulator** — the four ramp modes, both campaigns, CV ramp construction, Vzero, the X piezo.
3. **Never met its instrument** — every `nidaqmx` call in the new modules, the Keithley over GPIB, the second card, the electrochemical cell.

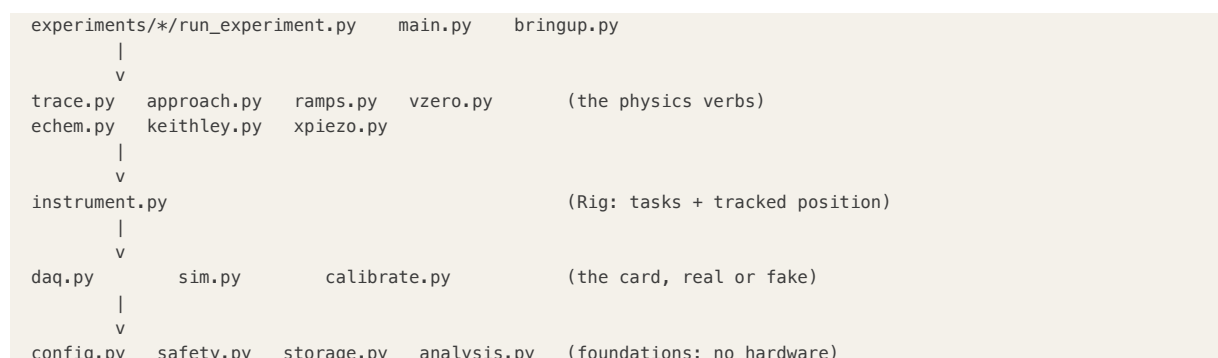
Simulate first, scope second, tip last.

CHAPTER 1

Architecture

1.1 The module stack

stmLab/ is layered, and the layering is enforced by what each module is allowed to import:



- **Foundations.** `config.py` holds every constant as dataclasses (chapter 02). `safety.py` holds the limits and interlocks and imports *only* `config` – its own docstring says that if it ever needs `daq` or `instrument`, a check has drifted into the wrong layer. `analysis.py` and `storage.py` are pure functions of arrays and files; they run anywhere, which is how the analysis chain was validated against archived Igor data before any hardware existed.
- **The card.** `daq.py` is DAQmx and nothing else; `sim.py` is a drop-in replacement with the same `configure()`/`play()` surface that behaves like a break junction (group delay of 37 samples, noise floor, 1 G0 plateau, molecular plateau, degradable tip). `Rig.open()` picks one or the other from `cfg.simulate`, so nothing above this line knows which it got.
- **The instrument.** `instrument.Rig` owns the open tasks *and* the last commanded piezo voltage, because the two must live and die together (see the interlock below).
- **The physics verbs.** `trace.py` (one pull, many pulls), `approach.py` (engage, coarse approach, smash), `ramps.py` (the four mode trajectories), `vzero.py`, `keithley.py`, `echem.py`, `xpiezo.py`.
- **Orchestration.** `main.py` and the experiment runners: `argparse`, `logging`, `SafeSession`, and sentences about the experiment. If a line in them is about DAQmx, the abstraction has leaked.

1.2 The one-verb play() model

The whole card is exposed through a single verb: `play(waveform) -> record`, where `waveform` is (2, N) volts on the outputs (row 0 the piezo command, row 1 the bias) and the return is (2, N) volts on the inputs (row 0 the junction voltage on ai0, row 1 the preamp output on ai1).

An approach step, a DC hold, a constant-bias pull, a push-pull cycle, an IV sweep – all are the same operation with different waveforms. This is not a simplification imposed on the hardware; it is how the hardware behaves: every AO start fires `ao/StartTrigger`, and AI is triggered off

exactly that terminal (daq.py), so every output automatically has a synchronised input record. Igor did the same thing with `WriteToHighRes` followed by a read (Functions_STMBJ.ipf:1153, 1347-1352); `Rig.hold()` is that pair.

Two consequences:

- **Python never times anything.** Trajectories are precomputed arrays; AO and AI run on the card's clock; loop overhead in Python lands *between* plays, never inside one.
- **Between plays the outputs hold their last sample.** The AO idle behaviour is set to `MAINTAIN_EXISTING_VALUE` (daq.py) – deliberately different from the `ZERO_VOLTS` the project-architecture document specified, because the step-wise approach depends on the piezo holding position across dozens of separate plays. Bring-up step 3 verifies this on a real card.

1.3 The tracked-piezo interlock

The most expensive accident on this rig is a coarse-actuator step taken while the fine piezo is extended: one coarse step advances further than the piezo's entire 10 V range, and taking it with the tip near the surface drives the tip into the sample.

The guard is `safety.require_retracted` (safety.py:96): a coarse step is refused unless the piezo is at or below `coarse_step_max_piezo_v` (0.1 V, i.e. 6.2 nm of residual extension at 62 nm/V). There is **no position sensor** – the check trusts the last *commanded* piezo voltage, tracked by `Rig`. That is why `Rig.play()` is the only path to the card from above the instrument layer, and why it updates the tracker itself from the waveform's final piezo sample rather than asking callers to remember: one direct `task.write()` that bypasses it and the interlock becomes confidently wrong. `Rig.coarse_step()` is in turn the only thing allowed to touch the actuator, and it re-checks the interlock and the step budget (2000 steps) on every single step, not just at the start of a loop.

1.4 Clamp versus refuse

Out-of-range commands are handled asymmetrically, and the asymmetry is deliberate (safety.py:42-69):

- **Piezo commands clamp, loudly.** A ramp that overshoots by a millivolt should be truncated, not aborted mid-pull – so `clamp_piezo` clips into [0, 10] V and logs every time it fires. A clamp that fires repeatedly shows up in the session log instead of passing silently.
- **Bias refuses.** `check_bias` raises `SafetyViolation` above `limits.bias_max_v` (0.5 V by default). There is no case where you meant to apply more bias than the limit and would be happy with silently less: quietly reducing it produces data labelled with a bias that was never applied. Experiments that legitimately need more – IV to 1 V, AC and HB holds to 0.8 V – raise the limit deliberately, with a loud log line, in their own runner.
- **Geometry refuses before hardware is touched.** A pull that would descend through the piezo floor is refused by `check_pull_headroom` (safety.py:72), because on a unipolar piezo a clipped ramp produces a silently truncated trace that is indistinguishable from a junction that broke early. The mode builders in `ramps.py` extend this to **both** bounds: push-pull moves the tip *above* the contact point, so `_assemble` (ramps.py:90) refuses trajectories that would cross the 10 V ceiling as well – a hazard Igor never guarded, since Igor only checked the retracted side.

SafetyViolation is never caught inside the package. SafeSession parks every output at 0 V on entry, on normal exit, on exception, and on Ctrl-C; 0 V is fully retracted on this unipolar piezo and passes no current, so the parked state is the safe state.

1.5 Raw volts in, raw volts stored

Traces carry and store **raw volts at the ADC**, never conductance (trace.py, storage.py). Conductance depends on three measured constants – preamp gain, preamp zero, bias – any of which can be revised after the fact; baking them into stored data turns a calibration error into a lost dataset instead of a re-analysis. The corollary is that the full RigConfig travels as JSON in every HDF5 file’s attributes, so the file is self-describing six months later. `Session.conductance(i, cal=...)` accepts a corrected calibration for exactly this reason. Every file holds traces of equal length – one mode, one geometry, per file.

1.6 How ModeRamp + trace.capture generalise the validated path

The validated constant-bias path was: `trace.build_ramp` (Igor’s CreateInputs, constant branch, Functions_STMBJ.ipf:1601) builds a Ramp; `trace.single_trace` plays it and cuts the trace out of the record. The generalisation splits that in two:

- `trace.capture(rig, ramp, index, bias_v)` plays **any** prebuilt ramp – a `trace.Ramp` or a `ramps.ModeRamp`, anything with `waveform`, `pre_pad`, `n_pull`, `spike_front`, `start_piezo_v` – recovers the AI/AO delay, applies the plausibility cut, and returns a `TraceRecord` (or `None`, meaning discard and retry). It is the single capture path for every experiment.
- `ramps.build_push_pull / build_iv / build_ac_hold / build_hb_hold` are pure functions (`cfg, start_piezo_v, mode_cfg=None, sample_rate_hz=None`) → `ModeRamp`, translated index-for-index from CreateInputs’ four GUI-selected branches (Functions_STMBJ.ipf:1461, 1497, 1539, 1566). They share Igor’s conventions exactly: lengths in nanometres converted at the pull rate (a 3 nm “hold” lasts as long as pulling 3 nm would), trajectories built relative to the contact point and shifted to `start_piezo_v`, bias in write-volts at `ao1` with Igor’s sign convention (baseline $-(\text{TipBias}/1000)$); the IV wave built positive then negated wholesale, Igor line 1533).

What ModeRamp adds over Igor is bookkeeping that survives the file: segments maps names like `cycle0_push` or `iv_ramp` to (start, stop) sample indices *within the pull*, and because `TraceRecord` is already cut (pads and delay removed), those indices apply to the stored arrays directly. `meta` carries the mode’s own numbers (cycles, max bias, hold frequency). The one caveat is documented in capture: `TraceRecord.displacement_nm` assumes a constant pull rate and does not apply to mode ramps – use segments instead.

The pre/post pads (400 samples each) are also an addition to Igor: the leading pad gives the 4461’s decimation filter somewhere to settle that is not the metallic-contact region, and the trailing pad keeps the alignment spike inside the record after the group delay shifts it later.

1.7 The alignment spike, identical in every mode

Every ramp – constant pull and all four modes – ends the same way: a short spike written on the **bias** channel into [front+1, back-1] at $+(\text{TipBias}/1000)$ (the negation of the baseline),

with the front edge 7.5 ms and the back edge 2.5 ms before the end of the wave. That is Igor's own construction, applied in every `CreateInputs` branch (`Functions_STMBJ.ipf:1618-1631`), and `ramps._assemble` reproduces it for the modes exactly as `trace.build_ramp` does for constant bias.

The spike is read straight back on `ai0`, and `analysis.find_alignment_edge` locates its rising edge to sub-sample precision. The lag between where the spike was written and where it appears is the card's AI/AO group delay plus trigger skew – tens of samples on the 4461 – and measuring it **per trace** means it never has to be assumed and survives a sample-rate change. `trace._measure_delay` then applies a plausibility window (between -0.02 and +0.05 of a second's worth of samples): a delay of the wrong order means the search locked onto noise, and the trace is discarded rather than shifted by a thousand samples. Because capture is one code path, this alignment – including the discard-and-retry behaviour – is *identical* for a constant pull, a push-pull cycle, an IV sweep, and a hold. A missing spike with a measured `cal.group_delay_samples` falls back to that constant; with neither, the trace is discarded.

One special case: `build_hb_hold` keeps the spike at `+TipBias/1000` even though its hold runs at a different bias, and the Vzero-corrected constant pull (`trace.build_ramp(bias_offset_v=..)`) applies the offset to the baseline only, never to the spike – both exactly as Igor did (`Functions_STMBJ.ipf:1609-1614, 1631`), so the edge detector always sees the same edge.

Configuration reference

The source of truth is `stmlab/config.py`. Everything below is generated from reading it; if this chapter and the file ever disagree, the file wins.

`RigConfig` is one object holding thirteen dataclasses, split by lifecycle: `ChannelMap` changes when the rig changes, `SafetyLimits` is set once and rarely touched, `RampConfig` is tuned per experiment, `Calibration` is re-measured every session, and the mode/support configs (`PushPullConfig` through `XPiezoConfig`) belong to individual experiments but are present on every `RigConfig` so they travel in every data file. The whole config is written as JSON into every HDF5 file's attributes – a file whose gain you cannot reconstruct six months later is not data.

Igor origins: the channel names and scale constants come from `Setup1_STMBJ.ipf:25-41`, the `G_*` globals from `Declare_STMBJ_Variables (Functions_STMBJ.ipf:5-198)`, and the mode branches' globals from `Functions_STMBJ.ipf:91-123`. Where a default deliberately differs from Igor's, the table says so.

Module-level constant: `G0_SIEMENS = 7.7480917346e-5` – the conductance quantum $2e^2/h$, Igor's `K_G0` (`Setup1_STMBJ.ipf:25`).

2.1 ChannelMap (cfg.channels)

Which physical channel carries what. AI channels enter the DAQmx task in the order of `ai_channels` – voltage first, current second, the same order as Igor's `SetUpHRTaskID1 (Functions_STMBJ.ipf:243)`.

Field	Default	Unit	Igor origin	Meaning
<code>device</code>	<code>"Dev1"</code>	–	<code>Setup1_STMBJ.ipf:25-34</code>	DAQmx alias of the PXI-4461
<code>ao_piezo</code>	<code>"ao0"</code>	–	<code>Setup1_STMBJ.ipf</code>	Z-piezo command, via the piezo driver box
<code>ao_bias</code>	<code>"ao1"</code>	–	<code>Setup1_STMBJ.ipf</code>	Junction bias output
<code>ai_voltage</code>	<code>"ai0"</code>	–	<code>Setup1_STMBJ.ipf</code>	Junction-voltage monitor input
<code>ai_current</code>	<code>"ai1"</code>	–	<code>Setup1_STMBJ.ipf</code>	Current-preamp output input
<code>ai_range_v</code>	<code>10.0</code>	V	–	Symmetric range of both AI channels

Field	Default	Unit	Igor origin	Meaning
bias_ao_range_v	2.5	V	G_HighResOutput Range	Card-enforced range of the bias AO channel (the piezo AO range comes from SafetyLimits, so hardware clip and software check can never disagree – Igor passed G_PiezoChan Low/High into MXCreateAOVoltage Chan, Controls_STMBJ.ipf:152)
expected_product_type	"PXI-4461"	–	–	Product-type check in verify_devices(); None skips it
low_res_device	None	–	Setup1_STMBJ.ipf:36-41 ("dev2")	Igor's second card: piezo sense readback, X piezo on ao0, EChem counter electrode on ao1. None means "this rig has one card" and every feature needing it refuses cleanly

Class constants (ClassVar, deliberately excluded from asdict() and the data files because they describe this code's array layout, not the rig): ROW_VOLTAGE = 0, ROW_CURRENT = 1 (input rows), ROW_PIEZO = 0, ROW_BIAS = 1 (output rows). Derived helpers: path(ch), ai_channels, ao_channels, and ao_start_trigger (<device>/ao/StartTrigger – the terminal AI triggers off, derived rather than hardcoded).

2.2 SafetyLimits (cfg.limits)

Values beyond which something breaks – not values you intend to use. All piezo limits are expressed at the DAQ output, before the driver box, because the displacement calibration (cal.piezo_nm_per_volt) is also DAQ-side and already folds in the box's gain. This piezo is **unipolar, 0 to 10 V**: Igor's own bipolar defaults (G_PiezoChanLow/High = -10/+10, Functions_STMBJ.ipf:1294) would drive it 10 V the wrong way, and Igor itself flagged the unipolar case at Functions_STMBJ.ipf:44.

Field	Default	Unit	Igor origin	Meaning
piezo_ao_min_v	0.0	V	replaces G_PiezoChanLow (-10)	Piezo output floor; 0 V is fully retracted, the safe state
piezo_ao_max_v	10.0	V	G_PiezoChanHigh	Piezo output ceiling
bias_max_v	0.5	V	– (Igor's tip bias was 0.1 V; rungo reached 1.1 V)	Hard refusal, not a clamp; IV (1 V), AC (0.8 V) and HB (0.8 V) runners raise it deliberately with a loud log line

Field	Default	Unit	Igor origin	Meaning
preamp_saturation_v	9.5	V	–	Above this the preamp output is railed and the reading is meaningless; a railed channel counts as contact
max_coarse_steps	2000	steps	–	Approach runaway budget
coarse_step_max_piezo_v	0.1	V	–	THE interlock: a coarse step is refused unless the piezo is at or below this (0.1 V = 6.2 nm residual extension)

2.3 RampConfig (cfg.ramp)

Trajectory and sampling; Igor's Declare_STMBJ_Variables defaults.

Field	Default	Unit	Igor origin	Meaning
sample_rate_hz	40_000.0	Hz	G_AcquisitionRate	Requested AI/ AO sample rate (the <i>granted</i> rate is what time axes use)
pull_length_nm	5.0	nm	G_PseudoTotalLength_nm	Length of one constant-bias pull; mode runners overwrite it with the mode's net descent so the headroom pre-check matches Retraction speed; also the rate at which every mode length is converted to samples
pull_rate_nm_per_s	20.0	nm/s	G_PullOutRate	DC junction bias baseline
bias_v	0.100	V	G_TipBias (100 mV)	Contact threshold. Deliberately lowered: 5 G0 at $R_f = 1e6 \text{ V/A}$ and 100 mV needs 38.7 V at the ADC, unreachable on a +/-10 V input; 0.5 G0 needs 3.87 V. Saturation additionally counts as contact
engage_g0	0.5	G0	G_ConductanceThreshold (Igor: 5)	Below this the junction counts as broken
break_g0	5e-4	G0	G_EndOfTraceNoiseThreshold	Fine-approach step size
approach_step_nm	0.5	nm	G_MakeContactApproachStepSize	Separation step magnitude (applied negative)
retract_step_nm	5.0	nm	–	

Field	Default	Unit	Igor origin	Meaning
settle_samples	1000	samples	G_WriteBufferSize	Samples played/captured per DC move (Igor wrote 1000 per WriteToHighRes)
settle_discard	800	samples	cf. G_FastReadWaveSize (Igor averaged 200)	Leading samples discarded before averaging a DC record
pre_pad_samples	400	samples	– (new)	Constant lead-in so the 4461's decimation filter settles outside the region of interest
post_pad_samples	400	samples	– (new)	Constant tail keeping the delayed alignment spike inside the record
spike_front_ms	7.5	ms	Functions_STMBJ.ipf:1631	Alignment-spike front edge, before the end of the ramp
spike_back_ms	2.5	ms	Functions_STMBJ.ipf:1631	Alignment-spike back edge
smash_every	50	attempts	SmashFun, Functions_STMBJ.ipf:1375	Tip conditioning interval; 0 disables
smash_in_nm	30.0	nm	SmashFun	Smash drive-in distance
smash_out_nm	-40.0	nm	SmashFun	Smash pull-back distance
traces_target	1000	traces	G_StopNumber – G_PullOutNumber	Default accepted-trace target
max_attempts	100_000	attempts	–	Attempt budget for a run
piezo_park_v	0.0	V	–	Idle fine-piezo position; 0 V is retracted and is where the card parks itself
piezo_headroom_v	0.02	V	cf. Functions_STMBJ.ipf:44 (G_PiezoOffset_nm "MUST BE LARGER THAN EXCURSION SIZE")	Reserve above the floor; a pull that would descend below it is refused rather than clipped

Derived properties: `n_pull_samples` (Igor's `G_NumPtsInAppliedWaves`, $\text{round}(\text{pull_length} / \text{pull_rate} * \text{sample_rate})$), `n_record_samples` (pull plus both pads), `seconds_per_pull`.

2.4 Calibration (cfg.cal)

Measured constants; produced by `calibrate.py`, saved per session.

Field	Default	Unit	Igor origin	Meaning
preamp_gain_v_per_a	1e6	V/A	G_CurrentVoltGain (stored as the exponent 6)	Transimpedance of the current preamp

Field	Default	Unit	Igor origin	Meaning
current_zero_v	0.0	V	–	Preamp resting offset; hundreds of microvolts, drifts with temperature, re-measure every session (validate warns while it is 0)
piezo_nm_per_volt	62.0	nm/V	K_ZPiezoScale	Displacement per volt at the DAQ output, end to end including the driver box
hv_amp_gain	None	–	–	Driver-box gain if ever measured separately; documentation only, nothing multiplies by it
bias_output_sign	-1.0	–	Functions_STMBJ.ipf:347	Igor writes $-(\text{TipBias}/1000)$ to ao1
voltage_input_sign	-1.0	–	Functions_STMBJ.ipf:358	Igor negates ai0 to recover the junction voltage
ai_gain_error	1.0	–	–	Slope from the DC loopback sweep; identity until measured
ai_offset_v	0.0	V	–	Offset from the DC loopback sweep
granted_sample_rate_hz	None	Hz	–	Filled at run time; a delta-sigma card does not always grant the requested rate, and every time axis must use the granted one
group_delay_samples	None	samples	–	Fixed AI/AO delay if measured; left None, each trace recovers its own delay from the alignment spike

Conversion methods: volts_to_amps, volts_to_g0, g0_to_volts (used by the validator to catch clipping before acquiring), nm_to_piezo_volts, piezo_volts_to_nm.

2.5 ActuatorConfig (cfg.actuator)

Coarse-approach actuator. Igor drove a Newport NanoPZ over serial with 0M0 then 0PR<n> (NanoPZ_Actuator_Functions_STM.ipf:6-23).

Field	Default	Unit	Igor origin	Meaning
kind	"none"	–	–	"none" (approach by hand), "nanopz" (serial), or "simulated"
port	"COM1"	–	Igor hardcoded Com1	Serial port; USB connections usually appear as COM3 or higher
baud	19200	baud	NanoPZ_Actuator_Functions_STMBJ.ipf:1780	Serial rate (8N1, CRLF line endings)
step_size	5	steps	G_ActuatorStepSize	Steps per coarse move; negative counts move toward the sample
settle_s	0.10	s	–	Wait after each coarse step
stop_current_ua	0.1	uA	G_CurrentSamplingReadoutVal, Functions_STMBJ.ipf:1780	Current above which the coarse approach stops

2.6 Mode configs

Igor: CreateInputs' four GUI-selected branches (Functions_STMBJ.ipf:1461-1599) and their globals (:91-123). All lengths are nanometres *at the pull rate* – Igor's parameterisation, so a 3 nm "hold" lasts as long as pulling 3 nm would (150 ms at the default 20 nm/s).

2.6.1 PushPullConfig (cfg.push_pull)

Igor declared these globals with no defaults (set from the GUI, Functions_STMBJ.ipf:91-96); the values below are working values, not Igor's.

Field	Default	Unit	Igor origin	Meaning
initial_pull_nm	3.0	nm	G_InitialPullLength	Pull before the first cycle
push_pull_nm	1.0	nm	G_PushPullLength	Excursion of each push and each pull within a cycle
hold_nm	1.0	nm	G_HoldLength	Hold length either side of each push
final_pull_nm	4.0	nm	G_FinalPullLength	Pull after the last cycle, breaking the junction for good
cycles	2	–	G_NumPushPullCycles	Cycles per trace

Net descent is `initial_pull_nm + final_pull_nm`: the cycles net to zero.

2.6.2 IVConfig (cfg.iv)

Igor defaults, Functions_STMBJ.ipf:99-105.

Field	Default	Unit	Igor origin	Meaning
init_pull_nm	3.0	nm	G_IVInitPull	Pull before the held sweep
final_pull_nm	4.0	nm	G_IVFinPull	Pull after it
cap_nm	0.5	nm	G_IVCapLength	DC settling cap either side of the sweep
ramp_nm	2.0	nm	G_IVRampLength	Length (i.e. duration at the pull rate) of the triangular sweep window
max_bias_v	1.0	V	G_IVMaxBias	Sweep peak, both polarities
positive_first	False	–	G_IVSignFlag	Negate the ramp segment a second time (Igor line 1535), so the sweep goes to the POSITIVE apex first. Left False, Igor's wholesale negation at line 1533 makes the sweep go negative first.

2.6.3 ACHoldConfig (cfg.ac_hold)

Igor defaults, Functions_STMBJ.ipf:108-114.

Field	Default	Unit	Igor origin	Meaning
init_pull_nm	3.0	nm	G_ACInitPullLength	Pull before the hold
final_pull_nm	4.0	nm	G_ACFinPullLength	Pull after it
hold_nm	3.0	nm	G_ACHoldLength	Hold length; the sine replaces the DC baseline entirely here (Igor line 1564)
cap_nm	0.5	nm	G_ACCapLength	DC cap either side of the hold
amp_v	0.8	V	G_ACAmp	Sine amplitude
freq_khz	10.0	kHz	G_ACFreq	Sine frequency, in kHz as Igor's was

2.6.4 HBHoldConfig (cfg.hb_hold)

Igor defaults, Functions_STMBJ.ipf:117-123.

Field	Default	Unit	Igor origin	Meaning
init_pull_nm	3.0	nm	G_HBInitPullLength	Pull before the hold
final_pull_nm	4.0	nm	G_HBFinPullLength	Pull after it
hold_nm	3.0	nm	G_HBHoldLength	Hold length
cap_in_nm	0.1	nm	G_HBCapLengthIN	Cap before the hold
cap_fin_nm	0.1	nm	G_HBCapLengthFIN	Cap after the hold

Field	Default	Unit	Igor origin	Meaning
hold_bias_v	0.8	V	G_HBBias	Hold bias, written as -hold_bias_v like the baseline; overridden by the measured Vzero when build_hb_hold(vzero_mv=...) is passed (Igor lines 1595-1599)

2.7 VzeroConfig (cfg.vzero)

The Voltage_Offset workflow: find the applied bias that nulls the current. Igor: Offset Voltage/SaveOffset, Functions_STMBJ.ipf:920-1041.

Field	Default	Unit	Igor origin	Meaning
interval_mv	5.0	mV	G_VoltageInterval	The bias sweep is +/- this
n_points	5	points	G_NumPoints	Points per polarity (2N total)
every_n_traces	50	traces	G_VzeroFrequency	VzeroTracker.maybe_measure re-measures at this interval; <= 0 disables
settle_s	0.05	s	Igor Sleep/T 3 (3 ticks = 50 ms)	Wait after each bias step

2.8 KeithleyConfig (cfg.keithley)

Keithley 428 current amplifier over GPIB. Igor: SetUpGPIB_Keithley.ipf (address 22) and Set Gain/SetCurrentSuppress, Controls_STMBJ.ipf:290-311, 460-477.

Field	Default	Unit	Igor origin	Meaning
resource	"GPIB0::22::INSTR"	–	ibdev={0,22,...}	VISA resource string
gain_exponent	6	log10(V/A)	G_CurrentVoltGain	Transimpedance gain exponent; set_gain programs H6R<g>X
suppress_const	0.0	–	G_CurrentSuppress Const	Constant from which the suppress current is derived: suppress_ua = const * 10**(3 - gain)
series_resistance_ohm	106130.0	ohm	G_SeriesResistance	Series resistance for the corrected conductance when the Keithley sources the bias
enabled	False	–	G_KeithleyBias Enabled (started 0)	Whether the Keithley's internal bias source is in use

2.9 EChemConfig (cfg.echem)

Electrochemistry. Igor: EChem_Module.ipf; the counter electrode lived on the low-res card's ao1 (Setup1_STMBJ.ipf:11), the high-res CV drove the junction bias channel (dev1 / ao1) instead.

Field	Default	Unit	Igor origin	Meaning
counter_electrode_channel	"ao1"	–	Setup1_STMBJ.ipf:11	Channel on channels.low_res_device carrying the gate
gate_mv	0.0	mV	G_CounterElectrode Bias	DC gate potential; updated by CounterElectrode.set_mv so it travels in every data file (Igor saved it as ParameterWave[18])
cv_rate_hz	1000.0	Hz	G_CVAcquisition Rate	CV sample rate
scan_rate_mv_per_s	100.0	mV/s	G_ScanRate	Potential sweep rate
peak_one_v	-1.0	V	G_VoltagePeakOne	First triangle vertex
peak_two_v	1.0	V	G_VoltagePeakTwo	Second triangle vertex
cycles	1	–	G_NumCVCycles	CV cycles per run

2.10 XPiezoConfig (cfg.xpiezo)

Lateral X piezo on the low-res card, for monolayer experiments. Igor: SetupXPiezo/MoveXPiezo, NanoPZ_Actuator_Functions_STM.ipf:25-100.

Field	Default	Unit	Igor origin	Meaning
channel	"ao0"	–	Setup1_STMBJ.ipf:40	Channel on channels.low_res_device
nm_per_volt	522.0	nm/V	K_XPiezoScale	Lateral scale – almost an order of magnitude coarser than the Z piezo's 62 nm/V
min_v	0.0	V	–	Output floor; moves outside the range are refused (Igor did not check)
max_v	10.0	V	–	Output ceiling

2.11 RigConfig (top level)

Field	Default	Meaning
channels .. xpiezo	the thirteen dataclasses above	See each table
simulate	False	Run against sim.SimulatedDaqSession and the simulated instruments instead of hardware

Field	Default	Meaning
notes	""	Free text, travels in every data file

Serialisation: `to_dict()` / `to_json(path)` and `from_dict()` / `from_json(path)`. `from_dict` tolerates unknown keys (with a logged warning) so files written by older versions of the package still open – re-analysis with a corrected calibration is the whole reason raw volts are stored.

2.12 `validate(cfg)`

`validate` checks a config **before any hardware is touched**. It raises `ConfigError` on fatal problems and returns a list of warning strings otherwise; every runner calls it first and logs the warnings. Fatal checks include: 1 G0 clipping the AI range, bias over its own limit or zero, engage threshold at or below the break threshold, a pull longer than the piezo range or its usable headroom, a park position or interlock threshold outside the piezo range, non-positive sample rate, a pull under 100 points, and an alignment spike that does not fit in the pull. Warnings include: 1 G0 above preamp saturation (usually acceptable), an engage threshold only reachable by saturation (the failure Igor's own defaults would have hit), a break threshold at the noise floor, a granted rate differing from the requested one, an unmeasured `current_zero_v`, an unknown `hv_amp_gain`, and a pull using more than half of the usable piezo range.

Hardware

3.1 The rig as wired

This rig runs a **single NI PXI-4461** (two AO, two AI, delta-sigma), which differs from the two-card layout in `project-architecture.pdf` but matches the Igor code exactly (`config.py`'s module docstring): the junction bias comes from `ao1` on the same card, and there is no current-suppression output on the DAQ – suppression, when used, is the Keithley 428's own.

Channel	Signal	Range	Notes
Dev1/ao0	Z-piezo command	0 to +10 V (card-enforced from <code>SafetyLimits</code>)	Through the piezo driver box; 62 nm/V end to end (<code>K_ZPiezoScale</code>). Unipolar: 0 V is fully retracted, the safe state
Dev1/ao1	Junction bias	+/-2.5 V (<code>G_HighRes OutputRange</code>)	Written as $-(\text{TipBias}/1000)$, Igor's sign convention (<code>Functions_STMBJ.ipf:347</code>); also carries the alignment spike, and the high-res CV repurposes it
Dev1/ai0	Junction voltage monitor	+/-10 V	Negated to recover the junction voltage (<code>Functions_STMBJ.ipf:358</code>); the alignment spike is found here
Dev1/ai1	Current preamp output	+/-10 V	$R_f = 1e6 \text{ V/A}$ by default; readings above 9.5 V count as railed, and railed counts as contact

AI is hardware-triggered off `/Dev1/ao/StartTrigger`, so every output play has a synchronised input record and Python never times anything. The AO idle behaviour is `MAINTAIN_EXISTING_VALUE` so the piezo holds position between plays; if the card refuses it, `daq.py` warns loudly and bring-up step 3 is the test that decides whether the approach can work at all.

The piezo driver box sits between `ao0` and the piezo. Its gain is deliberately not used in any arithmetic: `cal.piezo_nm_per_volt = 62.0` is an end-to-end number for the whole chain, and all piezo limits are expressed at the DAQ output (`SafetyLimits` docstring). The coarse approach is a Newport NanoPZ; the STM head carries the gold tip over a gold-on-mica substrate, with the current preamp reading the tip.

3.2 What needs the second card

Igor's rig had a second, low-res card ("dev2", Setup1_STMBJ.ipf:36-41). In this package it is `channels.low_res_device`, default `None` – and `None` means every feature that needs it **refuses cleanly** instead of guessing:

Feature	Channel on the second card	Refusal without it
Lateral X piezo (experiment 07)	<dev2>/ao0, 0-10 V, 522 nm/V (K_XPiezoScale, Setup1_STMBJ.ipf:40)	XPiezoError: no low-res device configured (xpiezo.py setup)
ECHEM counter electrode / gate	<dev2>/ao1, +/-5 V (Igor MXCreateA0VoltageChan(-5, 5))	ECHEMError: no low-res device configured (echem.py CounterElectrode.on)
Low-res CV (run_cv(kind="lowres"))	Same counter-electrode channel, driven +/-10 V as Igor did	ECHEMError: lowres CV needs channels.low_res_device
Piezo sense readback	Igor read the driver's output voltage back and stored it as POExtension	Not translated: with one card the displacement axis is the commanded trajectory (analysis.py displacement_nm); the piezo is open-loop either way

The high-res CV needs **no** second card – it drives Dev1/ao1, the junction-bias channel, with the tip as working electrode. That is exactly why it must be treated with care (see the pre-flight checks below).

3.3 The Keithley 428 (GPIB address 22)

The current amplifier is a Keithley 428 on GPIB0::22::INSTR (Igor: `ibdev={0,22,...}`, SetupGPIB_Keithley.ipf:9), reached through pyvisa and an NI GPIB interface. It speaks a terse command language where `X` executes the buffer; the init string on open is Igor's `C1P0B0N0X` – zero-check on, filter off, bias off, suppress off – so the amplifier wakes up inert, and `close()` leaves it inert again (`C1B0N0X`).

What it controls, all in `stmlab/keithley.py`:

- **Gain:** `set_gain(exponent)` programs 10^{exponent} V/A ($H6R < g < X$) and re-derives Igor's globals: conversion $10^{*(6-g)}$ uA/V and suppress `suppress_const * 10^{*(3-g)}` uA (Controls_STMBJ.ipf:304-310). The config's `cal.preamp_gain_v_per_a` must agree with the exponent actually programmed or every conductance is scaled.
- **Current suppression:** `set_suppress_ua / suppress_enable`, and `find_suppress(rig, keithley)` – Igor's TestVirtualGround (Functions_STMBJ.ipf:1097): sweep the suppress -1 to +1 uA in 21 steps at zero bias, fit the middle points, set the zero crossing.
- **Keithley-sourced bias:** `bias_enable (B1X)` and `set_bias_mv` (quantised to 5 mV, as Igor's Set TipBiasVoltage did). When the Keithley sources the bias, the junction voltage is no longer measured at ai0 and conductance needs `series_corrected_conductance(current_a, bias_v, r_series)` – Igor's formula from GenerateTrace:390 with `G_SeriesResistance = 106130 ohm`.

SimulatedKeithley has the same surface and records every command, so the Keithley experiments run on a laptop.

3.4 The NanoPZ (serial)

The coarse actuator is a Newport NanoPZ controller on a serial port: 19200 baud, 8N1, CRLF line endings (Igor's in=2, out=2). Igor hardcoded Com1; on USB it is usually COM3 or higher – `actuator.list_serial_ports()` or bring-up step 5 finds it, and if nothing appears at all the controller may be on a proprietary USB driver rather than a virtual COM port, which pyserial cannot reach.

Protocol (`actuator.py`): `0M0` motor on, `0PR<n>` relative move (a **negative** count moves toward the sample – `StepActuatorCloser`, `NanoPZ_Actuator_Functions_STM.ipf:22`), `0ST` stop, `0TS?/0TP?` status and position (replies echo the command; the reply parser strips it). One configured step is `actuator.step_size = 5` controller steps (`G_ActuatorStepSize`), and each `Rig.coarse_step()` passes the retracted- piezo interlock and the 2000-step budget first. A step advances the tip further than the piezo's whole 10 V range: nothing but `Rig.coarse_step` is allowed to call the actuator, and it never runs with the piezo extended above 0.1 V.

3.5 What refuses cleanly when hardware is absent

The package is importable and testable with no instrument stack installed; every hardware dependency is imported inside the function that needs it.

Missing	What happens
nidaqmx not installed	<code>Rig.open()</code> fails at <code>from .daq import DaqSession</code> unless <code>cfg.simulate</code> is set; <code>--simulate</code> replaces the card with <code>sim.SimulatedDaqSession</code> and never imports <code>nidaqmx</code> . <code>bringup</code> prints "Cannot run ... must run on the rig PC" and exits 1
Configured device absent or renamed	<code>safety.verify_devices</code> (run by <code>SafeSession.__enter__</code>) raises <code>SafetyViolation</code> , naming what is present; aliases move when cards change slots
Wrong card model	<code>verify_devices</code> refuses: the channel map assumes the 4461's pinout, and driving the piezo ramp into a preamp input destroys the preamp
<code>low_res_device = None</code>	X piezo and EChem gate/low-res CV raise <code>XPiezoError/EChemError</code> with an explicit message (see above)
pyserial missing / port wrong	<code>ActuatorError</code> , listing every serial port the OS can see; <code>actuator.kind = "none"</code> makes the coarse approach a by-hand step (<code>coarse_approach</code> raises <code>ApproachError</code> telling you so)
pyvisa / GPIB missing	<code>Keithley428.open()</code> fails at <code>import pyvisa</code> ; <code>cfg.simulate</code> gives <code>SimulatedKeithley</code> instead
igor2 missing	Only <code>storage.load_ibw</code> (reading archived Igor waves) needs it

Simulated stand-ins exist for every instrument: the card (`sim.py`), the actuator (`SimulatedActuator`), the Keithley (`SimulatedKeithley`), the counter electrode (`SimulatedCounterElectrode`, plus a simulated electrochemical cell in `run_cv`), and the X piezo (`SimulatedXPiezo`).

3.6 The bring-up ladder

First contact with the card is five ordered steps (stm32lab/bringup.py), run on the rig PC as python run.py constant_bias bringup <step> or python -m stm32lab.bringup <step>. **Every step runs with the piezo disconnected and no tip near the sample.** Each prints PASS / CHECK / FAIL rows; do not continue past a FAIL.

Step	Name	Wiring	What it decides
1	devices	none	The card exists, is a PXI-4461, has the four channels; the granted sample rate (a delta-sigma card grants discrete rates – put the granted value in the config)
2	timing	one BNC: ao1 -> ai0	AI is genuinely triggered off ao/StartTrigger: group delay measured 25 times, scatter must be well under one sample or displacement and current are not aligned
3	output hold	one BNC: ao0 -> ai0	The decisive test for the approach: does ao0 keep its value between plays (MAIN-TAIN_EXISTING_VALUE), and does the piezo line ever go negative (it must not – unipolar)
4	preamp gain	known resistor (default 1 MOhm) at the preamp input	100 mV across 1 MOhm at 1e6 V / A must read 0.100 V; measures the preamp zero and the noise floor in G0. A gain error here scales every conductance the rig ever reports
5	coarse actuator	NanoPZ on USB	Serial ports listed, controller replies to 0TS? / 0TP?; motion only with --move, and it steps <i>away</i> from the sample first

The last rung is deliberately not automated (bringup.py's module docstring): only after the electrical steps pass, connect the piezo, watch ao0 on a scope through one engage(), and confirm it ramps 0 to 10 V and never goes negative. Only then does a tip go in.

3.7 Before an EChem or Keithley experiment

The bring-up ladder validates the 4461 path only. The hardware-only code in `keithley.py`, `echem.py` and `xpiezo.py` has never met its instruments (see the trust order in chapter 00), so check the following first.

Keithley:

- The GPIB chain answers at address 22 (pyvisa resource `GPIB0::22::INSTR`) and `open()` reaches the inert init state – with the amplifier's input **disconnected from the tip** the first time.
- `keithley.gain_exponent` matches `cal.preamplifier_gain_v_per_a`; if you change the gain with `set_gain`, change the calibration with it.
- Run `find_suppress` with the applied bias zeroed and restored around the call, as Igor's Find-Suppress button did (`Controls_STMBJ.ipf:479`); the amplifier must be out of zero-check or the sweep is flat and the fit refuses.
- If the Keithley is to source the bias (B1X): the junction voltage is no longer at `ai0`, conductance must use `series_corrected_conductance`, and `series_resistance_ohm` (default 106130) must be the measured value for this wiring.

EChem:

- The gate and the low-res CV need `channels.low_res_device` set and the counter electrode actually wired to `<dev2>/ao1`; both refuse otherwise.
- The **high-res CV repurposes Dev1/ao1, the junction-bias output**, with the tip as working electrode. Withdraw the tip (`rig.withdraw()`) before `run_cv(kind="highres")`, expect the bias to be whatever the ramp left – the function parks the channel at 0 V when it finishes, as Igor reset the card afterwards – and unplug nothing mid-run.
- CV peak potentials (`peak_one_v/peak_two_v`, default -1/+1 V) exceed the 0.5 V junction-bias limit by design; they are electrode potentials in solution, applied by a dedicated task with its own range. Confirm the cell chemistry tolerates them before starting.
- Simulate first: `run_cv` under `cfg.simulate` produces the classic capacitive-plus-faradaic hysteresis loop and exercises the full analysis.

CHAPTER 4

The snippets

Folder: snippets/. Sixteen runnable scripts. Fifteen need no hardware.

Reading code tells you what it does. Running it tells you what it *means*. Each snippet prints what it found, explains it, and where useful draws it.

```
cd python_code_entire_igor
./venv/bin/python snippets/01_numbers.py
```

Use the project virtualenv, not your system Python. The snippets need numpy (and h5py for 07); a system interpreter almost certainly has neither. On Windows the interpreter is `venv\Scripts\python`. If `.venv` is missing:

```
python3 -m venv .venv
./venv/bin/pip install -r requirements.txt
```

Any working directory is fine. Each snippet puts the package on `sys.path` itself and writes figures to an absolute path, so `../venv/bin/python 01_numbers.py` from inside `snippets/` works exactly as well as running from the folder root.

Run one with the wrong interpreter and it prints a single absolute command to use instead — copy the whole line, and it works from wherever you are.

4.1 Which snippet for which experiment

This is the table to use. About to run experiment 04? Read snippets 01, 02, 04 and 10.

Snippet		Covers	Experiments
01_numbers.py	the numbers	config, the one equation, which mode needs the bias limit raised	all
02_ramps.py	the waveforms	all five trajectories side by side, segments, the shared spike	01–05
03_safety.py	the refusals	all seven, provoked deliberately	all
04_engage.py	approach	break contact, close in, the interlock	all
05_one_trace.py	one pull	volts → conductance → verdict	01
06_histogram.py	the measurement	200 traces, the 1 G ₀ ruler, a deliberate gain error	01
07_storage.py	the file	write, read back, re-calibrate without re-acquiring	all

Snippet		Covers	Experiments
08_push_pull.py	re-forming	segments per cycle, and the push/pull constraint	02
09_iv_sweep.py	spectroscopy	the sweep, the caps, the sign convention	03
10_ac_hold.py	the sine	the hold, and the sampling limit	04
11_high_bias_hold.py	the field	the hold, and the Vzero zero-field control	05
12_keithley.py	the amplifier	GPIB commands, find_suppress, series correction	support
13_echem_cv.py	the third terminal	the gate, the CV triangle, Igor's masking	06
14_lateral_xpiezo.py	the lateral axis	X piezo range, and the withdraw-move-approach order	07
15_bias_series.py	the campaign	control repeats, the refusal, resume after a crash	08
16_real_card.py	the lab PC	the real 4461 — the only untested code	bring-up

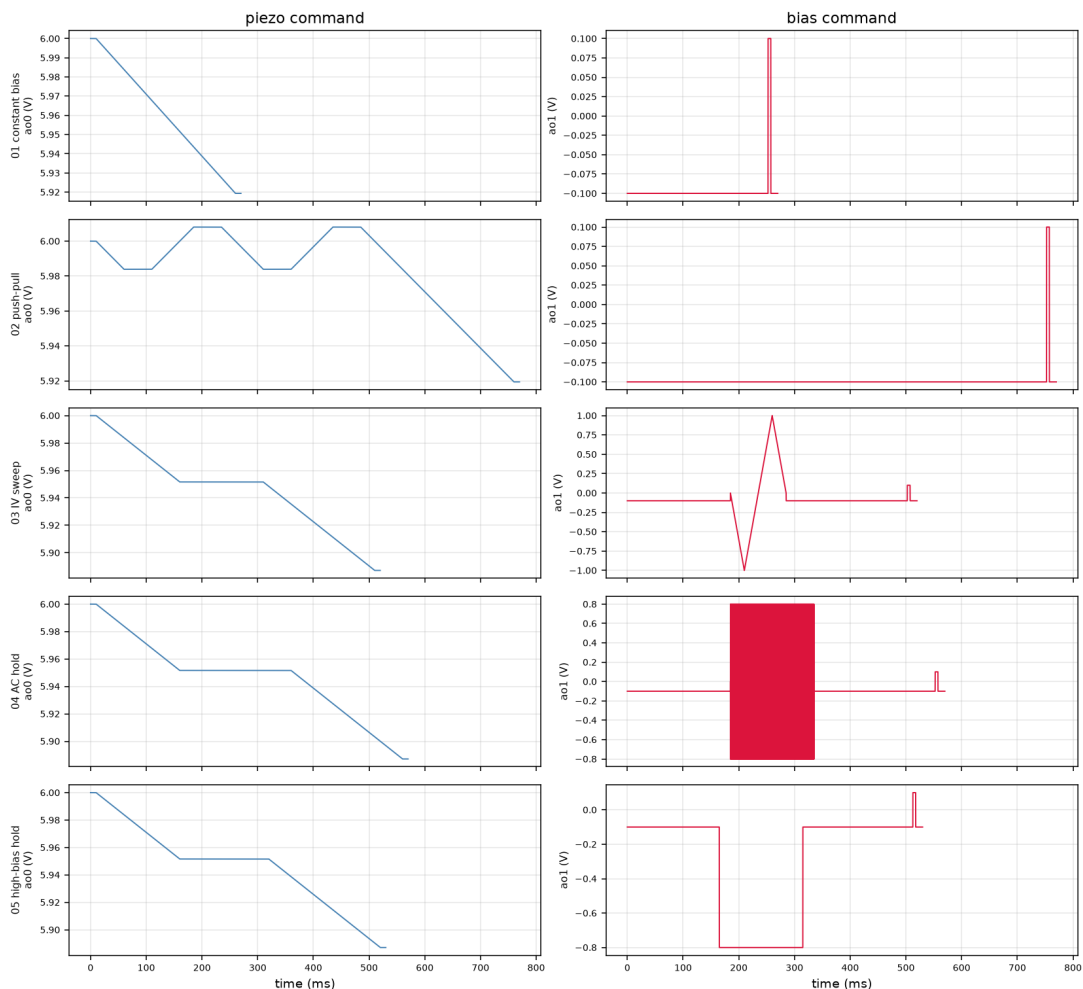
4.2 Suggested order

First time through: 01 → 02 → 03 → 04 → 05 → 06. That is the whole instrument: the numbers, the waveforms, the refusals, getting into contact, one trace, and the measurement. About twenty minutes.

Before running a specific experiment: its row above, plus 01 and 03.

Before touching hardware: 03, then 16 on the lab PC, then the packaged bring-up steps.

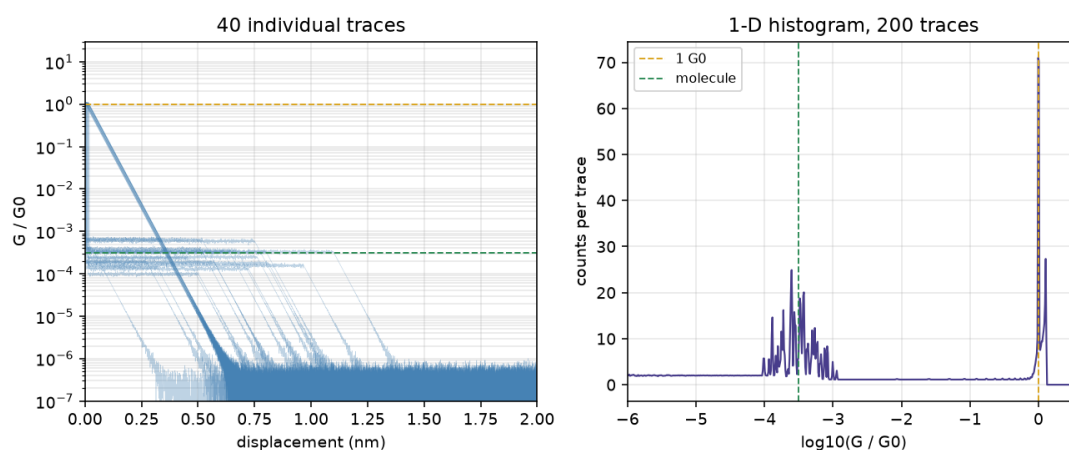
4.3 Four that are worth your time even if you skip the rest



Snippet 02: all five trajectories. Left column is the piezo command at ao0, right column the bias at ao1. Every one is a (2, N) array of volts played through the same `play()`.

02_ramps.py shows the central idea: constant pull, push-pull, IV sweep and both holds are all just different (2, N) arrays of volts, played through the same `play()`, captured by the same code, aligned by the same spike. Once that lands, the whole package is one experiment with five waveforms.

06_histogram.py puts the gold peak at +0.02 decades, then deliberately corrupts the preamp gain by 10 $\times$ and shows the peak move to +1.02 — exactly one decade. That is your calibration self-check, demonstrated.



Snippet 06: 200 simulated traces, and their histogram. The gold peak lands at +0.02 decades because 1 G0 is a constant of nature.

09_iv_sweep.py shows that `ao1` and the junction see **opposite** polarity. Both statements are true and they are opposites. This is the single easiest way to publish a mirrored IV curve.

10_ac_hold.py shows that at 40 kHz sampling and a 10 kHz drive you have **four samples per cycle** — enough for the fundamental, not enough for any harmonic. Read it before you believe a second-harmonic number.

4.4 What the simulator cannot tell you

Every snippet that uses the simulated card says so at the end, and says what that particular simulation does *not* model:

Snippet	The simulator has no...
08 push-pull	memory between breaks — cycle 3 knows nothing about cycle 1
09 IV	non-linearity — its $I(V)$ is a straight line, always
10 AC hold	frequency response — no harmonics, no phase lag
11 HB hold	field-driven degradation — the junction always just sits
13 EChem	chemistry — a capacitor and one Gaussian bump
14 lateral	variation with X — every site is identical

They test the trajectory, the segment indices, the plumbing and the storage. They cannot rehearse a result.

4.5 Figures

Snippets 02, 05, 06, 08, 09, 10 and 13 write PNGs to `snippets/figures/` when `matplotlib` is installed, and skip them cleanly when it is not. Nothing else depends on `matplotlib`, so the lab PC does not need it.

4.6 Things the snippets found

These were not planted. Writing snippets that print real numbers is how they turned up:

- **push-pull never re-formed the junction.** The shipped defaults pulled 3 nm and pushed 1 nm, leaving every hold 2 nm *below* contact. Experiment 02 recorded tunnelling for every cycle and measured nothing. Defaults fixed; `validate()` now warns. See snippet 08.
- **the IV sign flag was inverted.** `negative_first=True` swept *positive* first, and the CLI help and log line both said the opposite of what the card was doing. Renamed to `positive_first`. See snippet 09.
- **the gate survived being switched off.** `CounterElectrode.off()` cleared its own state but not `cfg.echem.gate_mv`, so files written afterwards claimed a gate no longer applied. See snippet 13.
- **nothing connected the amplifier's gain to the arithmetic's gain.** `keithley.gain_exponent` programs the box; `cal.preamp_gain_v_per_a` is what every conductance divides by. `validate()` now checks they agree. See snippet 12.

Constant bias

The reference measurement. Igor: MeasureBreakJunctions, Functions_STMBJ.ipf:1664. Experiment folder: experiments/01_constant_bias/.

This is the only path in this folder that has been validated against Igor’s own archived data, and everything else inherits its correctness. Read this chapter before any of the mode chapters, because they are all variations on its loop.

10.1 What it measures

A gold tip is driven into a gold surface and withdrawn, thousands of times. On each withdrawal the contact thins to a chain of atoms, then to a single atom, then breaks. The single-atom gold contact conducts at almost exactly

$$G_0 = 2e^2/h = 7.7480917346e-5 \text{ S} = 1 / 12,906 \text{ ohm}$$

If molecules are in solution, one sometimes bridges the gap as it opens, holding a plateau three to five decades lower. Once the junction is fully open, current decays about **one decade per angstrom**.

A single trace proves nothing — every junction breaks differently. The measurement is the **histogram** of $\log_{10}(G/G_0)$ over hundreds or thousands of accepted traces. Recurring configurations make peaks; noise does not.

10.2 The loop

```
while accepted < target:
    if attempt % smash_every == 0: smash(rig)      # recondition the tip
    engage(rig)                                   # break contact, close in
    trace = single_trace(rig)                     # play the ramp, capture
    verdict = select_trace(...)                   # accept or reject
    if verdict.accepted: writer.append(trace)
```

Four things happen per attempt, and each has a chapter’s worth of reasoning behind it. The short version:

engage breaks any existing contact *first*, then closes in at 0.5 nm per step. The order is not cosmetic — approaching from an already-shorted junction welds the tip to the sample.

single_trace plays a precomputed (2, N) waveform: the piezo row descends 5 nm over 10,000 samples at 40 kHz, the bias row sits at −100 mV with a 5 ms inverted spike near the end. Python does not time anything; the card does.

The spike is read straight back on ai0. The lag between where it was written and where it appears is the AI/AO group delay, measured **per trace** so it is never assumed. A trace whose delay cannot be recovered, or comes back implausible, is discarded rather than misaligned.

select_trace applies Igor’s TestTrace criteria. This is the most dangerous function in the pack-

age — see below.

10.3 The 1 G0 ruler

The gold peak must land at $\log_{10}(G/G_0) = 0.000$, because 1 G0 is a constant of nature and not a property of your sample. That makes the histogram a **self-check on the entire measurement chain**:

Gold peak lands at	Diagnosis
0.000 ± 0.05	the analysis path is correct
off by exactly ± 1.000	a $10\times$ error in preamp gain
off by ± 0.301	a factor of two in bias — or G0 defined as e^2/h , not $2e^2/h$

`python run.py 01 --summarise <file>` prints these interpretations. Nothing else in this folder gives you a check this strong, which is why the other seven experiments are all judged against constant-bias data taken the same day.

10.4 Two Igor findings you inherit

The engage threshold in Igor was unreachable. `G_ConductanceThreshold = 5 G0` would need 38.7 V at the ADC, but a ± 10 V input at `Rf = 1e6` and 100 mV saturates at 1.29 G0. Igor's approach loop could only ever have terminated with the Keithley set to a lower gain than its own default. Here `engage_g0` defaults to **0.5 G0**, contact is *also* detected by preamp saturation, and `validate()` warns if the threshold is unreachable.

Igor's "started in contact" criterion is switched off. The check exists at `Functions_STMBJ.ipf:1197` but its else branch is commented out, so a trace that fails it falls through to return 1 and is kept. `select_trace( require_engaged=False)` is the default here, reproducing that exactly — because matching archived Igor histograms is the validation milestone, and a translation that silently "fixes" the original cannot be compared to it.

Pass `--require-engaged` for the stricter criterion. If you do: **say so in the paper, and apply it identically to controls.** Selection is where this measurement becomes unfalsifiable if you are careless, which is why all of it lives in one function rather than being scattered through the acquisition code where it would be invisible.

10.5 Running it

```
python run.py 01 --simulate -n 200      # rehearse
python run.py 01 -n 1000 --coarse      # a real session
python run.py 01 --summarise data/run.h5 # where is the gold peak?
```

Ctrl-C once finishes the current attempt and stops cleanly; twice exits at once. Outputs park at 0 V either way.

10.6 What to watch during a run

The acceptance rate and the **rejection breakdown**. A rate that collapses means the tip needs smashing; *which* reason dominates tells you why:

Dominant rejection	Usually means
end $G \geq$ break threshold	the junction is not opening — tip too blunt, or drifted in
only N points between 0.5 and 2.5 G_0	no gold plateau — the contact is not metallic
end of trace is noisy	mechanical or electrical noise; check the isolation
alignment	the spike is not being found — a timing problem, not a physics one

The last one is different in kind from the others: it means the **instrument** is misbehaving, not the junction. If it appears at all, stop and re-run bringup 2.

10.7 See also

- Chapter 01 for `play()` and the capture path
- Chapter 02 for every `RampConfig` field
- Chapter 30 for the numbers
- `../STMBJ_Python_Manual.pdf` — the 45-page deep story of this one experiment

CHAPTER 11

Push-pull

Igor: CreateInputs push-pull branch, Functions_STMBJ.ipf:1461–1495. Experiment folder: experiments/02_push_pull/. Builder: ramps.build_push_pull.

11.1 What it measures

Constant bias breaks a junction once and throws it away. Push-pull **re-forms the same junction**, over and over, without ever fully separating the electrodes.

The trajectory, per trace:

```
initial pull 3 nm
  then, per cycle: hold 1 nm | push IN 1 nm | hold 1 nm | pull OUT 1 nm
  ... repeated `cycles` times ...
final pull 4 nm      -- breaks it for good
```

Bias stays at the DC baseline throughout; only the piezo moves.

The question it answers is **re-binding statistics**. When you pull a molecular junction apart and push it back together, does the same molecule re-bind? Does it bind in the same geometry, giving the same conductance? How many cycles before it is lost?

That matters because the alternative explanations for a conductance plateau — a molecule, a contamination, a gold point contact that happens to sit low — behave completely differently under cycling. A real molecular junction can re-form at the same conductance many times. A stray configuration does not come back.

11.2 Why the ceiling matters here and nowhere else

Every other mode only ever **retracts** from the contact point. Push-pull moves the tip *above* it — that is what “push” means.

On a unipolar 0–10 V piezo, retracting is descending, so the hazard for every other mode is the **floor** at 0 V. Igor guarded only that side, because for four of its five branches that is the only side that exists.

`build_push_pull` refuses at **both** bounds. A push through the ceiling would clip, and a clipped push is a push that did not happen: the trace would show a hold where the code believes there was a re-contact, and the cycle count would be a lie. Igor would have driven it into the rail silently.

This is one of the few places where this translation is deliberately stricter than the original, and the reason is written at the point it happens in `ramps.py`.

11.3 Reading the segments

`ModeRamp.segments` maps names to (start, stop) sample indices *within the pull*. `TraceRecord`'s arrays are already cut to the pull, so the indices line up directly:

```
ramp = ramps.build_push_pull(cfg, rig.piezo_v)
g0 = record.conductance_g0(cfg)
for name, (a, b) in ramp.segments.items():
    print(name, np.mean(g0[a:b]))
```

The segment names are `initial_pull`, `hold_out_<i>`, `push_<i>`, `hold_in_<i>`, `pull_<i>` and `final_pull`. Comparing `hold_in_0` with `hold_in_1` and `hold_in_2` is the experiment: three numbers that should agree if the same molecule re-bound, and will not if it did not.

11.4 Running it

```
python run.py 02 --simulate -n 50
python run.py 02 -n 500 --cycles 5
```

11.5 What the simulator can and cannot tell you

The simulated junction re-rolls its molecule on **every break**, independently. So a simulated push-pull run shows re-binding at the right *rate* but with no memory: cycle 3 knows nothing about cycle 1.

Real re-binding is correlated — that correlation is the physics, and the simulator does not have it. Use `simulate` to check that the trajectory is right and the segments land where you think. Do not use it to predict what your molecule will do.

11.6 Status

The trajectory is an index-for-index translation of Igor's branch with geometry tests. It has run end to end on the simulator. It has never pushed a real tip back into a real surface.

The specific risk on hardware is the one the ceiling check exists for: if `start_piezo_v` is high and `push_pull_nm` is large, the push has nowhere to go, and you will find out from a refusal rather than a crash — provided the piezo calibration is right. If `piezo_nm_per_volt` is wrong, the refusal fires at the wrong place. Verify 62 nm/V before running this mode in anger.

CHAPTER 12

IV sweep

Igor: CreateInputs IV branch, Functions_STMBJ.ipf:1497–1537. Experiment folder: experiments/03_iv_sweep/. Builder: ramps.build_iv. Analysis: experiments/03_iv_sweep/iv_analysis.py.

12.1 What it measures

Everything else in this folder measures conductance at **one** bias. An IV sweep holds a junction still and sweeps the bias in a triangle, recording current the whole way. One curve per junction.

initial pull 3 nm	at the DC baseline
cap 0.5 nm	baseline, lets the junction settle
IV ramp 2 nm worth	0 → +max → -max → 0, up to 1 V
cap 0.5 nm	baseline again
final pull 4 nm	breaks the junction

The shape of $I(V)$ is the physics:

- **Linear** through the origin — off-resonant tunnelling, the junction is a simple resistor over this range. The slope is the conductance you would have measured at constant bias.
- **Sigmoidal / S-shaped** — a molecular level is approaching resonance. The bias at which it turns over locates that level relative to the Fermi energy, which is a number a constant-bias histogram cannot give you at all.
- **Asymmetric** — the molecule couples more strongly to one electrode than the other. Real, common, and interesting.
- **Steps or spikes** — the junction changed *during* the sweep. Discard; it is not one junction's IV curve.

12.2 The caps are not padding

The 0.5 nm caps either side of the sweep hold the junction at the baseline bias, still, before and after the ramp. They exist so you can separate "what the bias sweep did" from "what holding the junction did".

If the conductance in the leading cap differs from the trailing cap, the junction changed over the course of the sweep — and the IV curve is a composite of two different objects, not a measurement of one. The analysis checks this.

12.3 Igor's sign convention, faithfully reproduced

Igor built the IV ramp **positive** and then negated the entire wave (*= -1, line 1533). Combined with the baseline already being $-(\text{TipBias}/1000)$, this means the sweep runs **negative first** by default.

That wholesale negation is what makes the **default sweep drive ao1 negative first**. `G_IVSign` Flag (line 1535) negates the ramp segment a second time, sending ao1 positive first.

Be careful which side of the sign convention you are talking about. `Calibration.voltage_input_sign` is also -1 , so the *junction* sees the opposite of what ao1 does:

	first quarter of the sweep
written to ao1	negative
seen by the junction	positive

Both statements are true and they are opposites. `iv_analysis.py` reports `v_junction`, so its first quarter runs $0 \rightarrow +1.0$ V with the default settings. If you compare a Python IV curve against an Igor one and they look mirrored, this is the first place to look — before suspecting the molecule.

This field was renamed during this work. It arrived called `negative_first`, which was backwards: setting it True made the sweep go *positive* first, and the CLI help and the run's log line both said the opposite of what the card was doing. It is now `IVConfig.positive_first` / `--positive-first`, named for what it does. If you have a config JSON written before this change, the old key is ignored with a warning — check it.

There are tests for both directions, because a sign error here does not crash. It silently mirrors every IV curve you take, and an asymmetric junction measured backwards looks like a different molecule.

12.4 Running it

```
python run.py 03 --simulate -n 10 --plot
python run.py 03 -n 200 --max-bias 1.0
```

`iv_analysis.py` extracts, per curve: the zero-bias conductance from a linear fit near the origin, the cap-to-cap conductance drift, and a symmetry ratio.

12.5 The bias limit

`IVConfig.max_bias_v` defaults to 1.0 V, which is above `SafetyLimits.bias_max_v` (0.5 V). The experiment raises the limit only when you ask it to, the same way experiment 08 does — see chapter 17 for why that obstacle exists.

12.6 Status

Translated index-for-index with geometry tests; runs end to end on the simulator. The simulated junction has a **linear** $I(V)$, so simulated curves will always fit a straight line. That is enough to test the sweep geometry, the caps, the segment indices and the analysis plumbing, and tells you nothing about what a real molecule will do.

CHAPTER 13

AC hold

Igor: CreateInputs AC-hold branch, Functions_STMBJ.ipf:1539–1564. Experiment folder: experiments/04_ac_hold/. Builder: ramps.build_ac_hold.

13.1 What it measures

Hold the junction still and make the bias a **pure sine** — 0.8 V amplitude at 10 kHz by default — instead of a DC level.

initial pull 3 nm	DC baseline
cap 0.5 nm	DC baseline
hold 3 nm worth	bias = amp * sin(2*pi*f*t), replacing the baseline entirely
cap 0.5 nm	DC baseline
final pull 4 nm	

The sine **replaces** the DC baseline during the hold; it is not added to it. That is Igor's behaviour and it matters for interpretation: the junction spends the hold symmetrically biased about zero, not about -100 mV.

13.2 Why drive a junction with a sine

Three reasons, in increasing order of ambition:

Noise rejection. The interesting response is concentrated at one frequency. Everything else — $1/f$ drift, mains pickup, the slow wander of the preamp zero — is somewhere else in the spectrum and can be filtered away. This is the lock-in principle, and it is why a 10 kHz sine can measure a conductance that a DC hold cannot resolve.

Harmonics. A linear (ohmic) junction driven at frequency f responds only at f . A **non-linear** one also responds at $2*f$, $3*f$, ... The amplitude of the second harmonic is a direct measure of the curvature of $I(V)$ at zero bias — the same physics chapter 12 gets from a full sweep, but at one frequency and in a fraction of the time.

Speed. An IV sweep takes the whole hold. A sine gives you a conductance estimate every cycle: at 10 kHz that is $100\ \mu\text{s}$, fast enough to watch a junction switch between states in real time.

13.3 The one number to check before trusting anything

The sample rate must be far above the drive frequency. At 40 kHz sampling and a 10 kHz sine you have **four samples per cycle**. That is above Nyquist, so the fundamental is recoverable in principle — but it is nowhere near enough to see the second harmonic at 20 kHz, which sits exactly *at* Nyquist and is indistinguishable from DC aliasing.

So with the shipped defaults:

- the fundamental amplitude is measurable;

- **harmonic analysis is not possible**, and any second-harmonic number you compute from this data is an artefact.

If harmonics are what you are after, either drop `freq_khz` to 1–2 kHz (20–40 samples per cycle) or raise `sample_rate_hz`. The 4461 will go to 204.8 kHz; check what it actually grants, because a delta-sigma card coerces (chapter 03).

This is not a defect in the translation — Igor’s defaults have the same property. It is a limit of the settings, and it is worth knowing before you believe a number.

13.4 Running it

```
python run.py 04 --simulate -n 20
python run.py 04 -n 200 --amp 0.5 --freq-khz 2
```

13.5 Status

Translated index-for-index with geometry tests; the sine’s amplitude, frequency and placement inside the hold are all asserted. It runs end to end on the simulator.

The **simulated junction has no frequency response at all** — its conductance depends on position, not on how fast the bias is changing. So a simulated AC hold returns a clean sine in current, exactly in phase, with no harmonics and no phase lag. That tests the waveform, the segments and the storage. It cannot tell you anything about a real junction’s dynamics, and it will never warn you about the sampling limit above.

CHAPTER 14

High-bias hold

Igor: `CreateInputs HB-hold branch, Functions_STMBJ.ipf:1566–1599.` Experiment folder: `experiments/05_high_bias_hold/`. Builder: `ramps.build_hb_hold`.

14.1 What it measures

Form a junction, retract a fixed distance to select a geometry, then **park it under a large DC bias** and watch.

<code>initial pull 3 nm</code>	<code>at the 100 mV baseline</code>
<code>cap 0.1 nm</code>	<code>baseline</code>
<code>hold 3 nm worth</code>	<code>bias = hold_bias_v (0.8 V default)</code>
<code>cap 0.1 nm</code>	<code>baseline</code>
<code>final pull 4 nm</code>	

A molecular junction that sits quietly at 100 mV can be a very different object at 0.8 V. Across a sub-nanometre gap that is a field approaching 10^9 V/m — enough to drive electromigration of the electrode atoms, tilt molecular levels toward resonance, heat the junction through inelastic current, and break weak bonds.

The current record during the hold shows whether the junction **sits**, **telegraphs** between two states, **drifts**, or **dies**. Those four behaviours are the result.

The short caps at the baseline on either side separate *what the field did* from *what stepping the bias did* — the same logic as the IV caps in chapter 12.

14.2 The Vzero link — this is the interesting part

The complement of a high-field hold is a **zero-field** hold. And zero applied bias is *not* zero field.

The amplifier chain has an input offset, so a small current flows at a commanded 0 V. The bias that actually nulls the current is **Vzero**, and it is not zero — it is a few millivolts, and it drifts.

Igor's Voltage_Offset panel measured it by sweeping ± 5 mV in contact and fitting $I(V)$ (Offset Voltage, Functions_STMBJ.ipf:920). With its VzeroCheckBox ticked, CreateInputs **replaced the hold bias with the measured Vzero** (lines 1595–1599).

That turns this experiment inside out. The "high-bias hold" becomes a true **zero-current hold**: the control that separates field-driven changes from the junction's own thermal drift. If a junction is unstable at 0.8 V *and* equally unstable at Vzero, the field is not what is destroying it.

--use-vzero reproduces this: after the first engage, `vzero.measure_offset` runs once, the result is logged, and every trace of the session holds at that offset. See chapter 21.

Expect the printed hold conductances to be **noisy** in this mode. The denominator is the near-zero measured junction voltage, and dividing by a number close to zero does what you would expect. The current record is the meaningful thing; the conductance column is

not.

14.3 Running it

```
python run.py 05 --simulate -n 20
python run.py 05 -n 200 --hold-bias 0.8
python run.py 05 -n 200 --use-vzero      # the control experiment
```

14.4 The bias limit

0.8 V exceeds `SafetyLimits.bias_max_v` (0.5 V). The experiment raises it deliberately and says so, the same pattern as chapters 12 and 17. If you want a lower hold, `--hold-bias 0.4` needs no override at all.

14.5 Status

Translated index-for-index with geometry tests, including the Vzero substitution. Runs end to end on the simulator.

The **simulated junction does not degrade under field** — it has a tip that blunts with hard contact, and nothing that responds to bias. So a simulated high-bias hold shows a flat, stable junction every time. The four behaviours this experiment exists to distinguish are all absent from the simulation by construction. Simulate to check the trajectory; you cannot rehearse the result.

EChem gate and cyclic voltammetry

Igor: *EChem_Module.ipf*. Experiment folder: *experiments/06_echem_gate_cv/*. Module: *stmlab/echem.py*.

15.1 The third terminal

A dry break junction has two terminals. An electrochemical one has three.

Tip and substrate sit in electrolyte; a **counter electrode** holds the solution at a potential you choose. Because the molecule is immersed in that solution, changing the solution potential shifts the molecule's energy levels relative to the Fermi level of the electrodes — **without touching the source–drain bias**.

That is a gate, in exactly the transistor sense. And it buys you something no two-terminal measurement can give: as a molecular level is dragged through resonance with the electrodes, transmission rises and then falls, producing a **conductance peak versus gate voltage**. The gate potential at that peak locates the level.

15.2 Three subcommands, because Igor's module held three things

Command	Igor	Does
gate	WriteToCounterElectrode:88	set the DC gate
cv	CVcurveHighRes / LowRes:106 / :234	a voltammogram
traces	the gate + StartMeasurement	the actual experiment

`t traces` is the point. The other two are the instrument controls it is built from, exposed separately because you will want to set a gate and walk away, or take a voltammogram to check the cell, without acquiring anything.

15.3 Two things that make this harder than it looks

The gate is slow. After changing the counter-electrode potential the electrical double layer takes time to re-equilibrate. Traces acquired during that are at an unknown gate. Hence `--settle`, defaulting to **5 s** rather than 0. If your electrolyte is viscous or your electrode large, 5 s may not be enough — that is an experimental question, not a software one.

The zero moves with the gate. The cell leaks: faradaic and capacitive currents through the electrolyte are gate-dependent. The preamp reading with no junction at -200 mV is *not* the reading at $+200$ mV.

So this experiment re-measures `current_zero_v` **at every gate**, not once per session. Skipping that puts a false floor under one gate's histogram and not another's — which is indistinguishable

from gate-dependent physics, and is the most convincing artefact this experiment can produce. It would show up as a beautiful, entirely fictional conductance peak.

15.4 Cyclic voltammetry

Sweep the electrode potential in a triangle at fixed scan rate, record the current. You get **capacitive current** (proportional to scan rate, flipping sign with direction — the rectangular box of the loop) and **faradaic peaks** where species are oxidised or reduced, ~60 mV apart for a reversible one-electron couple.

Run it to answer three questions before trusting any gated data: is the cell connected, is the electrolyte clean, and where are the redox peaks — so you know which gate range is worth scanning and which will just electrolyse your solvent.

15.4.1 Igor's masking, kept as a mask

Igor built $0 \rightarrow V1 \rightarrow 0 \rightarrow V2 \rightarrow 0$, then repeated the first half and NaN-ed the opening quarter and the repeat, so each saved cycle is a steady-state loop with the switch-on transient discarded.

This translation reproduces the same choice as a **boolean keep array stored beside the full record**, rather than destroying samples. A later reader can second-guess where the transient ended; with NaNs they could not. Same default behaviour, recoverable.

15.5 Which card

The counter electrode is on the **low-res card's ao1** (Setup1_STMBJ.ipf:11). This rig has one card, so:

	Needs the second card?
cv --kind highres	no — drives dev1/ao1, the junction-bias output
cv --kind lowres	yes
gate, traces	yes

gate and traces refuse with a clear message when `channels.low_res_device` is None.

cv --kind highres repurposes the junction-bias output. Retract before running it. The code warns; it cannot check.

15.6 The gate is left on, deliberately

gate --mv X without --hold releases the DAQmx task **without zeroing the channel**, so the card keeps driving it after the process exits. That is what Igor's always-open task looked like to an operator.

It also means a voltage stays on an electrode with no process watching it, so the command says so loudly. --mv 0 clears it. --hold N returns to 0 after N seconds. traces always returns the gate to 0 mV on exit, including on exception and Ctrl-C.

A bug found while writing this chapter: `off()` cleared the object's gate but not `cfg.echem.gate_mv`, so the config — which is written into every data file — went on claiming a gate that was no longer applied. Fixed, with a test.

15.7 Status

`echem.py` is a faithful translation and every line runs in `simulate`. **None of it has met an electrochemical cell.** The `nidaqmx` calls for the gate and both CV variants are exactly where `daq.py` was before bring-up: written, reviewed, never executed.

The simulated cell is a $1\ \mu\text{F}$ double layer plus one reversible couple near $+0.2\ \text{V}$. It produces the right *shape* to exercise the code and contains no chemistry whatsoever.

Simulate. Then a dummy cell with a known resistor. Then electrolyte.

Lateral monolayer mapping

Igor: *LateralEXPT, Setup1_STMBJ.ipf:91–136. Experiment folder: experiments/07_lateral_monolayer/. Module: stmLab/xpiezo.py.*

16.1 What it measures

A histogram from one spot on the sample tells you about **that spot**. If the monolayer is patchy, or the coverage varies, or there is a defect under the tip, you will never know from the histogram alone — a clean single peak is perfectly compatible with having measured one unrepresentative site a thousand times.

This experiment walks the tip laterally across the sample and takes a **fresh batch of traces at each site**. Comparing histograms site to site tells you whether the molecular peak is a property of the monolayer or a property of the place you happened to land.

16.2 The sequence, and why the order is not negotiable

Per site, Igor did:

StartMeasurement("")	a batch of constant-bias traces
StepActuatorApart("") x3	withdraw, 100 ms between steps
MoveXPiezo(XDistance)	the lateral move
G_ActuatorStepSize = 5	
SetPiezoBiasFromSlider(...)	reassert the bias
ApproachButton("")	coarse approach back into contact

The X piezo moves only while the tip is withdrawn — and withdrawn by three *coarse actuator* steps, not merely a fine-piezo retract.

A lateral move in contact drags the tip through the monolayer. That does three bad things at once: it destroys the film along the path, it reshapes the tip apex, and it does both *silently* — the next site's traces look fine and are measuring a different tip on damaged sample.

Three steps is the specific number Igor used and it is a good one: far enough that a sample tilt across a 200 nm lateral move cannot crash the tip, close enough that the coarse re-approach takes seconds rather than minutes.

16.3 The X piezo

522 nm/V ($K_{XPiezoScale}$), on the low-res card's *ao0*. So a 200 nm step is 0.383 V, and the full 0–10 V range is **5.22 μm** of travel — about 26 sites at 200 nm spacing before you run out.

xpiezo.XPiezo tracks its position and **refuses** out-of-range moves. Igor did not check; it would simply have clipped, and every site past the clip point would have been the *same* site while the file names went on incrementing.

16.4 Two cards

The X piezo is on the second card. Without `channels.low_res_device` this experiment cannot run on hardware. `--simulate` substitutes `SimulatedXPiezo`, which tracks position and enforces the same range.

16.5 Running it

```
python run.py 07 --simulate --sites 5 -n 20
python run.py 07 --sites 20 -n 200 --step-nm 200
```

One `site_<i>i>.h5` per site, each carrying `site_index`, `x_position_nm`, `x_piezo_v` and `x_step_nm` as file attributes — so a site's absolute position survives independently of its filename.

16.6 Reading the result

Compare the **peak position** and the **peak area per trace** across sites.

- Peak position stable, area stable → uniform coverage. Pool the sites.
- Position stable, area varies → patchy coverage; the molecule is the same where it is present. Still poolable, but report the variation.
- **Position varies** → the sites are not measuring the same thing. Do not pool. Something is wrong with the sample, or with the tip, and the site-to-site comparison has just saved you from publishing an average of two different molecules.

That third case is the reason to run this experiment at all.

16.7 Status

The site loop, the withdraw/move/re-approach ordering, and the X piezo range refusal are exercised end to end on the simulator and covered by tests. The hardware path needs a second card that this rig does not currently have.

The simulated surface does not vary with X — every site is identical by construction — so simulate tests the choreography and nothing about coverage.

CHAPTER 17

Bias series

Igor: rungo, Setup1_STMBJ.ipf:138–178. Experiment folder: experiments/08_bias_series/.

17.1 What it measures

A histogram at one bias locates the conductance peaks. A histogram at a *series* of biases shows how they **move** — and that is where the physics is.

For a molecule in the off-resonant tunnelling regime, conductance is roughly bias-independent at low bias. When it stops being independent, something happened:

- a molecular level is being pulled toward resonance by the applied field, so transmission rises faster than linearly — the onset of **resonant tunnelling**;
- the junction is heating and the molecule is desorbing or reorienting;
- at high enough field the molecule breaks, or the gold electrodes electromigrate, and you are no longer measuring the same object.

Telling those apart is what the series is for.

17.2 The control repeats are the point

Igor's list, built at lines 147–155 and then overridden by hand:

step:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
mV:	-100	-200	-300	-400	-500	-600	-700	-800	-900	-100	-900	-1000	-1100	-100
										^^^		^^^		^^^

–100 mV appears at steps **0, 9 and 13**; –900 mV at steps **8 and 10**. Those repeats were put there deliberately.

Fourteen thousand traces takes many hours. Over that time the tip blunts, the sample drifts, the solvent evaporates, the lab warms up. **If the –100 mV histogram from step 13 does not match the one from step 0, the series is not measuring bias dependence — it is measuring the passage of time**, and every intermediate point is suspect.

So analyse the controls first. If they agree, you have a bias series. If they do not, you have an equipment log. The run prints which steps are repeats when it finishes, so the comparison is hard to forget.

This is the single most important idea in this chapter, and it is worth copying into any campaign you design yourself: **a long unattended run needs a repeated point, and it needs to be at the end.**

17.3 The bias limit is a deliberate obstacle

Igor's series reaches **1100 mV**. `SafetyLimits.bias_max_v` defaults to 500 mV, so this experiment **refuses to start**:

```
ERROR this series reaches -1100, -1000, -900, -800, -700, -600 mV
      but limits.bias_max_v is 500 mV.
      Igor's rungo applied these with no guard. If you mean to,
      pass --max-bias 1.10 (volts).
      At 1.1 V a single-molecule junction can break, the electrodes
      can electromigrate, and any solvent present will do chemistry.
```

Two design points:

It is checked once, up front, for the whole list — not per step. Discovering at step 11 of 14 that the limit blocks 1100 mV would mean eleven hours of acquisition followed by a crash, and a series with a hole in it.

The message says exactly what to type. A refusal that does not tell you how to proceed is an obstacle; one that does is a checkpoint. Raising the limit should be something you did on purpose, in a command you can find again in your shell history. Igor had no guard at all.

17.4 Resume

Igor resumed with `FindLevel/EDGE=1 /P StartNumWave, SavedNumber` (line 163): look up how many traces are saved, find which segment that falls in, start there. Runs died; this mattered.

`--resume` here reads the output directory:

- a step whose file already holds its full complement is **skipped**;
- an incomplete one is **re-run from scratch**, overwriting the partial file.

The second is the deliberate part. Appending post-crash traces onto pre-crash ones would put two sessions — two preamp zeros, two tip states, possibly two days — into one file carrying **one** calibration. Losing a partial step is cheaper than a file that lies about what it contains.

A `series_index.json` manifest is written beside the data, so the campaign's shape survives someone later moving or renaming the .h5 files.

17.5 Per-step calibration

The preamp zero drifts with temperature and fourteen hours is long enough to matter, so `measure_zero` runs at **every step** and the result goes into that step's file. `--no-calibrate` skips it; then the low-conductance tail of the later steps sits on an offset measured when the lab was a different temperature.

17.6 File naming

`step_<NN>_<bias>mV.h5`. **The index is in the name because the same bias appears more than once and the order is the experiment.** Step 00 at -100 mV and step 13 at -100 mV are the control pair; a filename that collided them would destroy the comparison the series exists to make.

17.7 Running it

```
python run.py 08 --dry-run --max-bias 1.2      # see the plan, touch nothing
python run.py 08 --max-bias 1.2                # Igor's full series
python run.py 08 --max-bias 1.2 --resume -o data/series_20260828_193000
python run.py 08 --bias-mv -100 -200 -300 -n 500 # your own, gentler
python run.py 08 --simulate --max-bias 1.2 -n 20 # rehearse
```

Ctrl-C once finishes the current step; re-run with `--resume`. Twice exits at once.

17.8 Status

The loop, the plan, the refusal and the resume are exercised end to end on the simulator and covered by tests, including a deliberately corrupted file (which must read as incomplete) and a resume that must not touch already-complete steps.

What has never been done is **fourteen real hours on a real tip** — and that, not the code, is where this experiment is hard. Run it simulated once to see the shape, then four steps at low bias, before you commit a night to it.

CHAPTER 20

Keithley 428

*Igor: SetUpGPIB_Keithley.ipf, plus SetGain and SetCurrentSuppress in Controls_STMBJ.ipf.
Module: stmLab/keithley.py.*

20.1 What it is

The Keithley 428 is the **current preamplifier** — the box that turns the picoamps-to-microamps flowing through your junction into volts the DAQ card can read. Everything in this folder that says “conductance” ultimately says “the 428’s output voltage, divided by its gain”.

It sits on GPIB at address 22 (Igor’s `ibdev={0,22,...}`), and speaks a terse command language where X executes the buffer.

Command	Means	Igor
C1X / C0X	zero-check on / off	ZeroCheckProc :262
C2X	zero-correct	ZeroCorrectProc :277
H6R<g>X	gain = 10^g V/A	SetGain :290
H8S<amps>, 0X	suppress value	SetCurrentSuppress
N1X / N0X	suppress on / off	
B1X / B0X	internal bias source on / off	KeithleyBias :67
V<mV>E-3X	bias value	KeithleyBiasVolt :95
P1X / P0X, T<n>X	filter on/off, rise time	KeithleyFilter :37

Igor’s init string is C1P0B0N0X: zero-check on, filter off, bias off, suppress off. **The amplifier wakes up inert**, which is the right default for a box wired to a tip.

20.2 Gain is the number that scales every result

`gain_exponent = 6` means 10^6 V/A. That single number multiplies every conductance in every file. Get it wrong by a factor of ten and your gold peak lands exactly one decade off — which chapter 10 will tell you, but only if you look.

Two derived values Igor kept as globals live here as properties:

```
conversion_ua_per_v = 10**(6 - gain)      G_CurrentVoltConversion
suppress_from_const = const * 10**(3 - gain)  G_CurrentSuppress
```

20.3 Current suppress, and find_suppress

The amplifier has an offset. At zero junction current its output is not zero. **Suppress** is a compensating current you dial in to null that.

`find_suppress` is Igor’s `TestVirtualGround (Functions_STMBJ.ipf:1097)`:

1. sweep the suppress from -1 to $+1 \mu\text{A}$ in 21 steps;
2. after each step, wait for the amplifier to settle — Igor's `sleep/T 5`, with a comment about "the SPIKE problem" at line 1123;
3. read 2000 samples of the output;
4. fit a line through the **middle** points and set the suppress to the zero crossing.

The endpoints are excluded from the fit deliberately: the amplifier is still settling at the extremes of the sweep, and including them tilts the line.

Igor's FindSuppress button zeroed the applied bias first and restored it afterwards (`Controls_STMBJ.ipf:479`). **Do the same around this call** — the function itself does not, because it cannot know what bias you wanted back.

20.4 When the Keithley sources the bias

B1X turns on the 428's internal bias source. When that is driving the junction, the junction voltage is **no longer measured at ai0** — you know only the total two-terminal current, and the wiring's series resistance is in the path.

So the conductance needs a correction, and Igor had it at `GenerateTrace:390`:

```
G/G0 = (1 / (V_bias/I - R_series)) / G0
```

That is `series_corrected_conductance()`. `R_series` defaults to **106,130 Ω** — Igor's `G_Series Resistance`, an inherited number for that rig's wiring, not a measurement of yours.

If you use Keithley-sourced bias, **measure your own series resistance**. 106 k Ω is comparable to the resistance of a 0.1 G_0 junction, so a wrong value does not scale your result — it distorts the shape of the whole low-conductance region, and it does it in a way that looks like physics.

20.5 Simulated

`SimulatedKeithley` has the same surface and records state, so every experiment script runs on a laptop. It does not model the amplifier's noise, bandwidth, or settling.

20.6 Status

Never connected to a Keithley. The command strings are transcribed from the Igor source and reviewed; `pyvisa` is not even in `requirements.txt` by default. Before trusting it:

1. talk to the amplifier with `pyvisa` by hand and confirm it answers;
2. set a gain and verify it on the front panel;
3. run `find_suppress` and check the resulting output offset with a meter;
4. only then wire it to a tip.

Chapter 03 covers what needs what.

Vzero — the zero-current offset

Igor: OffsetVoltage / SaveOffset, Functions_STMBJ.ipf:920 / :993. Module: stmlab/vzero.py.

21.1 The problem

Zero applied bias is not zero current.

An amplifier chain has an input offset. Command exactly 0 V and a small current still flows through the junction. That offset is a few millivolts of equivalent bias, and it **drifts** — with temperature, with the electrolyte, with the age of the tip.

For a constant-bias run at 100 mV this is a small error. For anything that cares about the zero — a low-bias measurement, a rectification study, the zero-field control in chapter 14 — it is the whole experiment.

21.2 The measurement

measure_offset is Igor's OffsetVoltage:

1. hold the tip **in contact** (`_ensure_contact`, Igor's `HighResContact`);
2. step the applied bias through $\pm \text{interval_mv}$ (± 5 mV) in $2 * n_points$ steps — Igor's ten;
3. re-check contact before **every** point, settle 50 ms after each bias change, read 5000 samples;
4. fit a straight line to current against bias.

From that line:

<code>Izero</code> = current at zero applied bias	(the intercept)
<code>Vzero</code> = $-Izero / \text{slope}$	(the bias that nulls the current)

Igor fed Vzero back into the applied bias as $-(\text{TipBias} + Vzero)/1000$ (`CreateInputs`, line 1611).

Why in contact? The fit needs a measurable current at ± 5 mV. Out of contact, 5 mV across a tunnelling gap gives nothing to fit, and the slope comes out as noise. In contact the junction is a few k Ω and the line is clean. This is also why the routine restores your original bias afterwards (Igor line 980) — it has been driving the junction at a bias you did not ask for.

21.3 The history is the diagnostic

VzeroTracker is Igor's `SaveOffset`: re-measure every `every_n_traces = 50` accepted traces and **keep every result**.

The history matters more than any single value. A Vzero that drifts steadily over a session is an early warning that the amplifier is drifting, the junction chemistry is changing, or the temperature is not settled. Igor plotted it — the `Izero` graph — for exactly that reason.

```

tracker = vzero.VzeroTracker(cfg)
# inside the trace loop:
tracker.maybe_measure(rig, trace_number)
bias_offset = tracker.offset_v      # volts, add to the applied bias

```

`tracker.as_arrays()` returns flat `vzero_mv`, `izero_a` and `timestamp` arrays, ready to write into an HDF5 file beside the traces. **Do write them.** A session whose Vzero moved 3 mV is a session you want to know about six months later, and the traces alone will not tell you.

21.4 The cost

Each measurement is 10 bias points $\times$ (contact check + 50 ms settle + 5000 samples). At 40 kHz that is well over a second of acquisition plus the settling, and it happens **in contact** — so the junction you were measuring is gone afterwards and has to be re-formed.

`every_n_traces = 50` is Igor’s compromise: often enough to catch drift, rare enough not to dominate the run. Setting it to 1 would give you a beautifully characterised offset and almost no data.

21.5 Where it is used

Chapter	Use
14, high-bias hold	--use-vzero makes the hold bias <i>be</i> Vzero — the zero-field control experiment
any long run	periodic tracking, as a drift diagnostic
low-bias work	feed <code>offset_v</code> into the applied bias

21.6 Status

Exercised end to end on the simulator and covered by tests, including the tracker’s schedule. The simulated junction has **no amplifier offset**, so a simulated Vzero comes out at essentially zero every time — which tests the fit, the contact handling and the bookkeeping, and tells you nothing about what your amplifier actually does.

This is one of the routines most worth running early on real hardware, because its answer is a single number you can sanity-check against the preamp zero from `bringup 4`. If `measure_zero` says the preamp sits at 300 μV and Vzero says the nulling bias is 30 mV, one of the two is wrong.

CHAPTER 30

Quick reference

Keep this one open while working.

30.1 Every command

```
python run.py list                # what can I run?

# --- experiments; add --simulate to any of them -----
python run.py 01 -n 1000 --coarse    # constant bias (the reference)
python run.py 02 -n 500 --cycles 5   # push-pull
python run.py 03 -n 200 --max-bias 1.0 # IV sweep
python run.py 04 -n 200 --amp 0.5 --freq-khz 2 # AC hold
python run.py 05 -n 200 --use-vzero   # high-bias hold / zero-field control
python run.py 06 cv --kind highres --plot # cyclic voltammetry
python run.py 06 gate --mv -200        # set the gate (LEFT ON)
python run.py 06 traces --gate-mv -200 0 200 -n 200
python run.py 07 --sites 20 -n 200    # lateral map
python run.py 08 --max-bias 1.2 --resume # Igor's bias campaign

# --- diagnostics -----
python -m stmlab.main check          # validate a config, no hardware
python -m stmlab.main dump-config my.json # edit the defaults
python -m stmlab.main summarise data/x.h5 # where is the gold peak?
python -m stmlab.main bringup 1      # ... through 5
python -m pytest tests/ -q           # 93 tests
```

python run.py <NN|name> ... and python experiments/<NN>_<name>/run_experiment.py ... are always equivalent.

30.2 Every number

30.2.1 Physics

$G_0 = 2e^2/h$	$7.7480917346e-5 \text{ S} = 1 / 12,906 \Omega$
1 G_0 at 100 mV	$7.748 \mu\text{A} \rightarrow 7.748 \text{ V}$ at ai1 ($R_f = 10^6$)
input saturates at	1.23 G_0 — hard contact reads as a <i>railed channel</i>
tunnelling decay	~1 decade per ångström
gold peak must be at	0.000 $\pm$ 0.05 decades
off by ± 1.000 decade	10 $\times$ preamp gain error
off by ± 0.301 decade	factor of 2 in bias, or G_0 as e^2/h

30.2.2 Piezos

Z piezo	62 nm/V, 0–10 V unipolar , 620 nm total
Z: a 5 nm pull	80.6 mV
Z: a 0.5 nm step	8.06 mV
X piezo	522 nm/V, 0–10 V, 5.22 μm total

X: a 200 nm step coarse interlock	383 mV piezo must be below 0.1 V for a coarse step
--------------------------------------	--

30.2.3 Sampling

sample rate	40 kHz requested — use what the card grants
a 5 nm pull	250 ms, 10,000 points
whole record	10,800 (400 pre-pad + 10,000 + 400 post-pad)
alignment spike	5 ms, 7.5→2.5 ms before the end of the ramp
AI/AO group delay	tens of samples; measured per trace , never assumed

30.2.4 Biases, and which need the limit raised

SafetyLimits.bias_max_v defaults to **0.5 V**.

Mode	Default bias	Needs override?
constant bias	100 mV	no
push-pull	100 mV	no
lateral map	100 mV	no
IV sweep	±1.0 V	yes — the experiment raises it and says so
AC hold	0.8 V amplitude	yes
high-bias hold	0.8 V	yes
bias series	up to 1.1 V	yes — --max-bias 1.2, refuses without

30.2.5 Support systems

Keithley gain	10 ⁶ V/A (gain_exponent = 6), GPIB address 22
Keithley series R	106,130 Ω — inherited from Igor, measure your own
suppress sweep	−1 to +1 μA, 21 points, endpoints excluded from the fit
Vzero sweep	±5 mV, 10 points, in contact , re-measured every 50 traces
CV	1 kHz sampling, 100 mV/s scan, peaks at ±1.0 V
counter electrode	±5 V, low-res card ao1, entered in mV

30.3 Where to look when something is wrong

Symptom	Look at
gold peak one decade off	preamp gain — bringup 4, chapter 20
gold peak 0.301 off	bias, or a G ₀ definition — chapter 10
alignment rejections	the timing, not the junction — bringup 2
approach never terminates	engage threshold vs saturation — chapter 10
approach walks in place	MAINTAIN_EXISTING_VALUE refused — bringup 3
"refusing pull: no headroom"	contact made too low in the unipolar range
"refusing coarse step"	the fine piezo is extended — this is the interlock working
histogram floor rises at one gate	per-gate zero not measured — chapter 15
control steps disagree	the series measured time, not bias — chapter 17
IV curves all mirrored	the sign flag — chapter 12

30.4 The trust order

1. **Validated.** The constant-bias path, against Igor's archived data.
2. **Tested end to end on the simulator.** The four ramp modes, both campaigns, CV ramp construction, Vzero, the X piezo range logic. 93 tests.
3. **Never met its instrument.** Every nidaqmx call in the new modules, the Keithley over GPIB, the second card, the electrochemical cell.

Simulate first. Scope second. Tip last.

30.5 The five safety rules, in one place

Rule	Enforced by
piezo commands clamp , loudly	clamp_piezo, every DC move and waveform
bias refuses , never clamps	check_bias, every waveform
no coarse step while the piezo is out	require_retracted — the interlock
no pull without room beneath it	check_pull_headroom, every pull
approach runaway budget	check_step_budget, every coarse step

Outputs park at 0 V on normal exit, on exception, and on Ctrl-C. On this unipolar piezo 0 V is fully retracted, so the safe state is also the state a dead card falls into by itself.

30.6 The one thing still unverified that can damage hardware

Whether **0–10 V** is the limit at ao0 or at the piezo *after* the driver box. If the box has gain, the DAQ-side limit is lower than 10 V and the config as shipped would over-drive the piezo.

Settle that before connecting the piezo. Everything else can be run today.